KH 정보교육원

네트워크 프로그래밍 구현

JAVA

목차

Part 1. 개발환경 분석하기

Chapter 1. 네트워크의 이해 4

Chapter 2. 네트워크 프로토콜의 이해 6

Chapter 3. 서버와 클라이언트의 이해 10

Part 2. 기능 구현하기

Chapter 4. 스래드 12

Chapter 5. InetAddress 36

Chapter 6. URL 44

Chapter 7. TCP 47

Chapter 8. UDP 59

Part 3. 프로그램 디버깅하기

Chapter 9. UNICAST CHATTING 67

Chapter 10. MULTICAST CHATTING 71

Chapter 11. BROADCAST CHATTING 85

Part 4. 프로그램 최적화하기

Chapter 12. 자바 GUI 91

Chapter 13. 이벤트처리 125

Part 1. 개발환경 분석하기

Chapter 1. 네트워크의 이해

1. 네트워크란?

네트워크란 무엇인가? 먼저 단어를 살펴보면 Net + Work 이다 Net 는 본래 뜻이 '그물'이고 Work 는 작업이므로 그대로 직역한하면 '그물일'이 될 것이다. 그러나 우리가 다루려는 네트워크는 정확히 말하면 'Computer Networking'으로서 컴퓨터를 이용한 '그물작업'이 될 것이다.

네트워크의 정의를 내린다면, '어떤 연결을 통해 컴퓨터의 자원을 공유하는 것'. IEEE(Institute of Electrical and Electronics Engineers: 국제 전기 전자 공학회)에서는 네트워크, 그중에서도 LAN 을 다음과 같이 정의하였다. "몇 개의 독립적인 장치가 적절한 영역내에서 적당히 빠른 속도의 물리적 통신 채널을 통하여 서로가 직접 통신할 수 있도록 지원해 주는 데이타 통신 체계" 이다.

2. 네트워크의 역사

2.1 전화망

네트워크의 정의를 확대해서 데이터 통신 즉 원거리간의 정보교환까지 거슬러 올라가면 '전화'가 네트워크의 시초가 된다. 또한 전화망은 Cicuit Switching Network 의 모델이기도 하다.

시간이 흘러 1960 년대 초반, 벨사는 디지털 전송망의 필요성을 느끼기 시작했다. 1940 년대에 발표된 TDM(Time Division Multiplexing)기술을 실용화 하여 T1 방식의 디지털 전송기술을 개발하였다. 이제 교환국과 가입자 사이에는 아나로그 신호가, 교환국과 교환국 사이에는 디지털 신호가 흐르게 되었다. 이후 디지털 교환기가 개발되어 현재형태의 전화망이 이뤄졌다.

2.2 소리의 세계에서 데이터의 세계로

'소리'가 아닌 '정보'의 전송은 전화보다 먼저 1846 년, 모스에 의한 전보가 그 시초이다. 이 전보기는 장거리간에 신호의 시작과 멈춤만을 전송할 수 있는 매우 간단한 구조이다. 이것은 수천년 전부터 인류가 사용해온 빛에 의한 정보의 전달과 같은 원리를 지닌다. 밤에 봉화대에 불을 피워 정보를 알린 것이다.

최초의 통신회사는 1856 년에 세워진 Western Union 이었다. 최초의 통신 프로토콜은 모스 부호(dot 와 dash 로 이뤄진)였다. 서비스의 내용은 '전보'였고 접속방법은 전보기 (tap set)이나 전보조작자(operator)였다. 여기서 서비스, 프로토콜, 접속 이 세가지 개념은 현대 네트워크에서도 동일하게 적용된다.

이 통신망의 최초 사용자는 군인들이었다. 남북전쟁의 북군이 이 전보를 십분 활용하였다. 현대 통신망의 기초도 2 차세계대전중 군의 요구에 의해 만들어진 것이다. 대전후에 군은 모든 지휘 통제 체계의 중앙

통제를 원했고 DOD(Department of Defense)통신망구조을 구축하였으며 이 통신망의 구조는 오늘날 사용되고 있는 많은 통신망의 근간을 이룬다.

DOD 의 큰 업적은 ARPANET(Advanced Research Project Agency NETwork)의 설립이다. ARPANET 은 1971 년에 세워진 최초의 패킷 네트워크이다. 1983 년 ARPANET 에서 군사부분은 MILNET 으로 분리되고 나머지 망은 DARPA Internet 으로 남아 오늘날의 Internet 이 되었다.

호스트 컴퓨터와 터미널로 이뤄지는 최초의 컴퓨터 네트워크는 IBM 의 SNA(System Network Architecture)망이다. 1960 년대 그 개념이 발표되고 1974 년, 최초의 SNA 제품이 출시되었다. 이 망은 계층구조를 가지는 main frame 과 지능형 호스트 밑으로 수많은 더미 터미널을 붙이는 방식을 취하였다. 이 구조에서 모든 '지능'은 호스트와 main frame 에 집중되며 어플리케이션도 그 위에서 작동한다. 값싼 터미널은 단지 입출력 기능만을 가질 뿐이다.

다음 세대 컴퓨터 네트워크의 주자는 1974 년 Xerox 에 의해 개발된 Ethernet 이다. Ethernet 은 네트워크 구조를 Host-Terminal 구조에서 오늘날과 같은 Client-Server 구조로의 전환하는데 큰 역할을 담당한다.

오늘날 네트워크는 WWW 의 발달로 LAN, WAN 의 개념이 무색하게 전 세계의 모든 컴퓨터가 하나로 묶여지고 있다. 서비스의 종류도 다양해 졌고 접속방법에서는 ATM 이 대역폭의 부족에 목말라 하는 사용자들의 갈증을 채워주기 위해 대기중이다. ATM 망이 집집까지 들어가는 것이 정보고속도로(Information Super HighWay)의 최종목표이다.

Chapter 2. 네트워크 프로토콜의 이해

1. TCP/IP

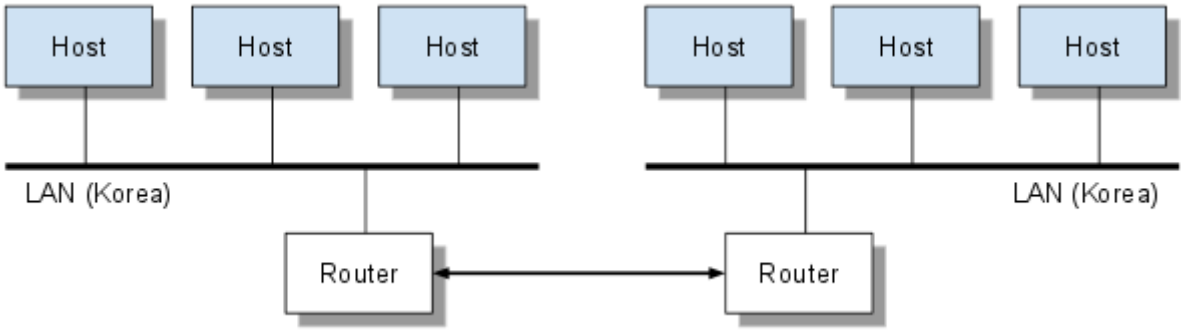
컴퓨터간 통신을 위해서 나타난 TCP/IP 는 가장 최근에 발명된 **컴퓨터와 컴퓨터**간의 지역네트워크(LAN) 혹은 광역네트워크(WAN)에서 원할한 통신을 가능하도록 하기 위한 통신규약(Protocol) 으로 정의할 수 있다.
최초 미국방성에서 구축한 **ARPANET** 에서 시작되었으며, 후에 미국 방위 통신청(DAC-Defense Communication Agency)에서 컴퓨터간 통신을 위해서 TCP/IP 를 사용하도록 한것이 그 시초가 되었다.
인터넷 서비스인 WWW, EMAIL, TELNET, FTP 등 대부분이 TCP/IP 기반에서 만들어져 있다. 인터넷으로 연결된 수많은 컴퓨터와 통신을 위해서 TCP/IP 를 선택한 이유는 그 개방성에 있다. 즉 하드웨어, 운영체제, 접속매체에 관계없이 동작할수 있다는 점 때문에, 인터넷 통신을 위한 핵심으로 선택되었다.
현재는 수천만대의 컴퓨터가 인터넷을 통하여 거미줄처럼 연결되어 있다.
TCP/IP 는 TCP 와 IP 의 2 개의 프로토콜로 이루어져 있는데, 통상 IP 프로토콜 위에 TCP 프로토콜이 놓이게 되므로 TCP/IP 라고 부르게 되었다.

- IP - node(단말기)와 node - 컴퓨터와 컴퓨터 - 간의 데이타 패킷을 전송하기 위해서는 각 node 에 (우편번호와 같은)주소를 필요로 한다. IP 는 4 바이트로 이루어진 주소번호를 사용하여서 각각의 node 를 구분하고, 목적지를 찾아가게 된다. 이를 IP Address 라고 하며, "192.168.100.100" 와 같은 점박이 3 형제 형식으로 사용한다. IP 는 **I**nternet **P**rotocol 의 줄임말이다. 숫자로된 인터넷주소를 사람이 식별하는건, 그리 쉽지 않기 때문에, IP 주소를 인간이 식별하기 쉬운 Domain 네임으로 변환시켜주는 Domain Name 서비스를 사용한다. www.yahoo.co.kr 이 211.234,109.2 보다는 이해하기가 수월 할것이다.
- TCP - 서버와 클라이언트간에 데이타를 신뢰성있게 전달하기 위해 만들어진 프로토콜이다. 데이타는 네트워크 선로를 통해 전달되는 과정에서 손실되거나 순서가 뒤바뀌어서 전달될 수 있는데, TCP 는 손실을 검색해 내서, 이를 교정하고 순서를 재조합할수 있도록 해준다. **T**ransmission **C**ontrol **P**rotocol 의 줄임말이다. 데이타를 전송하기 전에 데이타전송을 위한 연결을 만드는 연결지향 프로토콜이다.

2. LAN 과 WAN 그리고 TCP/IP

LAN 은 Local Area Network 의 줄임말로 지역내트웍(근거리 통신망) 를 말하며, WAN 은 Wide Area Network 의 줄임말로 광역네트웍(원거리 통신망) 이다.
즉 LAN 은 지역적으로 가까운 컴퓨터가 서로 연결(Network)된 상태를 말하며 WAN 은 지역적으로 멀리 떨어진 컴퓨터가 서로 연결된 상태를 말한다.

다음의 그림은 이러한 LAN 과 WAN 의 구성을 보여주는 그림이다.



HOST 와 HOST 그리고 HOST 와 Router 는 다양한 종류와 다양한 품질을 가지는 네트웍 연결 회선으로 연결될 수 있다. (광케이블, 구리선, 인공위성등...). 통화품질이 다르다는 것은 데이타를 보내는 중 데이타 손상이 일어날수도 있으며, 데이타의 순서가 뒤바뀔 수도 있다는 것을 의미한다.
데이타의 순서가 뒤바뀔 수 있는 이유는, 패킷이 전달되는데 하나의 고정된 전달 경로만을 이용하지 않고 임의의 경로를 사용하기 때문으로 각 경로를 구성하는 회선의 품질이 다르게 되면 먼저 보내어진 패킷이라도 나중에 보내어진 패킷보다 더 느리게 도착하는 문제가 생길 수 있다. 예를들어, 구리선으로 전달되는 패킷은 인공위성이나 광케이블로 전달된 패킷보다 느리게 움직일 것이다.

TCP 가 올바른 통신을 하도록 도와주는 기능을 가지고 있으며, IP 는 이러한 기능없이 오로지 TCP 패킷을 전송하는 일을 한다.

TCP 가 가지고 있는 기능은

1. 패킷이 빠졌을 경우, 재전송을 요청하는 기능
2. 패킷에 일련번호를 줌으로써, 서로 다르게 도착될지도 모르는 패킷의 순서를 재조합하는 기능

이다.

3. UDP 와 TCP

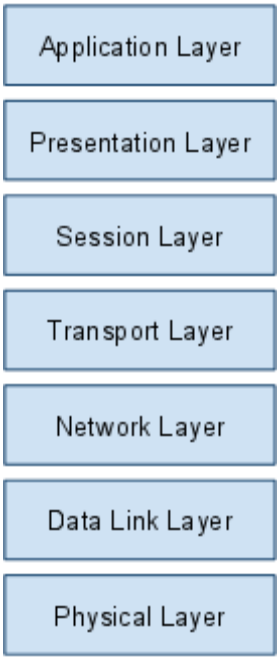
TCP 외에 UDP 라는 프로토콜 존재한다. 데이타를 전송한다는 점에서 TCP, UDP 모두 비슷하지만 특성에 있어서 약간의 차이가 있다.
UDP 는 **U**ser **D**atagram **P**rotocol 의 줄임말이다.

- TCP - 연결지향이며, 자체적으로 오류를 처리하며, 네트웍 전송중 순서가 뒤바뀐 메시지를 교정시켜주는 기능을 가지고 있다. 연결지향이란말은 데이타를 전송하는 측과 데이타를 전송받는 측에서 전용의 데이타 전송 선로(Session)을 만든다는 의미이다. 데이타의 신뢰도가 중요하다고 판단될때 주로 사용된다.
- UDP - 비연결지향이며, 오류를 처리하거나 순서를 재조합시켜주는 기능을 가지고 있지않다. 단순히 데이타를 받거나, 던져주기만 하는 프로토콜이다. UDP 는 특히 실시간 멀티미디어 정보를 처리하기 위해서 주로 사용한다.

TCP 를 실시간 멀티미디어 정보를 처리하는데, 사용할경우 TCP 의 오류정정 특성상 메시지가 도착하지 않거나 할경우 다음 메시지를 받지 않고, 메시지 재전송을 요구하므로, 실시간으로 전송하기에는 그리 적당하지 않기 때문이다. 반면 UDP 를 사용하면 중간에 패킷이 소실되더라도 개의치 않고 다음 패킷을 받아들이므로 실시간으로 메시지 처리가 가능하다.
물론 약간의 데이타 손실로 인해서 멀티미디어 데이타의 질이 떨어질수도 있으나, 화질이나 음질에 약간의 손상이 있더라도 계속적으로 서비스가 되는게 훨씬 더 유리할것이다. 전화를 하는데, 약간 잡음이 섞인다고 해서, 잡음을 정정하기 위해서 서로 통화를 못하는 사태가 발생하면 안되는것과 같은 이치이다.

4. 어떻게 TCP/IP 를 이용해서 컴퓨터간 데이타 통신이 가능한가 ?

지금까지 TCP/IP 그리고 UDP에 대한 개괄적인 내용을 살펴봤는데, 해결되지 않은 문제가 있다. 그렇다면, 각 컴퓨터간을 연결해주는 이더넷 카드를 통해서 어떻게 TCP/IP 메시지가 전달되는 것일까 ?
이것을 이해하기 위해서는 OSI7 에 대한 이해가 필요하다. OSI 는 각종 시스템간의 연결을 위하여 ISO 에서 제안한 모델로써, OSI(Open System Interconnection Reference Model)에서 유추할수 있듯이, 시스템에 상관없이 서로의 시스템이 연결될수 있도록 만들어주는 모델이다.
OSI 는 아래와 같이 7 개의 계층으로 되어 있다.

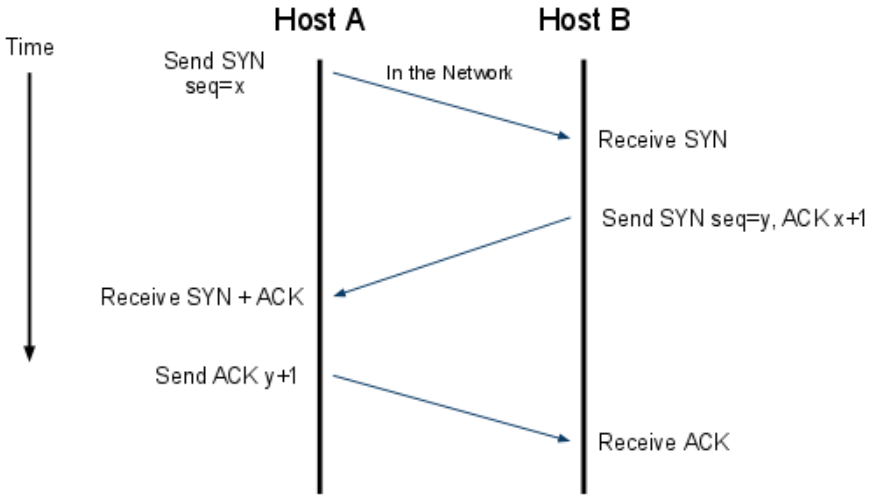


컴퓨터와 컴퓨터사이의 데이타전송을 위해서는 위의 7 개의 계층을 "직-간접적" 으로 거쳐서 전송이 되게 된다.
위와 같이 7 개의 계층으로 나눈 이유는, 각 계층에 대한 캡술화와 은닉이 가능하기 때문이다. 캡슐화, 은닉에 대해서는 아마도 C++ 을 공부해 본적이 있다면 많이 들어본 개념일건데, 예를들어 서비스 개발자는 Application Layer 와 Prsentation Layer 만 신경쓰면 된다. 실제 어플리케이션 개발자는 Session Layer 와 Transport Layer 정도만 신경쓰면 될것이다. Network Layer 계층 아래는 운영체제(:12)가 알아서 챙겨주므로 거의 신경쓸 필요가 없다. 마찬가지로 하드웨어를 만드는 사람은 Physical Layer 만 신경쓰면 되며, 그위의 계층에 대해서는 신경쓸필요가 없다.(물론 개발자가 Application Layer 와 Presentation Layer 까지 몽땅 신경써서 개발하는 경우도 있지만 - 사실은 거의 대부분이겠지만)

OSI 를 이렇듯 계층별로 나눔으로써, 각 계층에서 필요한 부분만을 개발자들이 신경쓰게 되고 통신서비스 개발시간을 줄일수 있도록 도와준다.

5. 연결지향 이란 무엇인가 ?

TCP 는 UDP 와 달리 연결지향이라고 배웠었다. 그렇다면 연결지향이란 무엇을 의미하는 것일까 ? 우리는 바로 위에서 TCP/IP 에 의해서 데이타가 어떻게 전송되어 지는지를 알아봤는데, 데이타가 전송되기전에, Browser 와 Server 간의 연결을 성립하는 과정이 데이타를 전송하는 과정 전에 이루어지게 된다. 연결을 만드는 과정은 이를테면 우리가 전화할 때 어떤 내용을 말하기에 앞서서, "안녕하세요?" "누구누구씨 맞아요" "네. 저 누구누구 맞습니다"라고 상대편을 먼저 확인하는 과정과 동일한 과정이다.



즉 데이타가 전송되기 전에, 먼저 Browser 는 Server 에 "서버 잘있읍니까?" 라고 메시지를 보내고, Server 는 다시 Browser(:12) 에게 "서버 준비되어 있으니, 데이타 보내시요" 라는 메시지를 보내고 Browser 는 다시 서버에게 "네, 그럼 지금부터 데이타를 보내겠습니다" 라고 서로의 존재를 확인하는 절차를 수행한 후, 정식 데이타를 교환하기 위한 통신 선로를 개설하게 된다. 통신선로를 하나 만들기 위해서는 3 번의 데이타 전송이 일어나게 되므로, 이것을 **three way handshake** 이라고 한다. 위 그림은 three way handshake 의 과정을 보여주고 있다.

이렇게 연결이 이루어지면 모든 정식 데이타는 연결된 통신 선로를 통해서 교환되게 되며, 이러한 이유로 TCP를 "연결 지향" 프로토콜이라고 부르는 것이다. UDP 는 이러한 과정이 없이 단순히 데이타만을 전송함으로 "데이타 그램" 프로토콜이라고 부른다.

Chapter 3. 서버와 클라이언트의 이해

1 서버와 클라이언트의 이해

서블릿을 공부하기에 앞서 기반이 되는 서버와 클라이언트가 무엇인지 먼저 알아보자.
서버(Server)와 클라이언트(Client)란 두 컴퓨터의 사이에 있어 서비스를 이용하는 사용자와 서비스를 제공하는 역할을 말하며, 사용하는 측이 클라이언트(Client), 제공하는 측을 서버(Server)라고 칭한다. 간단한 그림으로 이 관계를 표현해 보면 다음과 같다.



클라이언트(Client)가 웹 브라우저를 사용해 자신의 메일을 열람할 수 있는 페이지를 보고 싶어 해당 페이지에 접속한다고 하자. 그럼 웹 브라우저는 해당되는 요청문을 서버(Server)에 전달한다. 이 과정을 보통 **요청(Request)**이라고 한다. 서버(Server)에서는 요청 받은 프로세스를 처리하기 위해 일련의 로직을 구동하고, 해당 결과를 다시 웹 브라우저에 송신한다. 이것을 **응답(Response)**이라고 말한다. 웹 브라우저는 응답받은 정보를 일정 형식에 따라 화면에 표현하고, 클라이언트(Client)는 웹 브라우저를 통해 서비스를 요청한 결과를 확인할 수 있다.

이제까지 클라이언트(Client) 입장에서 웹 서비스를 이용해 왔다면, 앞으로는 서버(Server) 입장에서의 개발자로써, 웹 페이지를 만들고 서비스를 제공해야 한다. 이를 위해서 필요한 기술 중의 하나가 바로 서블릿이다.

erver의 종류

종류	설명
Web Server	웹 브라우저와 HTTP 프로토콜을 사용하여 사용자의 요구에 따른 특정 서비스를 제공하는 서버
Mail Server	인터넷을 통해 사용자 간의 전자 우편을 주고 받는 서비스를 제공한다
FTP Server	서버 내에 파일을 업로드, 다운로드 할 수 있도록 파일 관리 기능을 제공한다
Telnet Server	Terminal, 텍스트로만 이루어진 창에서 특정 명령어를 통해 원격지 서버를 접속, 관리한다
Database Server	Data를 저장하고, 원격지에서 접속할 경우 권한에 따라 해당 데이터를 열람, 추가, 수정, 삭제 기능을 처리한다

Part 2. 기능 구현하기

Chapter 4. 스레드

1 스레드

1.1 프로세스와 스레드

운영체제에서 실행중인 하나의 어플리케이션(프로그램)을 프로세스라고 부르는데, 스레드는 이러한 프로세스의 작업을 수행하는 단위이다

멀티 프로세스는 운영체제에서 할당받은 메모리를 가지고 독립적으로 실행하기 때문에 하나의 프로세스에서 오류가 발생해도 다른 프로세스에 직접적인 영향이 없다. 이와 반면에 둘 이상의 스레드를 가진 프로세스인 멀티스레드 프로세스는 하나의 프로세스 내부에서 스레드가 생성되기 때문에 하나의 스레드가 예외를 발생시킨다면 프로세스 자체가 종료 될 수 있다

1.2 메인 스레드

모든 자바 어플리케이션은 main() 메소드를 실행하면서 메인 스레드가 시작된다
필요에 의해 작업 스레드들을 만들어서 병렬로 코드를 실행할 수 있다(멀티 스레드를 이용한 멀티 태스킹)
멀티 스레드 어플리케이션 환경에서 실행 중인 스레드가 하나라도 있으면 프로세스가 종료되지 않는다

1.3 싱글 스레딩와 멀티 스레딩

싱글 스레딩 : 메인 스레드만을 가지고 작업을 처리하는 프로세스
멀티 스레딩 : 메인 스레드 외의 추가적인 스레드를 이용해 병렬적으로 작업을 처리하는 프로세스

하나의 자원을 사용하는 작업이 여러 개일 경우 싱글 스레드 프로세스가 멀티 스레드 프로세스보다 효율적일 수 있다. 하지만 별도의 자원을 각각 사용할 경우 멀티 스레딩 어플리케이션이 더 효율적이다.

1.4 멀티 스레딩의 장단점

- 멀티 스레딩의 장점
  CPU 이용률 향상
  효율적인 자원 활용
  수행 기능별로 분리하여 코드가 간결해짐
  어플리케이션의 응답성 향상

- 멀티 스레딩의 단점
  같은 프로세스의 자원을 공유하므로 동기화가 필요
  동기화 처리에 관한 이슈 처리 필요(교착상태, 기아상태 등)
  CPU 환경이나 OS의 스케줄러를 고려해야 함

2 스레드 생성과 실행

2.1 작업 스레드

멀티 스레딩 어플리케이션을 개발하려면 병렬로 실행할 작업이 몇 개인지 결정하고 작업 별로 스레드를 생성해야 한다

메인 스레드는 무조건 존재하므로 메인 작업 외의 병렬 작업 수만큼 작업 스레드를 생성한다

작업 스레드는 객체로 생성되므로 Thread 클래스를 이용하거나 Thread 클래스를 상속하여 하위 클래스로 생성하여 사용할 수 있다

2.2 Thread 클래스를 이용한 직접 생성

Runnable 타입 파라미터를 갖는 java.lang.Thread 클래스의 생성자를 호출해야 한다

Runnable은 인터페이스이기 때문에 구현 객체를 만들어 대입해야 한다

Runnable에는 작업 스레드가 실행할 코드를 작성할 수 있는 run() 메소드가 정의되어있다

- Thread 객체 생성

```
Thread thread = new Thread(Runnable r);
```

- Runnable 인터페이스를 구현한 작업 객체 만들기

```
class ThreadEx implements Runnable {
  public void run() {
    // 작업 코드
  }
}
```

- Runnable 인터페이스를 익명 구현하여 Thread 객체 생성

```
Thread thread = new Thread(new Runnable() {
  public void run() {
    // Code
  }
});
```

- 작업 스레드 실행

```
thread.start();
```

2.3 Thread 하위 클래스로 생성

작업 스레드가 실행할 Runnable을 따로 구현하지 않고 Thread 클래스를 상속받아 작업 스레드를 정의하면서 내용을 포함시킨다

Thread 클래스를 상속한 후 run() 메소드를 재정의하여 실행 코드를 작성

- Thread 클래스 상속

```
public void ThreadEx extends Thread {
   @Override
   public void run() {
      // 작업 코드
   }
}
```

2.4 스레드 이름

스레드는 각각 자신의 이름을 갖고 있다

디버깅할 때 어떤 스레드가 어떤 작업을 하는지 조사할 목적으로 사용된다

메인 스레드의 이름은 "main"

생성한 스레드는 자동적으로 "Thread-n" 형식

- 스레드 이름 직접 설정하기

```
Thread.setName("스레드이름");
```

- 스레드 이름 알아오기

```
Thread.getName();
```

test.thread1.ThreadA.java

```
01 package test.thread1;
02
03 public class ThreadA /*extends Thread*/ implements Runnable {
04         @Override
05         public void run() {
06                 // 다른 스레드 클래스와 동시 처리하고 싶은 내용을
07                 // 코드로 작성함
08                 for(int k = 1; k <= 500; k++) {
09                         System.out.println("★");
```

```
10                                  try {
11                                          Thread.sleep(100);
12                                  } catch (InterruptedException e) {
13                                          e.printStackTrace();
14                                  }
15                          }
16                  }
17  }
```

test.thread1.ThreadB.java

```
01  package test.thread1;
02
03  public class ThreadB /*extends Thread*/ implements Runnable{
04          @Override
05          public void run() {
06                  for(int k = 1; k <= 500; k++) {
07                          System.out.println(k);
08                          try {
09                                  Thread.sleep(100);
10                          } catch (InterruptedException e) {
11                                  e.printStackTrace();
12                          }
13                  }
14          }
15  }
```

test.thread1.TestThread.java

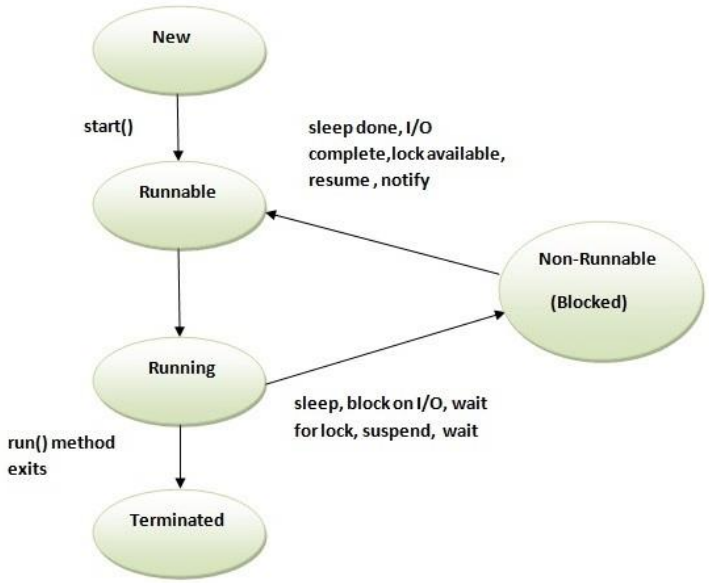
```
01  package test.thread1;
02
03  public class TestThread {
04          public static void main(String[] args) {
05                  // 멀티 스래드 테스트용
06                  /*A a = new A();
07                  B b = new B();
08
09                  Thread t1 = a;
10                  Thread t2 = b;*/
11
12                  /*Thread t1 = new A();
13                  Thread t2 = new B();*/
14
```

```
15                          //Runnable 인터페이스 상속시
16                          Thread t1 = new Thread(new A());
17                          Thread t2 = new Thread(new B());
18
19                          t2.setPriority(Thread.MAX_PRIORITY);
20
21                          t1.start();
22                          t2.start();
23              }
24  }
```

3 스레드 상태와 스케줄링

3.1 스레드 생명 주기

스레드 객체의 생성부터 종료까지를 스레드 생명 주기(Thread Life Cycle)이라고 부르며 5가지로 나눈다



상태	설명
NEW	스레드가 생성되고 아직 start()가 호출되지 않은 상태
RUNNALBE	실행 중 또는 실행 가능한 상태
BLOCKED	동기화블럭에 의해 일시정지된 상태
WAITING, TIMED_WAITING	스레드의 작업이 종료되지 않았지만 실행 가능하지 않은 일시정지 상태 TIMED_WAITING의 경우 일시 정지 시간이 지정된 경우
TERMIANTED	스레드의 작업이 종료된 상태

3.2 스레드 스케줄링

자바의 스레드 스케줄링은 우선 순위(Priority) 방식과 순환 할당(Round-Robin) 방식을 사용한다. 우선 순위 방식은 우선 순위가 높은 스레드가 실행 상태를 더 많이 가지도록 스케줄링을 하는 것이고, 순환 할당 방식은 시간 할당량을 정해서 스레드마다 정해진 시간만큼 작업시간을 주는 것이다.

우선 순위 스케줄링은 코드를 이용해 제어할 수 있고 순환 할당 스케줄링은 JVM에 의해 자동으로 정해진다.

3.3 스레드 우선 순위

Thread 클래스가 가지고 있는 priority 멤버변수를 이용해 우선 순위를 지정한다.

스레드 우선 순위는 int형 정수값 1~10의 값을 가지며 기본값은 5이다.

스레드의 우선 순위는 setPriority() 메소드를 이용해 지정할 수 있고, getPriority() 메소드를 이용해 확인할 수 있다.

단, 멀티코어 환경에서 OS마다 다른 방식으로 스케줄링을 하므로 반드시 자바 어플리케이션이 지정된 우선순위에 맞게 작업할 것이라고 기대할 수 없다

- Thread 클래스의 우선순위 상수

  public static final int MAX_PRIORITY = 10; // 최고 우선순위

  public static final int MIN_PRIORITY = 1; // 최저 우선순위

  public static final int NORM_PRIORITY = 5; // 기본 우선순위

test.thread2.PriorityTest.java

```
01  package test.thread2;
02
03  public class PriorityTest extends Thread {
04          //Field
05          private String threadName;  //스래드 이름 기억용
06
07          //Constructor
08          public PriorityTest(String threadName) {
09                  this.threadName = threadName;
10          }
11
12          //Method
13          @Override
14          public void run() {
15                  //스래드 이름 출력 3번 반복
16                  for(int i = 0; i < 3; i++) {
17                          System.out.println(threadName + "가 구동됨.");
18                  }
```

```
19              }
20
21              public static void main(String[] args) {
22                      // 우선 순위 테스트용
23                      Thread t1 = new PriorityTest("제일 높은 스래드");
24                      Thread t2 = new PriorityTest("보통 스래드");
25                      Thread t3 = new PriorityTest("제일 낮은 스래드");
26
27                      t1.setPriority(Thread.MAX_PRIORITY);   //10
28                      t2.setPriority(Thread.NORM_PRIORITY); //5
29                      t3.setPriority(Thread.MIN_PRIORITY);    //1
30
31                      t1.start();
32                      t2.start();
33                      t3.start();
34              }
35  }
```

4 스레드 상태 제어

4.1 스레드 일시 정지 - sleep()

sleep(long millis) - 밀리초 만큼 일시 정지

sleep(long millis, int nanos) - 밀리초+나노초 만큼 일시 정지

지정된 시간동안 스레드를 멈추게 한다

일시정지(BLOCKED) 상태가 된다

지정된 시간이 다 되거나 interrupt()가 호출되면 InterruptedException이 발생되어 실행대기 상태가 된다. sleep()을 호출 할 때는 항상 예외 처리를 해야 한다.

4.2 스레드 작업 취소 - interrupt()

void interrupt() - interrupted상태를 true로 변경

boolean isInterrupted() - interrupted상태를 반환

static boolean interrupted() - interrupted상태를 반환하고 false로 변경

진행 중인 스레드의 작업이 끝나기 전에 취소시킨다

interrupt() 메소드는 스레드의 작업을 일시정지시키며 종료시키지는 않는다

Thread 객체의 interrupted상태를 변경시킨다

4.3 스레드 실행 제어 - suspend(), resume(), stop()

suspend() - 스레드 일시 정지
resume() - suspend()로 일시 정지된 스레드를 실행 대기 상태로 변경
stop() - 스레드 종료

세 가지 메소드 모두 교착상태를 일으키기 쉬워 사용이 권장되지 않으며,
현재는 Deprecated 되었다

* Deprecated 메소드
에러는 아니지만 사용이 권장되지 않는 메소드로 하위 버전의 JDK와의 호환성을 위해 둔 메소드를 말한다.

4.4 다른 스레드에게 작업 양보 - yield()

yield() - 작업 시간 양보

스레드가 자신에게 주어진 작업 시간을 다음 차례의 스레드에게 양보한다.
yield() 메소드를 수행하면 실행 대기 상태가 된다
프로그램의 응답성을 높이고 효율적인 실행을 위해 사용한다.

4.5 다른 스레드의 종료 기다림 - join()

join() - 다른 스레드의 작업을 마칠 때까지 일시 정지
join(long millis) - 다른 스레드의 작업 종료 또는 밀리초 만큼 일시 정지
join(long millis, int nanos) - 다른 스레드의 작업 종료 또는 밀리+나노초 만큼 일시 정지

스레드가 작업을 잠시 멈추고 다른 스레드의 작업이 수행하도록 할 때 사용한다
interrupt()에 의해 대기 상태에서 벗어날 수 있다.
InterruptException이 발생하므로 try-catch문이 필요하다

5 데몬 스레드

일반 스레드의 작업을 돕는 보조 스레드
일반 스레드가 종료되면 데몬 스레드는 강제적으로 자동 종료
가비지 컬렉터가 데몬 스레드이다.
데몬으로 만드려는 스레드에 setDaemon()메소드를 호출하면 된다.

boolean isDaemon() - 스레드가 데몬인지 확인, true 면 데몬스레드
void setDaemon(boolean on) - 스레드를 데몬 스레드 또는 일반 스레드로 변경
setDaemon() 메소드는 start() 메소드 전에 호출되어야 한다.
start() 메소드 호출 이후 setDaemon() 메소드가 호출되면 IllegalThreadStateException이 발생

6 스레드 그룹

6.1 스레드 그룹

관련된 스레드를 묶어서 관리할 목적으로 그룹화시킨 것

자바 어플리케이션이 실행되면 JVM은 system 스레드 그룹을 만들고 JVM 운영에 필요한 스레드들을 생성해 포함시킨다. main스레드는 main스레드 그룹에 포함되어 system 스레드 그룹에 포함된다. 자바 어플리케이션에서 생성되는 모든 스레드는 main스레드 내에서 만들어지므로 main스레드 그룹에 자동으로 포함된다.

6.2 ThreadGruop 클래스

스레드를 그룹으로 묶어 관리하기 위한 클래스로 보안성이 향상된다

자신이 속한 스레드 그룹이나 하위 스레드 그룹을 변경할 수 있지만 다른 스레드 그룹의 스레드를 변경할 수는 없다

6.3 ThreadGroup의 생성자

생성자	설명
ThreadGroup(String name)	지정된 이름의 새로운 스레드 그룹 생성
ThreadGroup(ThreadGroup parent, String name)	지정된 이름의 새로운 스레드 그룹을 생성하고 부모 스레드 그룹을 지정

스레드를 스레드 그룹에 포함시키려면 Thread의 생성자를 이용한다

```
Thread(ThreadGroup group, String name)
Thread(ThreadGroup group, Runnable target)
Thread(ThreadGroup group, Runnable target, String name)
Thread(ThreadGroup group, Runnable target, String name, long stacksize)
```

스레드는 반드시 스레드 그룹에 포함되어야 하기 때문에 스레드 그룹을 지정하지 않은 생성자를 이용할 경우 기본적으로 자신을 생성한 스레드의 그룹에 포함된다

6.4 스레드 그룹 이름 얻기

스레드 그룹의 getName()함수를 이용

Thread 클래스의 정적 메소드인 getAllStackTraces()함수를 사용하면 프로세스 내에서 실행하는 모든 스레드에 대한 정보를 얻을 수 있다.

```
public static Map<Thread,StackTraceElement[]> getAllStackTraces()
```

6.5 스레드 그룹 일괄 interrupt()

스레드 그룹에서 제공하는 interrupt() 메소드를 이용하여 그룹 내의 모든 스레드에 일괄 interrupt을 발생시킬 수 있다. 개별 스레드에 interrupt()를 수행할 뿐 InterruptedException에 대한 예외 처리를 하지는 않는다.

test.thread3.CountDownTest.java

```
01  package test.thread3;
02
03  import java.awt.*;
04  import java.awt.event.*;
05
06  import javax.swing.JButton;
07  import javax.swing.JFrame;
08  import javax.swing.JLabel;
09  import javax.swing.JTextField;
10
11  public class CountDownTest extends JFrame {
12          //Field
13          private JLabel label;
14          private JButton startBtn;
15          private JTextField numberTF;
16
17          //내부클래스 : 클래스 안에 작성된 클래스
18          //Inner class / Nested class 라고 함
19          //내부클래스는 외부클래스의 멤버 개념이 적용이 됨
20          //내부클래스에는 static, private, protected 가 사용 가능함
21          private class CountThread extends Thread {
22                  @Override
23                  public void run() {
24                          int startNumber = Integer.parseInt(numberTF.getText());
25
26                          for(int count = startNumber; count >= 0; count--) {
27                                  try {
28                                          Thread.sleep(1000);
29                                  } catch (InterruptedException e) {
30
31                                          e.printStackTrace();
32                                  }
33
34                                  label.setText(String.valueOf(count));
```

```
35                                  }
36                          }
37                  }
38
39                  //Constructor
40                  public CountDownTest() {
41                          this.setTitle("카운트다운 프로그램");
42                          this.setSize(300, 200);
43                          this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
44
45                          label = new JLabel("시작");
46                          //label.setText("Start");
47                          label.setFont(new Font("Serif", Font.BOLD, 100));
48.                         label.setAlignmentX(CENTER_ALIGNMENT);
49                          this.add(label, BorderLayout.CENTER);
50
51                          startBtn = new JButton("카운트시작하기");
52                          this.add(startBtn, BorderLayout.SOUTH);
53
54                          //버튼에 이벤트 설정함 : 익명(Anonymus) 클래스 이용
55                          startBtn.addActionListener(new ActionListener() {
56                                  @Override
57                                  public void actionPerformed(ActionEvent e) {
58                                          //버튼을 클릭하면 스래드가 스타트되게 함
59                                          new CountThread().start();
60                                  }
61                          });
62
63                          numberTF = new JTextField("시작값을 입력하세요.");
64                          this.add(numberTF, BorderLayout.NORTH);
65
66                          this.setVisible(true);
67                  }
68
69                  public static void main(String[] args) {
70                          // 스래드를 이용한 카운트다운 테스트
71                          new CountDownTest();
72                  }
73          }
```

7 스레드 풀

7.1 스레드풀

병렬 작업이 많아지면서 증가된 스레드들의 생성과 스케줄링으로 인해 CPU 점유율 증가와 메모리 사용량 증가로 인해 어플리케이션의 성능이 저하된다. 이를 막기 위해 스레드 풀을 사용한다

작업 스레드의 개수를 제한하고 작업 큐에 들어오는 작업들을 하나씩 처리

java.util.concurrent 패키지의 ExecutorService 인터페이스와 Excutors 클래스를 사용

7.2 스레드풀 생성 및 종료

7.2.1 스레드풀 생성

newCachedThreadPool() 메소드를 이용하여 ExecutorService 객체를 생성하면 스레드 개수보다 작업 개수가 많으면 새 스레드를 생성하여 작업을 처리한다. 이론상 int타입의 최대값만큼 스레드가 추가되지만 OS의 성능이 감당할 수 있을 만큼만 생성된다. 추가된 스레드가 60초 동안 아무 작업을 하지 않으면 해당 스레드를 종료하고 스레드 풀에서 제거한다.

- 스레드 풀 생성

```
ExecutorService threadPool = Executors.newChachedThreadPool();
```

newFixedThreadPool(int nThreads) 메소드를 이용하여 ExecutorService 객체를 생성하면 스레드 개수보다 작업 개수가 많으면 새 스레드를 생성하여 작업을 처리한다. 최대 스레드 개수는 nThreads 만큼이며 스레드가 작업을 처리하고 있지 않아도 스레드를 종료하지 않는다.

- CPU 코어의 수만큼 최대 스레드를 사용하는 스레드 풀 생성

```
ExecutorService threadPool = Executors.newFixedThreadPool(
  Runtime.getRuntime().availableProcessors()
);
```

7.2.2 스레드 풀 종료

void shutdown() - 모든 작업을 처리한 뒤 스레드 풀을 종료

List<Runnable> shutdownNow() - 작업 중인 스레드를 interrupt해서 작업 중지 시도 후 스레드 풀 종료

boolean awaitTermination(long timeout, TimeUnit unit) - shutdown()호출 후 timeout 안에 완료하면 true 리턴, 완료하지 못하면 작업 중인 스레드에 interrupt하고 false 리턴

스레드 풀의 스레드는 데몬 스레드가 아니기 때문에 main스레드가 종료되더라도 실행 상태로 남게 된다. 때문에 프로세스가 종료되지 않게 된다.

스레드 풀을 종료시켜 스레드들이 종료 상태가 되도록 처리해야 한다.

남은 작업을 마무리하고 스레드풀을 종료할 때는 shutdown() 호출

남은 작업과 상관없이 강제로 종료할 때는 shutdownNow() 호출

7.3 작업 생성과 처리 요청

7.3.1 작업 생성

하나의 작업은 Runnable 또는 Callable 구현 클래스로 표현

Runnable은 작업 처리 완료 후 리턴 값이 없음

Callable은 작업 처리 완료 후 리턴 값이 있음

```
Runnable task1 = new Runnable() {
  @Override
  public void run() {
    // 작업 코드
  }
};
```

```
Callable<T> task2 = new Callable<T>() {
  @Override
  public T call() throws Exception {
    // 작업 코드
    return T;
  }
};
```

7.3.2 작업 처리 요청

ExecutorService의 작업 큐에 Runnable 또는 Callable 객체를 넣는 행위

- ExecutorService의 작업 처리 요청 메소드

메소드	설명
void execute(Runnable task)	task를 작업 큐에 저장
Future<?> submit(Runnable task) Future<V> submit(Runnable task, V result) Future<V> submit(Callable<V> task)	task를 작업 큐에 저장, Future를 통해 결과 리턴

execute()은 작업 처리 결과를 반환 받지 못하고 예외가 발생하면 스레드가 종료되고 스레드 풀에서 제거된다.

submit()은 작업 처리 결과를 Future 타입으로 리턴하고 예외가 발생하면 스레드는 종료되지 않고 다음 작업을 위해 재사용 된다.

7.4 블로킹 방식의 작업 완료 통보

submit() 메소드는 작업을 스레드 풀의 작업 큐에 저장하고 즉시 Future 객체를 리턴

반환된 Future 객체는 블로킹 되어있다가 작업이 완료되면 처리 결과를 리턴한다.

작업 완료 처리 결과는 get() 메소드를 통해 얻을 수 있지만 블로킹되어 있는 동안은 확인 불가 및 다른 작업 수행 불가

새로운 스레드를 이용하거나 다른 스레드 풀의 스레드를 이용하여 get() 호출

- Future 메소드

메소드	설명
V get()	작업 완료까지 블로킹되었다가 처리 결과 반환
V get(long time, TimeUnit unit)	time 전에 작업이 완료되면 결과 V 리턴, 그렇지 않으면 TimeoutException 발생
boolean cancel(Boolean running)	작업 처리가 진행 중일 경우 취소시킴
boolean isCancelled()	작업 취소 여부
boolean isDone()	작업 처리 완료 여부

7.4.1 리턴 값이 없는 작업 완료 통보

리턴 값이 없는 작업일 경우 Runnable 객체로 생성

작업 처리 요청은 submit(Runnable task) 메소드를 이용

결과 값이 없어도 Future 객체를 반환, 스레드의 작업 처리가 정상적인지 예외가 발생했는지 확인 가능

7.4.2 리턴 값이 있는 작업 완료 통보

처리 결과가 필요할 경우 작업 객체를 Callable로 생성

작업 처리 요청은 submit() 메소드를 이용

7.4.3 작업 처리 결과를 외부 객체에 저장

외부 객체에 작업 결과를 저장

외부 객체는 주로 공유 객체가 된다

두 개 이상의 스레드 작업을 취합할 목적으로 이용

7.4.4 작업 완료 순으로 통보

작업 요청 순서와 별개로 처리가 완료된 것만 통보를 받는 방법

CompletionService를 이용

- CompletionService의 처리 완료 작업 가져오는 메소드

메소드	설명
Future<V> poll()	완료된 작업의 Future 리턴, 없으면 즉시 null
Future<V> poll(long time, TimeUnit unit)	완료된 작업의 Future 리턴, 없으면 time만큼 블로킹
Future<V> take()	완료된 작업의 Future 리턴, 없으면 블로킹
Future<V> submit(Callable<V> task)	스레드 풀에 Callable 작업 처리 요청
Future<V> submit(Runnable task, V result)	스레드 풀에 Runnable 작업 처리

7.5 콜백 방식의 작업 완료 통보

콜백 방식이란 직접 메소드를 호출(Call)방식과 달리 이벤트의 발생에 대한 처리로써 메소드가 실행되는 방식을 뜻한다. 작업 처리를 요청한 후 스레드가 작업을 완료하면 특정 메소드를 자동 실행하도록 작업 완료 통보를 한다.

작업 처리 요청 후 결과를 기다릴 필요 없이 다른 기능을 수행할 수 있다.

- CompletionHandler를 이용한 콜백 객체 생성

```
CompletionHandler<V, A> c = new CompletionHandler< V, A >() {
    @Override
    public void completed(V result, A attachment) {
    }
    @Override
    public void failed(Throwable exc, A attachment) {
    }
}
```

completed() : 작업을 정상 처리 완료했을 때 호출되는 콜백 메소드

failed() : 작업 처리 도중 예외가 발생했을 때 호출되는 콜백 메소드

8 스레드 동기화

8.1 동기화

멀티 스레드 작업의 경우 여러 개의 스레드가 하나의 자원(객체)을 공유해서 작업하는 경우가 발생하기 때문에 서로의 작업에 영향을 주게 된다.

스레드가 사용 중인 자원을 다른 스레드가 변경할 수 없도록 하려면 스레드 작업이 끝날 때까지 객체에 잠금을 걸어 다른 스레드의 접근을 막는 것이다.

단 하나의 스레드만 실행할 수 있는 코드 영역을 임계 영역이라고 한다. 자바는 임계 영역에 대한 지원으로 동기화 메소드와 동기화 블록을 제공한다.

8.2 동기화 주의사항

8.2.1 공정성

여러 개의 스레드가 하나의 자원을 사용하기 위해 동시에 접근하는 프로그램을 작성할 경우 모든 스레드가 공정하게 자원에 접근할 수 있어야 한다.

8.2.2 기아상태

하나 또는 그 이상의 스레드가 자원에 접근이 필요하나, 우선순위에 밀려 BLOCKED 상태에 있으면 계속해서 그 자원에 접근하지 못하여 작업을 수행하지 못하는 상태가 될 수 있다.

8.2.3 교착상태

두 개 이상의 스레드가 각각 자원을 점유하고 서로의 자원에 대한 접근이 필요하게 되어 대기 상태가 될 수 있는데 서로 점유하고 있는 자원을 반납하지 않으면 대기 상태가 풀리지 않는다.

8.3 동기화 메소드 및 동기화 블록

동기화 메소드를 만드는 방법은 메소드 선언에 synchronized 키워드를 붙이는 것이다

메소드 전체 내용이 임계영역이다

메소드 실행 즉시 잠금이 일어나고 메소드가 종료되면 잠금 해제된다.

일부만 임계영역으로 지정 시 동기화 블록을 만든다.

- 동기화 메소드

```
public synchronized void sum() { ... }
```

- 동기화 블록

```
synchronized(객체 참조변수) { ... }
```

동기화 블록의 객체 참조변수는 락을 걸고자하는 객체를 참조하는 것이어야한다.

동기화 메소드, 동기화 블록 모두 락을 얻고 반납화는 과정이 자동이다.

임계 영역의 설정은 멀티 스레드 환경에서 성능에 영향이 많으므로 임계 영역이 넓은 동기화 메소드보다는 동기화 블록을 이용하는 것이 좋다.

test.synch.Account.java

```
01  package test.synch;
02
03  public class Account {
04          //Field
05          private static int balance;   //잔액
06          //입금스래드와 출금스래드가 함께 사용할 공유자원이 됨
07
08          //Method
09          public /*synchronized*/ void withDraw(int money) {
10                  try {
11                          Thread.sleep(100);
12                  } catch (InterruptedException e) {
13                          e.printStackTrace();
14                  }
15
16                  //출금하는 메소드
17                  synchronized(this) {
18                          if(balance >= money)
19                                  balance -= money;
20                  }
21          }
22
23          public static void setBalance(int balance) {
24                  Account.balance = balance;
25          }
26
27          public static int getBalance() {
28                  return balance;
29          }
30  }
```

test.synch.AccountManager.java

```
01  package test.synch;
02
03  public class AccountManager extends Thread {
04          private Account ac;
05          private String threadName;
06
07          public AccountManager() {}
```

```
08
09          public AccountManager(String threadName) {
10                  this.threadName = threadName;
11          }
12
13          @Override
14          public void run() {
15                  ac = new Account();
16                  ac.setBalance(1000);
17
18                  while(ac.getBalance() > 0) {
19                          System.out.println("현재 잔액 : " + ac.getBalance());
20                          //출금할 돈을 랜덤으로 처리함 : 백원 단위로 함
21                          int money = new java.util.Random().nextInt(10) * 100;
22                          System.out.println(threadName + "이/가 출금할 돈 : "
23                                          + money);
24                          ac.withDraw(money);
25                  }
26
27          }
28  }
```

test.synch.TestSynchronized.java

```
01  package test.synch;
02
03  public class TestSynchronized {
04
05          public static void main(String[] args) {
06                  // 동기화처리 테스트
07                  Thread t1 = new AccountManager("자동화기기");
08                  Thread t2 = new AccountManager("인터넷뱅킹");
09                  Thread t3 = new AccountManager("모바일앱");
10
11                  t1.start();
12                  t2.start();
13                  t3.start();
14          }
15  }
```

8.4 wait(), notify(), notifyAll()

wait() - 락을 반납하고 통지를 대기

wait(long millis) - 락을 반납하고 밀리초만큼 대기

wait(long millis, int nanos) - 락을 반납하고 밀리초+나노초만큼 대기

notify() - 객체 대기 풀의 임의의 스레드에 통지

notifyAll() - 객체 대기 풀의 모든 스레드에 통지

동기화 처리에서는 특정 스레드가 자원에 대하여 락을 가진 상태로 오랜 시간을 보내지 않도록 하는 것이 중요하다.

작업을 더 이상 진행할 상황이 아니라면 wait()을 호출하여 락을 반납하고 대기 풀에서 기다린다. 자신의 작업을 완료하면 notify() 메소드를 호출해 대기 풀에서 통지를 기다리고 있는 상대방 메소드를 일시 정지 상태에서 풀어준다. 대기 풀은 객체마다 존재한다.

notify()가 호출되면 대기 상태의 스레드 중 임의의 스레드만 통지를 받는다. notifyAll()이 호출되면 대기 상태의 모든 스레드에 통지를 하지만 락을 얻는 스레드는 우선순위가 높은 하나뿐이다.

test.common.Buffer.java

```
01  package test.common;
02
03  public class Buffer {
04          private int data;
05          private boolean empty = true;
06
07          public synchronized int get() {
08                  while (empty) {
09                          try {
10                                  wait();
11                          } catch (InterruptedException e) {
12                          }
13                  }
14                  empty = true;
15                  notifyAll();
16                  return data;
17          }
18
19          public synchronized void put(int data) {
20                  while (!empty) {
21                          try {
```

```
22                                      wait();
23                              } catch (InterruptedException e) {
24                              }
25                      }
26                      empty = false;
27                      this.data = data;
28                      notifyAll();
29              }
30      }
```

test.synch.Producer.java

```
01  package test.synch;
02
03  import test.common.Buffer;
04
05  public class Producer implements Runnable {
06          private Buffer buffer;
07
08          public Producer(Buffer buffer) {
09                  this.buffer = buffer;
10          }
11
12          public void run() {
13                  for (int i = 0; i < 10; i++) {
14                          buffer.put(i);
15                          System.out.println("생산자: " + i + "번 케익을 생산하였습니다.");
16                          try {
17                                  Thread.sleep((int) (Math.random() * 100));
18                          } catch (InterruptedException e) {
19                          }
20                  }
21          }
22  }
```

test.synch.Consumer.java

```
01  package test.synch;
02
03  import test.common.Buffer;
04
05  public class Consumer implements Runnable {
06          private Buffer buffer;
```

```
07
08              public Consumer(Buffer drop) {
09                      this.buffer = drop;
10              }
11
12              public void run() {
13                      for (int i = 0; i < 10; i++) {
14                              int data = buffer.get();
15                              System.out.println("소비자: " + data + "번 케익을 소비하였습니다.");
16                              try {
17                                      Thread.sleep((int) (Math.random() * 100));
18                              } catch (InterruptedException e) {
19                              }
20                      }
21              }
22  }
```

test.run.ProducerConsumerTest.java

```
01  package test.run;
02
03  import test.synch.*;
04  import test.common.Buffer;
05
06  public class ProducerConsumerTest {
07          public static void main(String[] args) {
08                  Buffer buffer = new Buffer();
09                  (new Thread(new Producer(buffer))).start();
10                  (new Thread(new Consumer(buffer))).start();
11          }
12  }
```

```
생산자: 0번 케익을 생산하였습니다.
소비자: 0번 케익을 소비하였습니다.
생산자: 1번 케익을 생산하였습니다.
소비자: 1번 케익을 소비하였습니다.
생산자: 2번 케익을 생산하였습니다.
소비자: 2번 케익을 소비하였습니다.
생산자: 3번 케익을 생산하였습니다.
소비자: 3번 케익을 소비하였습니다.
생산자: 4번 케익을 생산하였습니다.
소비자: 4번 케익을 소비하였습니다.
생산자: 5번 케익을 생산하였습니다.
소비자: 5번 케익을 소비하였습니다.
생산자: 6번 케익을 생산하였습니다.
```

```
소비자: 6번 케익을 소비하였습니다.
생산자: 7번 케익을 생산하였습니다.
소비자: 7번 케익을 소비하였습니다.
생산자: 8번 케익을 생산하였습니다.
소비자: 8번 케익을 소비하였습니다.
생산자: 9번 케익을 생산하였습니다.
소비자: 9번 케익을 소비하였습니다.
```

동기화 예제 2

test.synch2.SharedData.java

```
01  package test.synch2;
02
03  public class SharedData {
04          private int number = 12;
05          private volatile boolean isChanged;  //휘발성
06          //최적화(비휘발성) : 메모리에 기록된 값을 계속 가져다 사용하는 것
07
08          public SharedData() {}
09
10          public int getNumber() {
11                  return number;
12          }
13
14          public void setNumber(int number) {
15                  this.number = number;
16          }
17
18          public boolean isChanged() {
19                  return isChanged;
20          }
21
22          public void setChanged(boolean isChanged) {
23                  this.isChanged = isChanged;
24          }
25
26          //동기화 처리된 메소드
27          public synchronized void calcNumber() {
28                  number *= 3;
29                  System.out.println("changed....");
30          }
31
```

```
32          public synchronized void printNumber() {
33                  System.out.println("number : " + number);
34          }
35  }
```

test.synch2.SynchChangeNumber.java

```
01  package test.synch2;
02
03  public class SynchChangeNumber extends Thread {
04          private SharedData sdata;
05
06          public SynchChangeNumber(SharedData sdata) {
07                  this.sdata = sdata;
08          }
09
10          @Override
11          public void run() {
12                  for(int k = 1; k <= 5; k++) {
13                          sdata.calcNumber();
14
15                          //해당 스래드가 공유자원 사용이 끝난 상태라면
16                          if(this.getState() == Thread.State.TERMINATED) {
17                                  //다른 스래드한테 신호를 보냄
18                                  this.notify();
19                          }
20                  }
21          }
22  }
```

test.synch2.SynchReadNumber.java

```
01  package test.synch2;
02
03  public class SynchReadNumber extends Thread {
04          private SharedData sdata;
05
06          public SynchReadNumber(SharedData sdata) {
07                  this.sdata = sdata;
08          }
09
10          @Override
11          public void run() {
```

```
12                      for(int n = 1; n <= 5; n++) {
13                              try {
14                                      //다른 스래드의 동기화 사용이 끝나기를 기다리는 상태이면
15                                      if(this.getState() == Thread.State.BLOCKED) {
16                                              //다른 스래드가 신호를 보낼 때까지 기다려라.
17                                              this.wait();
18                                      }
19                              } catch (Exception e) {
20                                      e.printStackTrace();
21                              }
22
23                              sdata.printNumber();
24                      }
25              }
26      }
```

test.synch2.TestSynchronized.java

```
01  package test.synch2;
02
03  public class TestSynchronized {
04          public static void main(String[] args) {
05                  // 공유영역 동기화 오류 해결 테스트
06                  SharedData shareObj = new SharedData();
07
08                  Thread changeThread = new SynchChangeNumber(shareObj);
09                  Thread readThread = new SynchReadNumber(shareObj);
10
11                  changeThread.start();
12                  readThread.start();
13          }
14  }
```

Chapter 5. InetAddress

1 네트워크

1.1 클라이언트와 서버

서버는 클라이언트에게 네트워크를 통해 서비스를 제공하는 시스템을 말하며 클라이언트는 서비스를 사용하는 사용자(시스템)를 뜻한다. 브라우저를 통해 특정 웹에 접속을 하는 경우 브라우저를 사용하고 있는 사용자와 해당 시스템이 클라이언트가 되고 웹 서비스를 제공하는 측이 서버가 된다.

1.2 OSI 7 계층 모델

프로토콜을 기능별로 나눈 것

네트워크 통신 장비 업체 간의 표준을 맞춰 호환성을 증대시키기 위해 ISO단체에서 1984년에 발표되었다. 모든 시스템이 상호 연결에 문제가 없도록 표준을 만들어 7계층으로 구분하였다. 표준화에 따른 비용절감 효과와 통신 과정에 대한 정립으로 네트워크 교육에 용이하도록 하기 위한 목적이 있다.

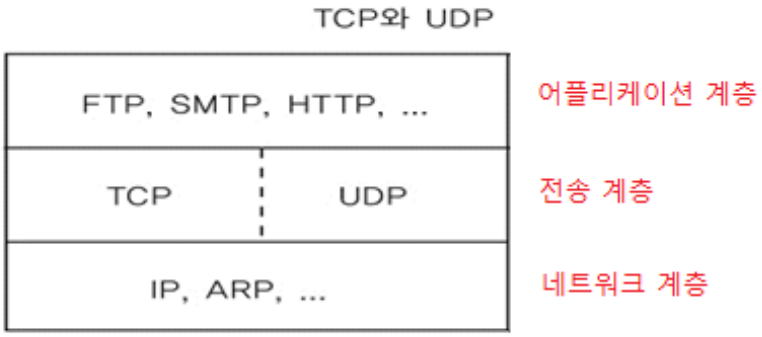
각 계층은 하위 계층의 기능만을 이용하고 상위 계층에게 기능을 제공한다.

7계층	어플리케이션 계층	사용자나 응용프로그램간의 데이터 교환이 가능하게 하는 계층이다. HTTP, FTP, 터미널 서비스, 메일 프로그램, 디렉토리 서비스 등을 제공
6계층	표현 계층	데이터의 구조를 하나의 통일 된 형식으로 설정하고, 데이터의 압축과 암호화 기능을 제공
5계층	세션 계층	두 시스템간의 통신 중 동기화를 유지하고 데이터 교환을 관리 한다. 정보 교환을 효과적으로 할 수 있도록 전송 계층에서 설정된 종단간의 논리적인 연결에 추가 서비스를 제공
4계층	전송 계층	종단간에 신뢰성 있고, 투명한 데이터 전송을 제공한다. 이를 위해 에러 제어, 통신량 제어, 다중화를 제공하며 응용프로그램간 통신을 위해 포트를 사용
3계층	네트워크 계층	IP주소 체계를 사용하며 경로제어와 통신량 제어등을 수행
2계층	데이터링크 계층	물리적인 링크를 통하여 동기화, 에러 제어, 흐름 제어 등을 통해 패킷을 전송한다. 12개의 16진수로 만들어진 MAC주소를 개체간 통신에 사용하며 MAC주소는 고유한 주소이다
1계층	물리계층	기계적, 전기적, 기능적, 절차적 특성을 정의하여 비트 스트림으로 물리적 매체를 통해 전송

1.3 TCP/IP 4 계층 모델

인터넷 프로토콜, 인터넷 모델이라고 부르며 대다수의 프로그램은 TCP 와 IP 프로토콜을 이용해 통신하고 있다.

TCP/IP 프로토콜을 이용해 연결하는 네트워크 체계를 이더넷(Ethernet)이라고 부른다



4계층	응용 계층	OSI 7계층의 세션 계층, 표현 계층, 응용 계층에 해당 TCP/UDP 기반의 응용 프로그램을 구현할 때 사용 FTP, HTTP, SSH 프로토콜이 있다
3계층	전송 계층	OSI 7계층의 전송 계층에 해당 노드 간의 연결을 제어하고, 신뢰성 있는 데이터 전송을 담당 TCP, UDP 프로토콜이 있다
2계층	인터넷 계층	OSI 7계층의 네트워크 계층에 해당 노드 간의 IP패킷을 전송하는 기능과 라우팅 기능을 담당 IP, ARP, RARP 프로토콜이 있다
1계층	네트워크 액세스 계층	OSI 7계층의 물리계층과 데이터 링크 계층에 해당 물리적인 주소로 MAC을 사용한다

1.4 IP Address

네트워크의 모든 Host는 Internet Address(Host 또는 IP Address)라 불리는 유일한 32bit 숫자로 구성된 Address체계를 이용하여 서로를 구분할 수 있다. 32bit 숫자를 한번에 모두를 표현하는 것이 힘들기 때문에 8 bit씩 끊어서 표현하고, 각 자리는 1byte로 0~255 까지의 범위를 갖게 된다. 32bit의 주소체계를 IP Version 4(IPv4) Address라고 한다.

모바일 디바이스 등 여러 장치들이 등장하면서 IP주소가 부족해지는 수준에 도달하여 IPv4가 포화 상태가 되자 이를 극복하고자 128bit를 이용한 주소체계인 IP Version 6(IPv6)가 나왔다. IPv6는 16bit씩 8부분으로 나누어 16진수를 사용하여 표시한다.

- IP주소 표시의 예시

IPv4	192.168.100.121
IPv6	FECD:BA98:7654:3210:FECD:BA98:7652:3210

IP 구조

IP Address 는 32bit(4byte) 길이로 구성된 논리적인 주소체계로서 형태는 OOO.OOO.OOO.OOO (ex. 184.51.65.127)로 표기한다. 여기서 **'.(dot)'으로 구분된 Octet(8bit / 1byte)** 4 개가 조합되어 IP 주소를 나타나게 된다. 그런데 실제 IP 는 2 진수로 표기되어 00000000.00000000.00000000.00000000 이와같은 형태로 구분되어 사람이 이해하고 외우기 어렵기에 10 진수로 나타내는 표기법을 사용하는 것이다. IP 주소의 이러한 표기법은 dot-decimal notice 또는 dotted-quad sequence 라고 부릅니다.

1 0 1 0 1 1 0 0 (8bit)	1 0 1 0 1 0 0 0 (8bit)	0 0 0 0 1 0 1 0 (8bit)	0 0 0 0 0 0 0 1 (8bit)
172	168	10	1
Octet 1	Octet 2	Octet 3	Octet 4

만약 2 진수를 계산할 줄 안다면, 8bit 가 전부 '1'이라고 가정하였을 때 255 라는 숫자가 나온다는 것을 알 수 있다. 따라서 각 옥탯 별로 **IP 는 0~255 까지의 범위**를 가지게 되며, 부여할 수 있는 **IP 의 개수는 256 개**가 되는 것이다. 그리고 전체 IP 의 수는 4,294,967,296 개로 약 42 억개 정도로 한정되어 있다.

물리적인 주소 체계

앞서 IP Address 가 논리적 주소체계도 존재하지만, 반대로 물리적 주소체계도 존재하며, 흔히 MAC (Media Access Control) 주소라고 말한다. 다른 이름으로는 Physical Address 라고도 한다.

이러한 MAC address 는 LAN(Local Area Network) 또는 Ethernet 이라 불리는 망에서 통신을 하기 위하여 사용된다. LAN 이라는 이름에서 알 수 있듯이 MAC 은 자신이 속한 네트워크 안에서만 통신이 된다. 이후 네트워크를 빠져나가는 장치인 Router 를 지나게 되면 IP 를 이용하여 통신하게 된다.

IP 주소의 NetworkID 와 HostID

하나의 IP 주소에는 Network ID 와 Host ID 가 존재하고 있다. 먼저 Network ID 는 인터넷 상에서 모든 Host 들을 전부 관리하기 힘들기에 한 Network 의 범위를 지정하여 관리하기 쉽게 만들어 낸 것이다. 그리고 Host ID 는 호스트들을 개별적으로 관리하기 위해 사용하게 된 것이다. 따라서 우리가 인터넷을 사용할 때 Routing 으로 목적지를 알아내고 찾아가는 등의 역할을 할 때에는 NetworkID 와 HostID 가 합쳐진 IP 주소를 보게 된다.

Subnet mask 를 활용하여 Network ID 를 올리거나 낮출 수 있게 된다. 반대로 Host ID 는 줄어들거나 늘어날 수 있게 된다. 라우터끼리의 통신에서는 IP 를 사용하기 때문에 Network ID 와 Host ID 보고 목적지가 어떤 네트워크에 속하는지 알 수 있게 된다.

IP Class 개념

Network ID 와 Host ID 를 설명한 이유는 바로 IP Class 의 개념을 알기 위함이다. **IP Class 의 경우 A, B, C, D, E Class 로 나누어 Network ID 와 Host ID 를 구분**하게 된다.

A Class 의 경우 처음 8bit(1byte)가 Network ID 이며, 나머지 24bit(3byte)가 Host ID 로 사용된다. 비트가 0 으로 시작하기에 네트워크 할당은 0~127 이다 . 즉, 128 곳에 사용 가능하며, 최대 호스트 수는 16,777,214 개이다.

B Class 의 경우 처음 16bit(2byte)가 Network ID 이며, 나머지 16bit(2byte)가 Host ID 로 사용된다. 비트가 10 으로 시작하기에 네트워크 할당은 16,384 곳에 가능하며, 최대 호스트 수는 65,534 개이다.

C Class 의 경우 처음 24bit(3byte)가 Network ID 이며, 나머지 8bit(1byte)가 Host ID 로 사용된다. 비트가 110 으로 시작하므로 네트워크 할당은 2,097,152 곳에 사용 가능하며, 최대 호스트 수는 254 개이다.

A Class

116	81	97	8
Network ID	Host ID	Host ID	Host ID

B Class

171	47	154	1
Network ID	Network ID	Host ID	Host ID

C Class

214	175		51
Network ID	Network ID	Network ID	Host ID

실제 Network 에서 사용되는 Class 는 A, B, C Class 이며, **D Class 는 Multicast(멀티캐스트), E Class 는 미래에 사용**하기 위해 남겨둔 것으로 예약되어 있다. 그리고 D 와 E Class 의 경우 실제 사용되는 경우가 거의 없다.

1.5 Port

컴퓨터의 주변장치를 접속하기 위한 물리적인 장치로써의 포트가 아닌 프로그램에서 사용되는 논리적인 장치로써의 접속 장소인 포트를 뜻한다

포트는 번호로 표현하며 Internet번호 할당 허가 위원회(IANA)에 의해 예약된 Port번호를 가지고 있다. 이러한 포트는 Well-known 포트(잘 알려진 포트)라고 부르며 대표적으로 80(HTTP), 21(FTP), 22(SSH), 23(TELNET)등이 있다.

포트 번호는 0~65535까지 이며, 0~1023까지는 잘 알려진 포트로 미리 서비스에 예약되어 있으므로 그 외의 포트번호를 사용할 것을 권장한다

프로토콜이 TCP인지 UDP인지에 따라 잘 알려진 포트가 달라질 수 있으므로 해당 서비스에 대한 포트를 알려면 TCP, UDP를 구분해서 확인해야 한다

1.6 Protocol

서버-클라이언트 간의 접속이나 통신방식, 전송 데이터 형식 등을 정의한 통신 규약이다

Internet 표준 Protocol에는 TCP와 UDP가 있다.

1.7 TCP(Transmission Control Protocol)

연결형 서비스를 지원하는 TCP/IP 전송계층 프로토콜

인터넷 환경에서 기본으로 사용하며 호스트 간 신뢰성 있는 데이터 전달과 흐름제어 및 혼잡제어 등을 제공하는 프로토콜이다. TCP 프로토콜 통신에 사용되는 데이터 조각인 패킷을 추적하고 관리하며 정상적으로 제대로 전달되지 않은 패킷에 대해서 재전송 처리를 하여 높은 신뢰성을 보장한다.

대부분의 응용 프로그램은 신뢰성과 다양한 서비스를 필요로 하기 때문에 주로 TCP 프로토콜을 사용한다. 대표적으로 HTTP, FTP, SMTP 가 이에 해당한다

1.8 UDP(User Datagram Protocol)

비연결형 서비스를 지원하는 TCP/IP 전송계층 프로토콜

인터넷상에서 서로 정보를 주고 받을 때 송신 신호나 수신 신호를 주고 받지 않고 일방적으로 데이터를 전달하는 프로토콜이다. UDP 프로토콜 통신에서 데이터 조각을 데이터그램이라고 부르며 데이터그램이 제대로 전달되지 않더라도 알 방법이 없다. 호스트 간 데이터 전달에 있어 데이터 일부가 손실되더라도 빠른 전송 속도가 필요한 비디오, 오디오등과 같은 서비스를 처리할 때 주로 사용되는 프로토콜이다.

2 네트워크 관련 클래스

2.1 InetAddress 클래스

인터넷 주소(IP)에 관한 정보를 다루는 클래스

생성자가 따로 없으며 InetAddress클래스의 정적 메소드에서 정보를 받아오는 형식으로 사용한다

InetAddress 클래스의 생성자

InetAddress 클래스의 생성자는 하나만 존재하지만, 특이하게 기본 생성자의 접근 제한자 default 이기 때문에 new 연산자 객체를 생성할 수 없다. 따라서 InetAddress 클래스는 객체를 생성해 줄 수 있는 5 개의 static 메서드를 제공하고 있다.

2.1.1 InetAddress 클래스의 주요 메소드

리턴 타입	메소드	설명
byte[]	getAddress()	IP주소를 byte배열로 반환
static InetAddress[]	getAllByName(String host)	도메인명(host)에 지정된 모든 호스트의 IP주소를 배열로 반환
static InetAddress	getByAddress(byte[] addr)	byte 배열을 통해 IP주소를 얻음
static InetAddress	getByName(String host)	도메인명(host)을 통해 IP주소 얻음
String	getHostAddress()	호스트의 IP주소를 반환
String	getHostName()	호스트의 이름을 반환
static InetAddress	getLocalHost()	지역호스트의 IP주소 반환
boolean	isMulticatAddress()	멀티캐스트 주소인지 확인
boolean	isLoopbackAddress()	loopback 주소인지 확인

- InetAddress 클래스 예제

test.inet.InetAddressEx.java

```
01  package test.inet;
02
03  import java.net.InetAddress;
04  import java.net.UnknownHostException;
05  import java.util.Arrays;
06
07  public class InetAddressEx {
08      public void testInetAddress(String domain) {
09          InetAddress ip = null;
10
11          try {
12              ip = InetAddress.getByName(domain);
13
14              System.out.println("HostName : " + ip.getHostName());
15              System.out.println("HostAddress : " + ip.getHostAddress());
16              System.out.println("HostName/HostAddress : ₩n₩t" + ip.toString());
17
18              System.out.println("----------------------------------");
19              byte[] ipAddr = ip.getAddress();
20              System.out.println("getAddress() : " + Arrays.toString(ipAddr));
21
22              System.out.print("HostAddress : ");
23              for(int i=0; i< ipAddr.length; i++) {
24                  if(ipAddr[i] < 0)       System.out.print(ipAddr[i]+256);
25                  else            System.out.print(ipAddr[i]);
26                  System.out.print(".");
27              }
28              System.out.println("");
29
30              System.out.println("----------------------------------");
31              ip = InetAddress.getLocalHost();
32              System.out.println("LocalHostName : " + ip.getHostName());
33              System.out.println("LocalHostAddress() : " + ip.getHostAddress());
34              System.out.println("Local HostName/HostAddress : ₩n₩t" + ip.toString());
35          } catch (UnknownHostException e) {
36              e.printStackTrace();
37          }
```

```
38        }
39    }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.inet.InetAddressEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new InetAddressEx().testInetAddress("www.oracle.com");
08      }
09  }
```

test.inet.TestINetAddress.java

```
01      package test.inet;
02
03      import java.net.*;
04
05      public class TestINetAddress {
06
07          public static void main(String[] args) throws UnknownHostException {
08              // InetAddress test
09              InetAddress ip1 = InetAddress.getLocalHost();
10              System.out.println(ip1.getHostAddress());
11              System.out.println(ip1.getHostName());
12
13              InetAddress ip2 = InetAddress.getByName("www.naver.com");
14              InetAddress[] ipes = ip2.getAllByName("www.google.com");
15
16              System.out.println("naver : " + ip2.getHostAddress());
17              System.out.println("google 의 서버는 " + ipes.length + "개 있습니다.");
18              for(InetAddress ip : ipes)
19                  System.out.println(ip.getHostAddress());
20
21              InetAddress[] ipes2 = InetAddress.getAllByName("www.naver.com");
22              System.out.println("naver 의 서버 ip는" + ipes2.length + "개 있습니다.");
23              for(InetAddress ip : ipes2)
24                  System.out.println(ip.getHostAddress());
```

```
25          }
26
27      }
```

test.inet.TestINetAddress2.java

```
01      package test.inet;
02
03      import java.util.*;
04      import java.net.*;
05
06      public class TestInetAddress2 {
07
08          public static void main(String[] args) throws UnknownHostException {
09              // 키보드로 호스트명을 입력받아, 해당 호스트가 가진 ip 주소 모두를 출력 처리
10              do{
11                  System.out.print("호스트명 (종료: none) : ");
12                  String host = new Scanner(System.in).next();
13
14                  if(host.equals("none") != true){
15                      InetAddress[] ips = InetAddress.getAllByName(host);
16                      System.out.println(host + "가 가진 ip는 " + ips.length + "개 입니다.");
17                      for(InetAddress ip : ips)
18                          System.out.println(ip.getHostAddress());
19
20                  }else
21                      break;
22              }while(true);
23          }
24      }
```

Chapter 6. URL

1 URL(Uniform Resource Locater)

URL이란 인터넷의 접근 가능한 자원의 주소를 표현하는 형식
인터넷에 있는 정보의 위치를 표현하는 체계

1.1 URL 형식

프로토콜, 인터넷 주소, 포트 번호, 파일 경로 등을 사용해 표현한다

```
프로토콜://인터넷주소[:포트번호]]/디렉토리[/파일이름]][?쿼리스트링]
```

- 프로토콜 : 저원에 접근하기 위해 서버와 통신에 사용되는 통신규약
- 인터넷주소 : 자원을 제공하는 서버의 주소(이름)
- 포트번호 : 통신에 사용되는 서버의 포트번호
- 디렉토리, 파일이름 : 자원이 저장되어 있는 서버상의 위치
- 쿼리스트링 : '?' 문자 뒤에 오는 통신에서 사용되는 매개변수

1.2 URL 클래스 생성자

생성자	설명
URL(String protocol, String host, int port, String file)	protocol : 프로토콜 host : 컴퓨터 이름 port : 포트 번호 file : ULR에 존재하는 자원(파일)의 이름 urlString : 모든 요소를 포함한 문자열
URL(String protocol, String host, String file)	
URL(String urlString)	

1.3 URL 클래스 메소드

메소드	설명
String getFile()	URL 파일 이름 반환
String getHost()	URL 호스트 이름 반환
String getPort()	URL 포트 이름 반환
String getProtocol()	URL 프로토콜 이름 반환
String toExternalForm()	전체 URL 문자열 반환
URLConnection openConnection()	URL 연결 후 URLConnection 객체 반환
InputStream openStream()	URL 정보를 읽기 위한 InputStream 객체 반환

- URL 클래스 예제

test.url.URLEx.java

```
01  package test.url;
02
03  import java.net.InetAddress;
04  import java.net.UnknownHostException;
05  import java.util.Arrays;
06
07  public class URLEx {
08      public void testURLExample(String urlStr) {
09          System.out.println(urlStr);
10
11          URL url;
12          try {
13              url = new URL(url_str);
14              System.out.println("프로토콜 : " + url.getProtocol());
15              System.out.println("포트번호 : " + url.getPort());
16              System.out.println("호스트 : " + url.getHost());
17              System.out.println("파일경로 : " + url.getFile());
18              System.out.println("URL 전체 : " + url.toExternalForm());
19          } catch (MalformedURLException e) {
20              e.printStackTrace();
21          }
22      }
23  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.url.URLEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new URLEx().testURLExample("https://www.oracle.com/downloads/index.html");
08      }
09  }
```

https://www.oracle.com/downloads/index.html
프로토콜 : https
- 포트번호 : -1 **// 포트번호가 '-1'인 것은 상대 측인 서버에서 보안상의 목적으로 접근을 차단한 것이다.**
호스트 : www.oracle.com

파일경로 : /downloads/index.html URL 전체 : https://www.oracle.com/downloads/index.html

2 URLConnection 클래스

어플리케이션과 URL간의 통신연결을 위한 작업을 하는 클래스

URL 내용을 읽어오거나, URL 주소에 GET / POST 메소드 형식으로 데이터를 전달할 때 사용

추상 클래스로써 자신의 객체를 생성할 수 없고 URL 객체의 openConnection()을 통해 사용한다

URL -> openConnection() -> URLConnection -> getInputStream -> InputStream(내용읽기) 순으로 처리한다

URL 클래스의 openStream()은 입력 스트림만 개설하지만 URLConnection을 이용하면 입력 스트림과 출력 스트림 모두 개설할 수 있다

- URLConnection 클래스 예제

  URL 자원을 읽어 출력

  URLConnection을 사용하지 않고 URL클래스의 openStream() 메소드를 이용해서 구현 가능

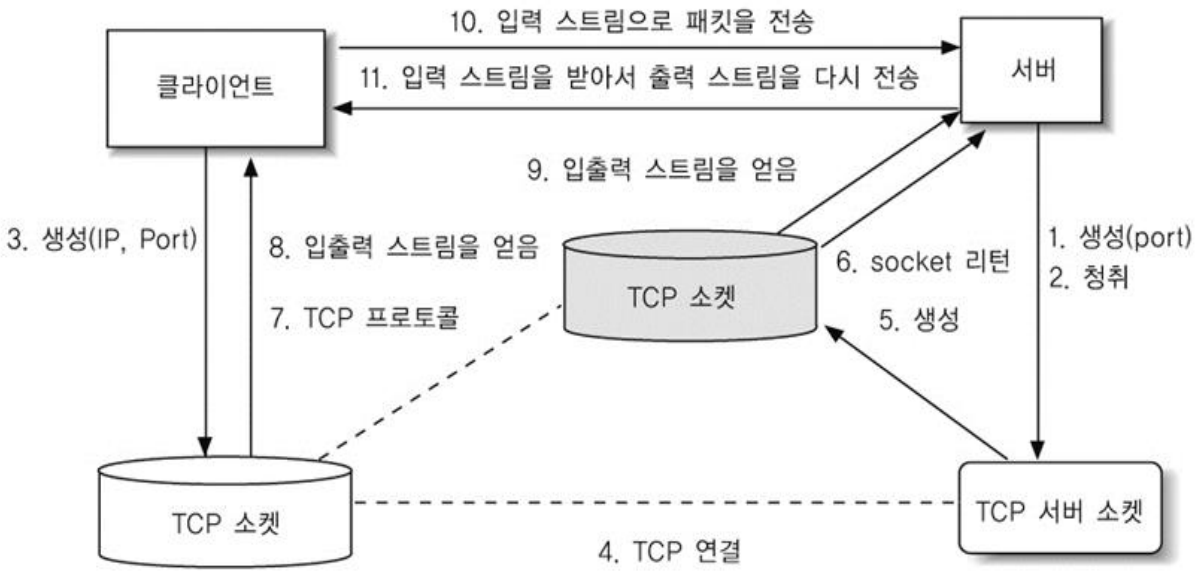
Chapter 7. TCP

1. 소켓

네트워크에 연결된 컴퓨터 간에 데이터를 주고 받을 때 사용하는 도구

하드웨어적인 의미로 소켓이라는 용어를 사용하기도 하지만 여기서는 소프트웨어적인 개발 도구를 뜻한다. 자바에서는 TCP 소켓 통신을 위해 Socket Class와 ServerSocket Class를, UDP 통신을 위해서는 DatagramPacket Class와 DatagramSocket Class를 제공한다.

- 소켓의 연결과 전송과정



2 Socket 클래스

2.1 Socket 주요 생성자

생성자	설명
Socket(InetAddress address, int port)	주어진 IP address와 포트를 가지고 소켓 생성
Socket(String host, int port)	주어진 host와 포트를 가지고 소켓 생성

- Socket 객체 생성 시 두 가지 예외 발생 가능성이 있다
  1. Host를 찾을 수 없거나 Port가 열려 있지 않은 경우 UnknownHostException 예외 발생
  2. Network 실패나 방화벽 때문에 서버에 접근할 수 없을 때 IOException 예외 발생

2.2 Socket 주요 메소드

메소드	설명
InetAddress getInetAddress()	InetAddress로 IP주소 반환
InputStream getInputStream()	입력 스트림 반환
InetAddress getLocalAddress()	로컬 주소 반환
int getPort()	포트 반환
boolean isClosed()	소켓이 닫혀있으면 true, 열려있으면 false 반환
boolean isConnected()	소켓이 연결되어 있으면 true, 비 연결이면 false 반환
void setSoTimeout(int itmeout)	소켓 객체의 timeout시간을 설정
void close()	객체 닫기

2.3 ServerSocket 클래스

ServerSocket 클래스는 TCP 서버 소켓을 의미한다

클라이언트의 TCP연결을 받기 위해서 ServerSocket 클래스의 객체를 생성해야 한다

객체가 생성되면 대기상태가 되어 클라이언트의 TCP 연결 요청이 있을 때까지 기다리게 된다. 클라이언트의 연결 요청이 들어오면 TCP 소켓을 반환하고 이를 통해 클라이언트와 통신할 수 있게 된다.

2.3.1 ServerSocket 클래스 생성자

생성자	설명
ServerSocket(int port)	주어진 포트로 서버소켓 생성
ServerSocket(int port, int backlog)	주어진 포트로 서버소켓을 생성하고 연결 대기 큐의 크기를 지정

2.3.2 ServerSocket 클래스 메소드

메소드	설명
Socket accept()	클라이언트의 연결 요청이 들어올 때까지 블로킹된 채로 기다리며 연결 수립이 되면 클라이언트와 통신 가능한 Socket 객체를 반환
void close()	ServerSocket 닫기
int getLocalPort()	청취하고 있는 Port 번호 반환
int getSoTimeout()	accept() 메소드가 유효할 수 있는 시간을 반환. 0이면 무한대
boolean isClosed()	객체가 닫혀있는지 여부 반환
void setSoTimeout(int timeout)	accept() 메소드가 유효할 수 있는 시간을 설정

3 TCP 서버 & 클라이언트 통신

- TCP ECHO 서버

test.tcp.TCPEchoServer.java

```
01  package test.tcp;
02
03  import java.io.*;
04  import java.net.*;
05
06  public class TCPEchoServer {
07      public void tcpServer(int port) {
08          // 서버 소켓 관련 객체
09          ServerSocket serverSock = null;
10          Socket sock = null;
11          InetAddress inetAddr = null;
12
13          // 입출력 객체
14          InputStream in = null;
15          OutputStream out = null;
16          BufferedReader br = null;
17          PrintWriter wr = null;
18
19          try {
20            // 서버 소켓 생성
21            serverSock = new ServerSocket(port);
22
23            while (true) {
24              // 클라이언트의 접속 대기
25              System.out.println("====== 클라이언트 접속 대기 중"+
26      "(port:" + serverSock.getLocalPort() + ")=====");
27              sock = serverSock.accept();
28
29              // 클라이언트의 접속 요청
30              inetAddr = serverSock.getInetAddress();
31              System.out.println("클라이언트(" + inetAddr.getHostAddress() + ") 접속");
32
33              // 클라이언트와 통신을 위한 stream 생성
34              in = sock.getInputStream();
35              out = sock.getOutputStream();
36              br = new BufferedReader(new InputStreamReader(in));
```

```
37                  wr = new PrintWriter(new OutputStreamWriter(out));
38                  String msg = null;
39
40                  // 클라이언트와 통신
41                  while ((msg = br.readLine()) != null) {
42                    System.out.println("₩tCLIENT> " + msg);
43                    wr.println(msg);
44                    wr.flush();
45                  }
46                }
47              } catch (IOException ie) {
48                System.out.println(ie);
49              } finally {
50                try {
51                  br.close();
52                  wr.close();
53                  in.close();
54                  out.close();
55                  sock.close();
56                  serverSock.close();
57                  System.out.println("종료.");
58                } catch (IOException ie) {
59                  System.out.println(ie);
60                }
61              }
62            }
63  }
```

test.main.Main.java

```
01  package test.main;
02
03      import test.tcp.TCPEchoServer;
04
05      public class Main {
06              public static void main(String[] args) {
07                      new TCPEchoServer().tcpServer(9007);
08              }
09      }
```

* 클라이언트의 접속이 끊기면 다음 클라이언트와 소켓 연결 통신 대기상태가 된다

- TCP ECHO 클라이언트

test.tcp.TCPEchoClient.java

```
01  package test.tcp;
02
03  import java.io.BufferedReader;
04  import java.io.IOException;
05  import java.io.InputStream;
06  import java.io.InputStreamReader;
07  import java.io.OutputStream;
08  import java.io.OutputStreamWriter;
09  import java.io.PrintWriter;
10  import java.net.Socket;
11
12  public class TCPEchoClient {
13  public void clientTCP(String ip, int port) {
14      Socket sock = null;
15
16      InputStream in = null;
17      OutputStream out = null;
18      BufferedReader br = null;
19      PrintWriter wr = null;
20      BufferedReader stdin = null;
21
22      try {
23       // 서버로 접속
24       sock = new Socket(ip, port);
25       System.out.println("접속성공 !!");
26
27       // 서버와 통신을 위한 stream, 표준입력 변수
28       in = sock.getInputStream();
29       out = sock.getOutputStream();
30       br = new BufferedReader(new InputStreamReader(in));
31       wr = new PrintWriter(new OutputStreamWriter(out));
32       stdin = new BufferedReader(new InputStreamReader(System.in));
33       String msg = null;
34       String echo = null;
35
36       // 서버와 통신
37       do {
```

```
38                  System.out.print("₩tInsert Msg > ");
39                  msg = stdin.readLine();
40
41                  // "exit" 입력으로 접속 끊기
42                  if (msg.equals("exit")) {
43                  sock.close();
44                  break;
45                  }
46
47                  // 에코 메시지 출력
48                  wr.println(msg);
49                  wr.flush();
50                  echo = br.readLine();
51                  System.out.println("₩tSERVER> " + echo);
52              } while (msg != null);
53              } catch (IOException ie) {
54               System.out.println(ie);
55              } finally {
56               try {
57                  stdin.close();
58                  br.close();
59                  wr.close();
60                  in.close();
61                  out.close();
62                  sock.close();
63                  System.out.println("종료.");
64               } catch (IOException ie) {
65                  System.out.println(ie);
66               }
67              }
68      }
69  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.tcp.TCPEchoClient;
04
05  public class Main {
06      public static void main(String[] args) {
```

```
07        new TCPEchoClient().clientTCP("localhost", 9007);
08    }
09  }
```

```
접속성공 !!
        Insert Msg > Hello
        SERVER> Hello
        Insert Msg > Java
        SERVER> Java
        Insert Msg > exit
종료.
```

net.multi.chatt.server.TcpServerMulti.java

```
01  package net.multi.chatt.server;
02
03  import java.util.*;
04  import java.net.*;
05  import java.sql.ClientInfoStatus;
06  import java.io.*;
07
08  public class TcpServerMulti {
09      private HashMap<String, User> users;  //회원 정보 저장용
10      private Vector<Socket> clients;                    //로그인한 클라이언트 소켓들 저장용
11
12      public TcpServerMulti(){
13          users = new HashMap<String, User>();
14
15          User[] userList = new User[]{
16                  new User("user01", "pass01", "smileman"),
17                  new User("user02", "pass02", "prittywoman"),
18                  new User("user03", "pass03", "javajjang"),
19                  new User("test123", "test123", "testboy"),
20                  new User("p1234", "p1234", "pick")};
21
22          for(User u : userList)
23              users.put(u.getUserId(), u);
24      }
25
26      public Map<String, User> getUsers(){
```

```
27                  return users;
28          }
29
30          public Vector<Socket> getClients(){
31                  return clients;
32          }
33
34          public String loginCheck(String id, String pwd){
35                  String nickName = null;
36
37                  if(users.containsKey(id)){
38                          User loginUser = users.get(id);
39
40                          if(loginUser.getUserPwd().equals(pwd))
41                                  nickName = loginUser.getNickName();
42                  }
43
44                  return nickName;
45          }
46
47          public void displayMsg(String clientMessage){
48                  System.out.println(clientMessage);
49          }
50
51          public static void main(String[] args){
52                  TcpServerMulti tServer = new TcpServerMulti();
53
54                  int port = 7979;
55                  ServerSocket server;
56                  try {
57                          server = new ServerSocket(port);
58
59                          while(true){
60                                  Socket client = server.accept();
61                                  tServer.clients.addElement(client);
62
63                                  new ClientHandler(tServer, client).start();
64                          }
65                  } catch (IOException e) {
```

```
66                                  e.printStackTrace();
67                          }
68                  }
69  }
```

net.multi.chatt.server.User.java

```
01  package net.multi.chatt.server;
02
03  public class User {
04          private String userId;
05          private String userPwd;
06          private String nickName;
07
08          public User(){}
09
10          public User(String userId, String userPwd, String nickName) {
11                  super();
12                  this.userId = userId;
13                  this.userPwd = userPwd;
14                  this.nickName = nickName;
15          }
16
17          public String getUserId() {
18                  return userId;
19          }
20
21          public void setUserId(String userId) {
22                  this.userId = userId;
23          }
24
25          public String getUserPwd() {
26                  return userPwd;
27          }
28
29          public void setUserPwd(String userPwd) {
30                  this.userPwd = userPwd;
31          }
32
33          public String getNickName() {
34                  return nickName;
```

```
35              }
36
37              public void setNickName(String nickName) {
38                      this.nickName = nickName;
39              }
40
41              @Override
42              public String toString(){
43                      return userId + ", " + userPwd + ", " + nickName;
44              }
45      }
```

net.multi.chatt.client.ClientHandler.java

```
01  package net.multi.chatt.client;
02
03  import java.net.*;
04  import java.util.*;
05  import java.io.*;
06
07  public class ClientHandler extends Thread{
08          private Socket client;                  //로그인한 클라이언트 소켓
09          private TcpServerMulti server;
10          private BufferedReader br;
11          private PrintWriter pw;
12
13          public ClientHandler(Socket client) throws IOException{
14                  this.client = client;
15                  br = new BufferedReader(new InputStreamReader(client.getInputStream()));
16                  pw = new PrintWriter(client.getOutputStream());
17          }
18
19          public ClientHandler(TcpServerMulti server, Socket client){
20                  this.server = server;
21                  this.client = client;
22                  try {
23                          br = new BufferedReader(new InputStreamReader(client.getInputStream()));
24                          pw = new PrintWriter(client.getOutputStream());
25                  } catch (IOException e) {
26                          e.printStackTrace();
27                  }
```

```
28          }
29
30          @Override
31          public void run(){
32                  //한 클라이언트로 부터 메세지 읽어들임
33                  //읽어들인 메세지를 연결된 모든 클라이언트에게 전송함
34                  String nickName = null;
35                  try{
36                          nickName = br.readLine();
37                          String sendMessage = "[" + nickName + "]님이 입장하셨습니다";
38                          server.displayMsg(sendMessage);
39                          sendAll(sendMessage);
40
41                          while(true)  //대화시작
42                          {
43                                  String message = br.readLine();
44                                  sendMessage = nickName + " : " + message;
45                                  server.displayMsg(sendMessage);
46                                  sendAll(sendMessage);
47                          }  //해당 클라이언트가 접속종료하면 Exception 발생함.
48                  }catch(Exception e){
49                          //System.out.println(e.getMessage());
50                  }finally{
51                          String sendMessage = "[" + nickName +"]님이 퇴장하셨습니다";
52                          server.displayMsg(sendMessage);
53                          sendAll(sendMessage);
54                  }
55          }
56
57          public void sendAll(String sendMessage){
58                  //저장된 소켓들에 대한 출력 스트림 생성하고, 로그인한 클라이언트 모두에게
59                  //메세지 전송 처리 : for 문 사용
60                  Vector<Socket> clients = server.getClients();
61                  synchronized(clients){
62                          for(int i = 0; i < clients.size(); i++){
63                                  ClientHandler otherClient = null;
64                                  try {
65                                          otherClient = new ClientHandler(clients.get(i));
66                                          otherClient.pw.println(sendMessage);
```

```
67                                          otherClient.pw.flush();
68                                  } catch (IOException e) {
69                                          e.printStackTrace();
70                                  } finally{
71                                          try{
72                                                  otherClient.br.close();
73                                                  otherClient.pw.close();
74                                          }catch(IOException e){
75                                                  e.printStackTrace();
76                                          }
77                                  }
78                          }
79                  }
80          }
81  }
```

Chapter 8. UDP

1 DatagramPacket 클래스

자바 UDP 통신에서 데이터그램를 표현하기 위한 클래스이다. (Datagram : UDP통신에서 데이터조각을 뜻함)

1.1 DatagramPacket 클래스 생성자

생성자	설명
DatagramPacket(byte[] buf, int len)	데이터 그램과 데이터의 길이를 가지고 객체 생성
DatagramPacket(byte[] buf, int len, InetAddress addr, int port)	데이터 그램과 데이터의 길이, 통신에 사용될 주소와 포트를 가지고 객체 생성

1.2 DatagramPacket 클래스 메소드

메소드	설명
byte[] getData()	데이터 버퍼를 반환
int getLength()	전송받거나 전송할 데이터의 길이를 반환
InetAddress getAddress()	전송받거나 전송할 주소를 반환
int getPort()	전송받거나 전송할 포트를 반환
void setData(byte[] buf)	데이터 버퍼 설정
void setLength(int length)	데이터의 길이 설정
void setAddress(InetAddress addr)	데이터그램을 전송할 주소 설정
void setPort(int port)	데이터그램을 전송할 포트 설정

2 DatagramSocket 클래스

데이터그램 패킷을 전송하거나 수신하기 위한 클래스로, 실시간 데이터의 전송을 책임진다.

2.1 DatagramSocket 클래스 생성자

생성자	설명
DatagramSocket(int port)	주어진 포트로 UDP 소켓 생성
DatagramSocket(int port, InetAddress addr)	주어진 포트와 주소로 UDP 소켓 생성

2.2 DatagramSocket 클래스 메소드

메소드	설명
void send(DatagramPacket p)	데이터그램패킷을 소켓으로 송신
void receive(DatagramPacket p)	소켓으로부터 데이터그램패킷을 수신
close	데이터그램소켓 닫기

3 UDP 서버 & 클라이언트 통신

- UDP ECHO 서버

test.udp.UDPEchoServer.java

```
01  package test.udp;
02
03  import java.net.*;
04
05  public class UDPEchoServer {
06     public void serverUDP (int port){
07        DatagramSocket dSock = null;
08
09        try{
10           dSock = new DatagramSocket(port);
11
12           while(true){
13              System.out.println("====== 클라이언트 접속 대기 중 (port:"
14                 + dSock.getPort() + ")=====");
15
16              // 데이터 수신
17              byte[] buffer = new byte[1024];
18              DatagramPacket receive = new DatagramPacket(buffer, buffer.length);
19              dSock.receive(receive);
20              String msg = new String(receive.getData(), 0, receive.getLength());
21              System.out.println("₩tCLIENT > " + msg);
22
23              // 데이터 송신
24              DatagramPacket send = new DatagramPacket(
25                 receive.getData(),
26                 receive.getData().length,
27                 receive.getAddress(),
28                 receive.getPort());
29              dSock.send(send);
30           }
31        }catch(Exception e){
32           System.out.println(e);
33        }finally{
34           dSock.close();
35        }
36     }
```

```
37  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.udp.UDPEchoServer;
04
05  public class Main {
06      public static void main(String[] args) {
07          new UDPEchoServer().serverUDP(9007);
08      }
09  }
```

- UDP ECHO 클라이언트

test.udp.UDPEchoClient.java

```
01  package test.udp;
02
03  import java.io.*;
04  import java.net.*;
05
06  public class UDPEchoClient {
07      public void clientUDP(String ip){
08          // 주소 설정 - localhost
09          InetAddress inetaddr = null;
10
11          try {
12              inetaddr = InetAddress.getByName(ip);
13          } catch (UnknownHostException e) {
14              System.out.println(e);
15          }
16
17          DatagramSocket dSock = null;
18          try{
19              BufferedReader br =     new BufferedReader(new InputStreamReader(System.in));
20              dSock = new DatagramSocket();
21
22              String str = null;
23              do {
24                  // 데이터그램 송신
25                  System.out.print("₩tInsert Msg > ");
```

```
26              str = br.readLine();
27              DatagramPacket send = new DatagramPacket(
28                 str.getBytes(),
29                 str.getBytes().length,
30                 inetaddr,
31                 9007);
32              dSock.send(send);
33
34              if(str.equals("exit")) break;
35
36              // 데이터그램 수신
37              byte[] buffer = new byte[str.getBytes().length];
38              DatagramPacket receive = new DatagramPacket(buffer, buffer.length);
39              dSock.receive(receive);
40              String echo = new String(receive.getData(), 0, receive.getData().length);
41              System.out.println("₩tSERVER> "+echo);
42           } while(str != null);
43
44           System.out.println("UDPEchoClient를 종료합니다.");
45        }catch(Exception ex){
46           System.out.println(ex);
47        }finally{
48           dSock.close();
49        }
50     }
51  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.udp.UDPEchoClient;
04
05  public class Main {
06     public static void main(String[] args) {
07        new UDPEchoClient().clientUDP("localhost");
08     }
09  }
```

messenger.udp.Messenger.java

```
01  package messenger.model;
02
03  import java.io.IOException;
04  import java.net.*;
05  import javax.swing.*;
06
07  public class Messenger {
08          private MyFrame f;
09          private DatagramSocket socket;
10          private InetAddress address = null;
11          private final int myPort;
12          private final int otherPort;
13
14          public Messenger(int myPort, int otherPort){
15                  this.myPort = myPort;
16                  this.otherPort = otherPort;
17                  try {
18                          address = InetAddress.getByName("127.0.0.1");
19                          socket = new DatagramSocket(myPort);
20                          f = new MyFrame(address, otherPort, socket);
21                  } catch (UnknownHostException e) {
22                          e.printStackTrace();
23                  } catch (SocketException e) {
24                          e.printStackTrace();
25                  }
26          }
27
28          public MyFrame getMyFrame(){
29                  return f;
30          }
31  }
```

messenger.udp.MyFrame.java

```
01  package messenger.view;
02
03  import java.awt.*;
04  import java.awt.event.*;
05  import java.io.IOException;
06  import java.net.*;
```

```
07  import javax.swing.*;
08
09  public class MyFrame extends JFrame implements ActionListener{
10          private DatagramSocket socket;
11          private DatagramPacket packet;
12          private JTextField textField;
13          private JTextArea textArea;
14          private InetAddress address;
15          private int otherPort;
16
17          public MyFrame(InetAddress address, int otherPort, DatagramSocket socket){
18                  this.address = address;
19                  this.otherPort = otherPort;
20                  this.socket = socket;
21                  this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
22
23                  textField = new JTextField(30);
24                  textField.addActionListener(this);
25
26                  textArea = new JTextArea(10, 30);
27                  textArea.setEditable(false);
28
29                  this.add(textField, BorderLayout.PAGE_END);
30                  this.add(textArea, BorderLayout.CENTER);
31                  this.pack();
32                  this.setVisible(true);
33          }
34
35          @Override
36          public void actionPerformed(ActionEvent arg0) {
37                  String str = textField.getText();
38                  byte[] buffer = str.getBytes();
39                  DatagramPacket sendPacket = new DatagramPacket(buffer, buffer.length,
40  address, otherPort);
41                  try {
42                          socket.send(sendPacket);
43                  } catch (IOException e) {
44                          e.printStackTrace();
45                  }
```

```
46                      textArea.append("SENT : " + str + "₩n");
47                      textField.selectAll();
48                      textArea.setCaretPosition(textArea.getDocument().getLength());
49              }
50
51              public void process(){
52                      while(true){
53                              byte[] buf = new byte[256];
54                              packet = new DatagramPacket(buf, buf.length);
55                              try {
56                                      socket.receive(packet);
57                                      textArea.append("RECIEVE : " + new String(buf) + "₩n");
58                              } catch (IOException e) {
59                                      e.printStackTrace();
60                              }
61                      }
62              }
63      }
```

messenger.udp.Run.java

```
01      package messenger.controller;
02
03      import java.net.DatagramSocket;
04      import javax.swing.JOptionPane;
05
06      public class Run {
07              public static void main(String[] args) {
08                      int myPort = Integer.parseInt(JOptionPane.showInputDialog("당신의 포트번호를
09      입력하세요"));
10                      int otherPort = Integer.parseInt(JOptionPane.showInputDialog("상대방 포트 번호
11      를 입력하세요"));
12
13                      Messenger a = new Messenger(myPort, otherPort);
14                      MyFrame f = a.getMyFrame();
15                      f.process();
16              }
17      }
```

Part 3. 프로그램 디버깅하기

Chapter 9. UNICAST CHATTING

network.tcp.Server.java

```
01  package network.tcp;
02
03  import java.io.BufferedReader;
04  import java.io.IOException;
05  import java.io.InputStream;
06  import java.io.InputStreamReader;
07  import java.io.OutputStream;
08  import java.io.PrintWriter;
09  import java.net.ServerSocket;
10  import java.net.Socket;
11
12  public class Server {
13          public static void main(String[] agrs) throws IOException{
14                  int port = 8500;
15
16                  ServerSocket server = new ServerSocket(port);
17
18                  while(true){
19                          Socket client = server.accept();
20
21                          InputStream input = client.getInputStream();
22                          OutputStream output = client.getOutputStream();
23
24                          BufferedReader br = new BufferedReader(new InputStreamReader(input));
25                          PrintWriter pw = new PrintWriter(output);
26
27                          while(true){
28                                  String message = br.readLine();
29
30                                  if(!message.equals("exit")){
31                                          System.out.println(client.getInetAddress().getHostAddress()  +  "
32  가 보낸 메세지 : " + message);
33                                          pw.println("메세지 받기 성공");
34                                          pw.flush();
35                                  }else{
36                                          System.out.println("접속 종료");
```

```
37                                                  break;
38                                          }
39                                  }
40
41                                  br.close();
42                                  pw.close();
43                                  client.close();
44
45                          }
46                  }
47  }
```

network.tcp.Client.java

```
01  package network.tcp;
02
03  import java.io.BufferedReader;
04  import java.io.IOException;
05  import java.io.InputStream;
06  import java.io.InputStreamReader;
07  import java.io.OutputStream;
08  import java.io.PrintWriter;
09  import java.net.InetAddress;
10  import java.net.Socket;
11  import java.util.Scanner;
12
13  public class Client {
14
15          public static void main(String[] args) {
16                  int port = 8500;
17                  BufferedReader br = null;
18                  PrintWriter pw = null;
19
20                  try {
21                          String serverIP = InetAddress.getLocalHost().getHostAddress();
22
23                          Socket socket = new Socket(serverIP, port);
24
25                          if(socket != null){
26                                  InputStream input = socket.getInputStream();
27                                  OutputStream output = socket.getOutputStream();
```

```
28                                      br = new BufferedReader(new InputStreamReader(input));
29                                      pw = new PrintWriter(output);
30
31                                      Scanner sc = new Scanner(System.in);
32
33                                      do{
34                                              System.out.print("대화 입력 : ");
35                                              String message = sc.nextLine();
36
37                                              pw.println(message);
38                                              pw.flush();
39
40                                              if(message.equals("exit")){
41                                                      break;
42                                              }
43
44                                              String recieveMessage = br.readLine();
45                                              System.out.println(recieveMessage);
46                                              sc.nextLine();
47
48                                      }while(true);
49
50                              }
51
52                              socket.close();
53
54                      } catch (Exception e) {
55                              e.printStackTrace();
56                      }
57              }
58  }
```

network.tcp.NetworkTest.java

```
01  package network.tcp;
02
03  import java.net.InetAddress;
04  import java.net.UnknownHostException;
05
06  public class NetworkTest {
07
```

```
08          public static void main(String[] args) throws UnknownHostException {
09                  // InterAddress Test
10                                  InetAddress localIP = InetAddress.getLocalHost();
11
12                                  System.out.println(localIP.getHostAddress());
13                                  System.out.println(localIP.getHostName());
14
15                                  InetAddress naverIP =
16  InetAddress.getByName("www.naver.com");
17                                  InetAddress[] googleIPs =
18  InetAddress.getAllByName("www.google.com");
19
20                                  System.out.println("네이버 서버 ip : " +
21  naverIP.getHostAddress());
22                                  System.out.println("구글 서버 ip 갯수 : " + googleIPs.length);
23
24                                  for(InetAddress ip : googleIPs)
25                                          System.out.println(ip.getHostAddress());
26
27          }
28
29  }
```

Chapter 10. MULTICAST CHATTING

최적화된 프로그램이란 더 이상 잘 짜인 소스코드로 구성된 빠른 소프트웨어를 의미하지 않는다. 지금의 최적화된 프로그램이란 지정된 사양에 맞는 성능을 가지면서 신뢰성 있고, 유지 보수하기 쉬우며 변화하는 요구사항에 빠르게 적응 할 수 있는 소프트웨어를 의미한다.

소프트웨어에서의 아키텍처 설계란 다음과 같은 사항들을 의미한다.
- 큰 규모의 컴포넌트를 어떻게 조합하는가
- 시스템 수준에서 여러 소프트웨어 부품을 어떻게 조합할 것인가
- (특히 분산 아키텍처에서) 시스템 수준에서 응용 프로그램을 어떻게 설계할 것인가
- 재사용할 수 있는 용어의 정리

소프트웨어 아키텍처는 컴포넌트의 규모, 범위를 정의하고 정의된 컴포넌트간의 연결을 정의한다.
아키텍처를 설계할 때는 잘 파악된 요구사항에 따라 어떤 요구사항을 반영하는 비즈니스 객체를 어떤 컴포넌트에 포함시키느냐를 포함하는 소프트웨어의 규모 혹은 범위 정의가 무엇보다 중요하다.

하지만 소프트웨어의 최적화 관점에서 더욱 중요한 것은 소프트웨어 컴포넌트의 연결을 어떻게 구성할 것인가이다.

두 개의 컴포넌트를 같은 프로세스에서 동작하도록 아키텍처를 설계 했다고 가정하자. 같은 프레임워크를 기반으로 같은 프로세스에서 동작하는 두 컴포넌트는 프레임워크가 지원하는 어떠한 방법으로도 객체 및 데이터를 교환할 수 있다.

이런 경우 프레임워크가 지원하는 데이터 교환 방식 및 데이터 포맷 중 가능 빠르고 가벼운 방법으로 두 컴포넌트가 통신할 수 있도록 설계하는 것이 가장 바람직하다.

chat.client.ClientGUI.java

```
01 package chat.client;
02
03 import java.awt.BorderLayout;
04 import java.awt.Color;
05 import java.awt.Font;
06 import java.awt.event.ActionEvent;
07 import java.awt.event.ActionListener;
08
09 import javax.swing.JFrame;
10 import javax.swing.JOptionPane;
11 import javax.swing.JTextArea;
```

```
12  import javax.swing.JTextField;
13
14  public class ClientGUI extends JFrame implements ActionListener{
15
16          private static final long serialVersionUID = 10000000L; // 컴파일 시 인식할 객체 고유
17  ID
18      private JTextArea jta = new JTextArea(40,25);   // 채팅 창
19      private JTextField jtf = new JTextField(25);    // 글 입력 부분
20      private ClientBackground client = new ClientBackground(); // 채팅을 담당할 백그라운드(뒷작
21  업 영역) 선언
22      private static String nickName;
23
24      public ClientGUI(String nickName){
25          ClientGUI.nickName = nickName;
26
27          setBounds(800, 100, 400, 600);
28          jta.setFont(new Font("맑은고딕", Font.PLAIN, 18));   // 글자 폰트, 크기 변경
29          setTitle("클라이언트부분");
30
31          add(jta, BorderLayout.CENTER);
32          add(jtf, BorderLayout.SOUTH);
33          jtf.addActionListener(this);
34          jta.setEditable(false);
35          jta.setBackground(new Color(230, 255, 230));
36
37
38          setDefaultCloseOperation(EXIT_ON_CLOSE);
39          setVisible(true);
40
41          client.setGui(this);
42          client.setNickname(nickName);
43          client.connect();
44      }
45
46      @Override
47      public void actionPerformed(ActionEvent e) {
48          // 문자 입력 창에 글을 입력 시 구동할 이벤트 설정
49          String msg =nickName+":"+ jtf.getText() + "₩n";
50          client.sendMessage(msg);
```

```
51              jtf.setText("");  // 입력 후 내용 초기화
52
53          }
54          public void appendMsg(String msg) {
55              jta.append(msg);
56          }
57
        }
```

chat.client.ClientBackground.java

```
01  package chat.client;
02
03  import java.io.DataInputStream;
04  import java.io.DataOutputStream;
05  import java.io.IOException;
06  import java.net.Socket;
07  import java.util.Scanner;
08
09  public class ClientBackground {
10      private Socket socket;
11      private DataInputStream in;
12      private DataOutputStream out;
13      private ClientGUI gui;
14      private String msg;
15      private String nickName;
16
17      public void setGui(ClientGUI gui) {
18          this.gui = gui;
19      }
20
21      //클라이언트 접속에 대한 메소드
22      public void connect(){
23          try {
24                  // 연결할 서버의 정보를 설정
25              socket = new Socket("127.0.0.1", 7777);
26              System.out.println("서버에 연결됨");
27
28              // 서버와 주고 받을 입,출력 통로 생성
29              out = new DataOutputStream(socket.getOutputStream());
30              in = new DataInputStream(socket.getInputStream());
```

```
31
32              // 접속 후 서버에서 닉네임을 인식
33              out.writeUTF(nickName);
34              System.out.println("Client : 닉네임 전송완료 ");
35
36              while(in!=null){ // 입력한 문자열을 화면에 출력하는 기능
37                  msg = in.readUTF();
38                  gui.appendMsg(msg);
39              }
40
41          } catch (IOException e) {
42              //e.printStackTrace();
43          }
44      }
45
46      public void sendMessage(String msg){
47          try {
48              out.writeUTF(msg);
49          } catch (IOException e) {
50              e.printStackTrace();
51          }
52      }
53
54      public void setNickname(String nickName){ // 닉네임 Setter
55          this.nickName = nickName;
56      }
57
58  }
```

chat.client.ClientRun.java

```
01  package chat.client;
02
03  import javax.swing.JOptionPane;
04
05  public class ClientRun {
06      public static void main(String[] args) {
07          // 사용자 GUI를 구동하기 위한 메소드
08          String nickName = JOptionPane.showInputDialog("닉네임을 입력하세요.");
09          new ClientGUI(nickName);
10      }
```

11	}

chat.server.ServerGUI.java

```
01  package chat.server;
02
03  import java.awt.BorderLayout;
04  import java.awt.Color;
05  import java.awt.Font;
06  import java.awt.event.ActionEvent;
07  import java.awt.event.ActionListener;
08  import java.io.IOException;
09
10  import javax.swing.JFrame;
11  import javax.swing.JTextArea;
12  import javax.swing.JTextField;
13
14  public class ServerGUI extends JFrame implements ActionListener{
15
16          private static final long serialVersionUID = 20000000L; // 컴파일 시 인식할 객체 고유
17  ID
18      private JTextArea jta = new JTextArea(40,25);   // 채팅 창
19      private JTextField jtf = new JTextField(25);    // 글 입력 부분
20      private ServerBackground server = new ServerBackground(); // 채팅을 담당할 백그라운드(뒷
21  작업 영역) 선언
22
23      public ServerGUI() throws IOException{
24          setBounds(200, 100, 400, 600);
25          setTitle("서버부분");
26
27          jta.setEditable(false);
28          jta.setFont(new Font("맑은고딕", Font.PLAIN, 18));   // 글자 폰트, 크기 변경
29          jta.setBackground(new Color(230, 255, 230));
30
31          add(jta, BorderLayout.CENTER);
32          add(jtf, BorderLayout.SOUTH);
33          jtf.addActionListener(this);
3435
36          setDefaultCloseOperation(EXIT_ON_CLOSE);
37          setVisible(true);
38
```

```
39              server.setGui(this);
40              server.setting(); // GUI 실행 시 서버도 같이 동작 시킨다.
41          }
42
43          @Override
44          public void actionPerformed(ActionEvent e) {
45               // 문자 입력 창에 글을 입력 시 구동할 이벤트 설정
46              String msg = "운영자 : " +jtf.getText() + "₩n";
47              System.out.print(msg);
48              server.sendMessage(msg);
49              appendMsg(msg);
50              jtf.setText("");  // 입력 후 내용 초기화
51          }
52
53          public void appendMsg(String msg) {
54              jta.append(msg);
55          }
    }
```

chat.server.ServerBackground.java

```
01  package chat.server;
02
03  import java.io.DataInputStream;
04  import java.io.DataOutputStream;
05  import java.io.IOException;
06  import java.net.ServerSocket;
07  import java.net.Socket;
08  import java.util.Collections;
09  import java.util.HashMap;
10  import java.util.Iterator;
11  import java.util.Map;
12
13  public class ServerBackground {
14
15      private ServerSocket serverSocket; // 서버 소켓
16      private Socket socket; // 받아올 소켓 저장
17      private ServerGUI gui;
18      private String msg;
19      private int count; // 현재 접속자 수를 세는 변수
20
```

```
21      //  사용자들의 정보를 저장하는 맵 객체 선언
22      private Map<String, DataOutputStream> clientMap = new HashMap<String,
23  DataOutputStream>();
24
25      public void setGui(ServerGUI gui) {
26          this.gui = gui;
27      }
28
29      //서버 생성 시 서버 정보 설정
30      public void setting() {
31          try {
32              // 사용자들이 동시 접속 시 맵 에 정보가 순차적으로 저장될 수 있게 동기화
33              Collections.synchronizedMap(clientMap);
34
35              // 서버의 포트 번호 설정
36              serverSocket = new ServerSocket(7777);
37
38              while (true) {
39                  // 방문자를 제한없이 받아 멀티쓰레드로 배분
40                  System.out.println("접속자 대기 중.....");
41                  socket = serverSocket.accept(); // 여기서 클라이언트 받음
42                  System.out.println(socket.getInetAddress() + "에서 접속했습니다.");
43
44                  // 새로운 사용자의 정보를 소켓에 담아 쓰레드로 생성한다.
45                  Receiver receiver = new Receiver(socket);
46                  receiver.start();
47              }
48
49          } catch (IOException e) {
50              e.printStackTrace();
51          }
52      }
53
54      // 맵에 사용자 정보를 저장하고 화면에 접속 정보를 표현하는 메소드
55      public void addClient(String nick, DataOutputStream out) throws IOException{
56          String message=nick + "님이 접속하셨습니다.₩n";
57          sendMessage(message);
58          gui.appendMsg(message);
59          clientMap.put(nick, out);
```

```
60          System.out.println(">>> 현재 접속자 : "+(++count));
61      }
62
63      // 맵에 사용자 정보를 삭제하고 화면에 접속 정보를 표현하는 메소드
64      public void removeClient(String nick){
65          String message=nick + "님이 나가셨습니다. ₩n";
66          sendMessage(message);
67          gui.appendMsg(message);
68          clientMap.remove(nick);
69          System.out.println(">>> 현재 접속자 : "+(++count));
70      }
71
72      //메세지 내용을 BroadCasting 방식으로 전파
73      public void sendMessage (String msg){
74          Iterator<String> iterator = clientMap.keySet().iterator(); // keySet() 으로 연속처리
75          String key = "";
76
77          while(iterator.hasNext()){  // 각 사용자 정보 객체마다 하나씩 처리
78              key = iterator.next();
79              try{
80                  clientMap.get(key).writeUTF(msg);
81              } catch(IOException e){
82                  // e.printStackTrace();
83              }
84          }
85      }
86      // ------------------리시버--------------------------
87      class Receiver extends Thread {
88          //
89          private DataInputStream in; // 사용자가 전송한 데이터를 읽어 들이는 통로 생성
90          private DataOutputStream out; // 사용자에게 데이터를 출력할 통로 생성
91          private String nick;
92
93          // 사용자 정보를 받아 Thread를 통해 소켓을 생성
94          public Receiver(Socket socket) {
95              try {
96                  out = new DataOutputStream(socket.getOutputStream());
97                  in = new DataInputStream(socket.getInputStream());
98                  nick = in.readUTF();
```

```
99                      addClient(nick,out);
100                 } catch (IOException e) {
101                     e.printStackTrace();
102                 }
103             }
104
105             @Override
106             public void run() {
107
108                 try { // 입력한 문자열을 화면에 출력하는 기능
109                     while (in != null) {
110                         msg = in.readUTF();// UTF 문자셋으로 읽어 오는 메소드
111                         sendMessage(msg);
112                         gui.appendMsg(msg);
113                     }
114                 } catch (Exception e) {
115                     //사용자가 접속 종료, 즉 GUI 종료 시 실행할 내용 작성
116                     removeClient(nick);
117                 }
118             }
119         }
120
121 }
```

chat.server.ServerRun.java

```
01  package chat.server;
02
03  import java.io.IOException;
04
05  public class ServerRun {
06
07      public static void main(String[] args){
08          // 서버를 구동하기 위한 메소드
09          try {
10                              new ServerGUI();
11                  } catch (IOException e) {
12                              System.err.println(e.getMessage()+"₩n서버 프로그램 실행 실패!!");
13                  }
14      }
15  }
```

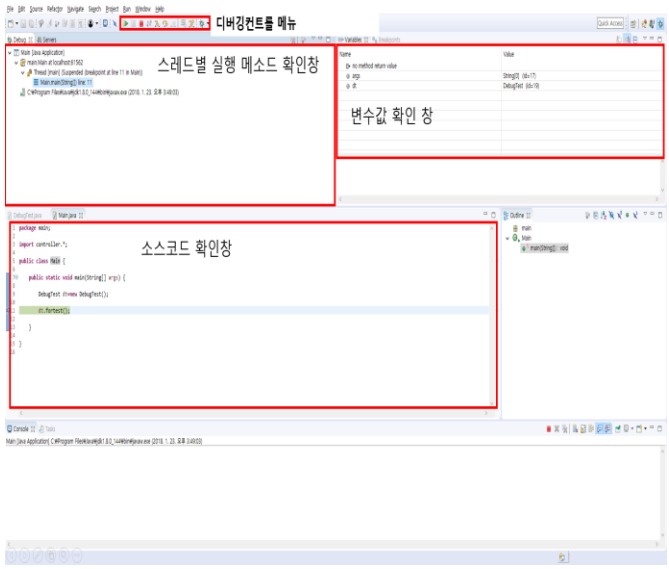
1 디버깅 이란

프로그램의 개발에서 프로그램의 오류를 발견하고 그 원인을 밝히는 과정을 말한다. 쉽게 말해 구성된프로그램의 흐름을 따라가면서 로직에 맞게 변수(객체, 배열, 기본자료형 등)에 제대로된 값이 들어가는지 확인하고 처리로직(메소드)의 매개변수, 반환값도 제대로 된 값이 나오는지 확인하는 것이다. 그리고 디버깅을 도와주는 소프트웨어가 있는데 이를 디버거라고 한다. 우리는 Eclipse에서 디버깅 하는 방법을 절차에 따라 공부해보자.

2 이클립스 디버깅 순서

문제가 될 부분에 BreakPoint설정 -> 디버그 실행(run-debug 또는 )-> 디버그모드 화면전환 -> variables에서 각변수별 저장된 값 확인

3 이클립스 디버그 모드 화면



3.1 디버깅 컨트롤 메뉴

①  (F8) : 중단된 코드 재실행 명령

②  : 실행중인 코드를 중단하는 명령

③  : 디버깅 모드 중단 명령

④  (F5) : 브레이크 포인트 다음에 호출되는 소스코드 실행
(내부에서 실행되는 메소드까지 들어가서 확인)

⑤  (F6) : 브레이크 포인트 다음에 있는 소스코드 실행
(내부에서 실행되는 메소드는 제외)

⑥  (F7) : 다음 브레이크 포인트 실행

⑦  (F5) : 디버그모드 실행

3.2 스레드별 메소드 확인창

각 스레드별로 BreakPoint를 설정한 메소드를 확인할 수 있는창으로 해당하는 메소드를 클릭하면 변수값을 확인하는 창이 그 메소드를 포함하고 클래스(this) 정보와 메소드에서 사용하는 변수를 보여준다.

3.3 변수값 확인창

Variables탭과 BreakPoint탭으로 나눠진다.

Variables : BreakPoint로 지정된 소스코드가 있는 범위의 모든 변수의 값을 표시해준다. Name에는 변수명이 values에는 값들이 들어간다. 일반 변수는 그냥 value값이 출력되지만 객체나 배열은 클릭해야 그안의 멤버변수 및 변수를 확인할 수 있다.

☞ 객체에 선언된 메소드는 확인할 수 없다.

BreakPoint : BreakPoint가 설정되어있는 코드를 확인할 수 있으며 앞에 있는 체크박스를 해제하면 BreakPoint가 해제된다.

4 BreakPoint 설정방법

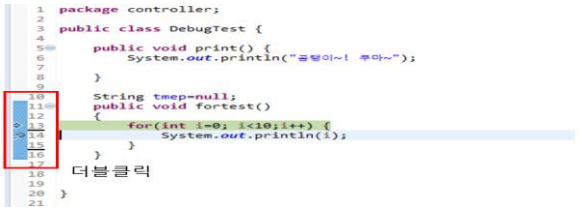
4.1 BreakPoint란

소스코드의 흐름대로 실행중인 프로그램이 진행이 되다가 중단되는 위치를 의미한다.

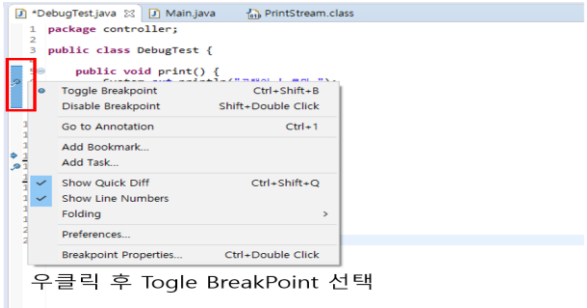
4.2 설정방법

설정방법에는 두가지가 있는데 소스코드 창에서 확인을 원하는 소스코드의 앞쪽 부분에서 더블클릭하는 방법과 우클릭하여 컨택스트메뉴를 이용하는 방법이 있다. 이렇게 설정하게 되면 해당부분의 line부분에 점같이 생긴 것이 나타나면서 BreakPoint가 생성이 되고 한번 더 더블클릭을 누르면 해제가 된다.

더블클릭 이용



우클릭 후 컨택스트 메뉴이용 (disable BreakPoint : 해제)



우클릭 후 Togle BreakPoint 선택

5 디버그 모드 실행

디버그실행 아이콘을 클릭하거나 Run-debug선택, 단축키 F11로 실행

6 for문의 증감값 변경 확인하기

main메소드에서 for문이 실행되는 fortest()메소드를 호출하여 값을 출력하는 코드

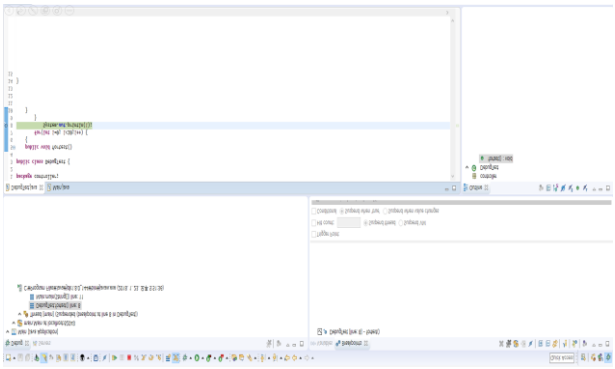
원하는 소스코드 System.out.println(i) 부분에 BreakPoint설정

소스코드

```
1  package main;
2
3  import controller.*;
4
5  public class Main {
6
7      public static void main(String[] args) {
8
9          DebugTest dt=new DebugTest();
10
11         dt.fortest();
12
13     }
14
15 }
16
```

```
1  package controller;
2
3  public class DebugTest {
4
5      public void fortest()
6      {
7          for(int i=0; i<10;i++) {
8              System.out.println(i);
9          }
10     }
11
12
13
14 }
15
```

디버그모드 실행



debug모드가 실행되면 debug모드로 화면을 변경하는지에 대한 여부를 선택하는 메시지 창이 뜨고 yes를 클릭하면그 다음 위와 같은 debug창이 나온다. 우선 debug를 실행하면 프로그램 흐름상 가장 먼저 나오는 BreakPoint지점에서 프로그램 실행이 중단되고 메소드확인창에는 실행된 메소드가 나오게 되고, variables창과 BreakPoint창에서 각각 변수과 BreakPoint리스트가 나온다. 상세화면으로 보면 아래와 같다.

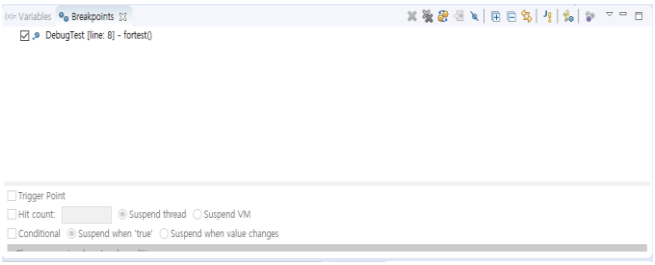
메소드창

메소드 확인 창 main 스레드에서 실행된 메소드인 main()와 fortest()를 보여준다. 프로그램의 흐름에 따라 실행되는 객체와 메소드가 클래스명.메소드방식으로 표시된다.



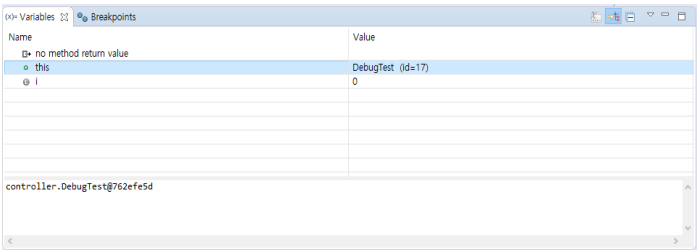
BreakPoint창

BreakPoint창에는 현재 BreakPoint가 설정되어있는 부분을 표시해준다. 현재는 BreakPoint를 DebugTest객체의 fortest메소드 내부 System.out.println에 한 개만 설정이 되어 있어서 하나만 보이는 것을 확인 할 수 있다. 그리고 옆에는 그 BreakPoint의 위치가 line으로 나와있다. 앞에 checkbox가 있는데 이부분을 체크해제하면 설정된 것이 해제 되어 더이상 이곳에서 멈추지 않고 계속 실행된다.



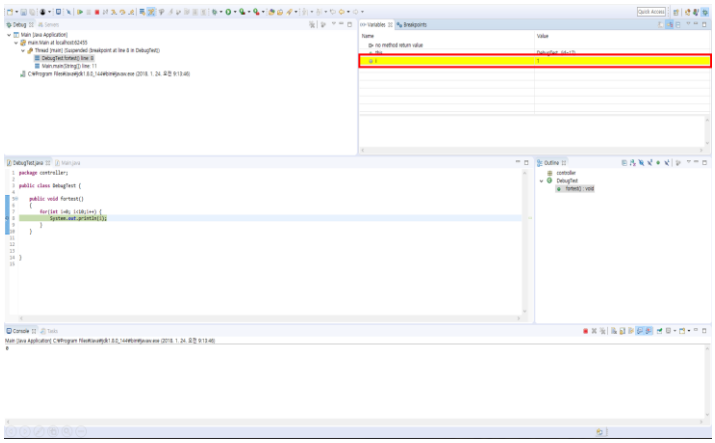
Variables창

Variables창에는 지금 선언되어 있는 객체 및 변수에 대한 정보가 나타난다. This와 i라는값이 보이는데 this는 DebugTest객체이고 value값으로 Id값이 출력되고 아래부분 출력창에는 객체의 주소값이 출력된다. i는 for문에서 선언된 지역변수이다. for이 시작되고 아직 증감연산이 되지 않았기 때문에 Value가 0으로 되어 있다.



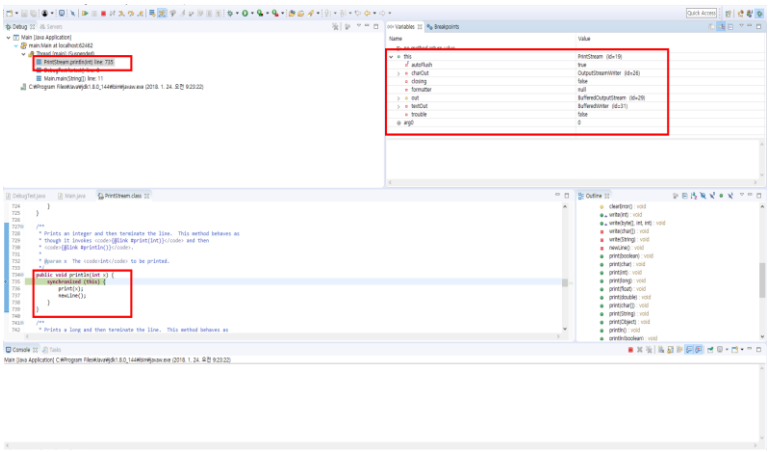
값을 확인하고 프로그램을 계속해서 실행하려면 (F8)를 클릭하고 BreakPoint다음에 실행되는 메소드로 이동하려면 (F5), (F7)을 실행하면 된다. 두개의 차이는 위 메뉴에서 설명했다.

(F8)실행



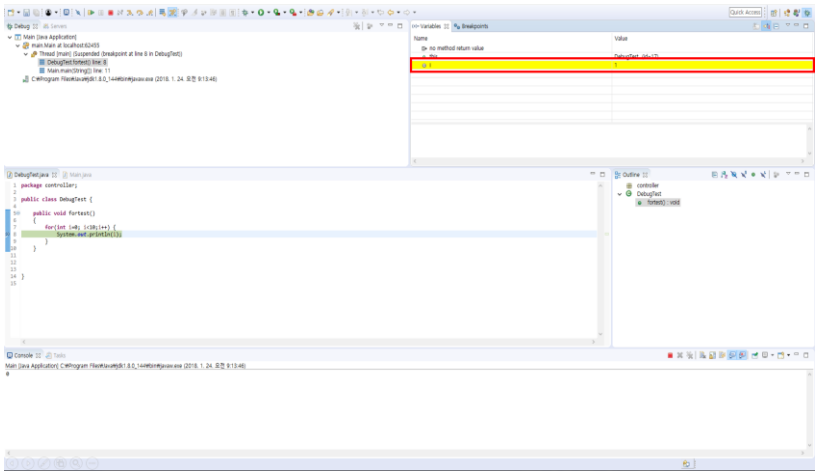
위 결과와 같이 Resume(F8)을 실행을 하면 프로그램이 이어서 실행되면서 다음 BreakPoint에서 실행을 멈추고 그 시점의 저장된 값을 출력하고 프로그램에 대한 결과를 Console에 출력해준다. 출력값과 변수값이 차이가 있는 것은 로직상에서 변수는 이미 증감연산을 했기 때문에 값이 증가하여 1이 출력되고 console창에 출력된 값은 출력된 시점이기 때문에 0으로 표시된 것이다.

(F5) 실행



위 결과와 같이 step into(F5)을 실행을 하면 BreakPoint에서 실행하는 System.out.println()메소드에 내부를 출력해주고 관련값을 확인할 수 있다. 메소드출력창을 보면 PrintStream객체의 print()매소드가 출력된걸 확인 할 수 있고, variables창을 보면 PrintStream객체가 가지고 있는 변수들의 값을 출력하는 것을 볼 수 있다. 다시 Step Into(F5)실행하면 소스코드창에서 볼 때 print(x)가 실행되는 곳으로 넘어가서 그것을 확인할 수 있다. 이후 진행은 각자가 해보자.

(F7) 실행



위 결과와 같이 step over(F6)을 실행을 하면 Step Into와 다른점은 BreakPoint에 있는 내부 메소드까지 확인하는 반면 지금 작성된 작성코드만 프로그램 흐름에 따라 진행된다. 그래서 System.out.println()메소드에는 접근하지 않고 바로 for문이 진행되는 것을 확인할 수 있다.

Chapter 11. BROADCAST CHATTING

server.MultiChatServer.java

```
01  package server;
02
03  import java.io.DataInputStream;
04  import java.io.DataOutputStream;
05  import java.io.IOException;
06  import java.net.ServerSocket;
07  import java.net.Socket;
08  import java.util.*;
09
10  public class MultiChatServer {
11          private HashMap clients;
12
13          public MultiChatServer(){
14                  clients = new HashMap();
15                  Collections.synchronizedMap(clients);
16          }
17
18          class ServerReceiver extends Thread{
19                  private Socket socket;
20                  private DataInputStream in;
21                  private DataOutputStream out;
22
23                  public ServerReceiver(Socket socket){
24                          this.socket = socket;
25                          try {
26                                  this.in = new DataInputStream(socket.getInputStream());
27                                  this.out = new DataOutputStream(socket.getOutputStream());
28                          } catch (IOException e) {
29                                  e.printStackTrace();
30                          }
31                  }
32                  public void run(){
33                          String name = null;
34                          try {
35                                  name = in.readUTF();
```

```
36                                      sendToAll("#" + name + "님이 들어오셨습니다.");
37                                      clients.put(name, out);
38                                      System.out.println("현재 서버접속자 수는 " + clients.size() + "
39  입니다.");
40                                      while(in != null){
41                                              sendToAll(in.readUTF());
42                                      }
43
44                              } catch (IOException e) {
45                                      e.printStackTrace();
46                              }finally {
47                                      sendToAll("#" + name + "님이 나가셨습니다.");
48                                      clients.remove(name);
49                                      System.out.println("현재 접속자 수는 " + clients.size() + "입니
50  다.");
51                              }
52                      }
53              }
54
55              public void start(){
56                      ServerSocket serverSocket = null;
57                      Socket socket = null;
58
59                      try {
60                              serverSocket = new ServerSocket(8888);
61                              System.out.println("서버가 시작 되었습니다.");
62
63                              while(true){
64                                      socket = serverSocket.accept();
65                                      System.out.println("새로운 유저가 로그인 하였습니다.");
66                                      ServerReceiver thread = new ServerReceiver(socket);
67                                      thread.start();
68                              }
69
70                      } catch (IOException e) {
71                              // TODO Auto-generated catch block
72                              e.printStackTrace();
73                      }
74              }
```

```
75
76          public void sendToAll(String msg){
78                  Iterator it = clients.keySet().iterator();
79                  while(it.hasNext()){
80                          try {
81                                  DataOutputStream out = (DataOutputStream)
82  clients.get(it.next());
83                                  out.writeUTF(msg);
84                          } catch (Exception e) {
85                          }
86                  }
87          }
88  }
```

client.ClientReceiver.java

```
01  package client;
02
03  import java.io.*;
04  import java.net.Socket;
05
06  public class ClientReceiver extends Thread{
07          Socket socket;
08          DataInputStream in;
09
10          public ClientReceiver(Socket socket){
11                  this.socket = socket;
12                  try {
13                          in = new DataInputStream(socket.getInputStream());
14                  } catch (IOException e) {
15                          e.printStackTrace();
16                  }
17          }
18
19          public void run(){
20                  while(in != null){
21                          try {
22                                  System.out.println(in.readUTF());
23                          } catch (IOException e) {
24                                  e.printStackTrace();
25                          }
```

```
26                      }
27              }
28      }
```

client.ClientSender.java

```
01      package client;
02
03      import java.io.DataOutputStream;
04      import java.io.IOException;
05      import java.net.Socket;
06      import java.util.Scanner;
07
08      public class ClientSender extends Thread{
09              Socket socket;
10              DataOutputStream out;
11              String name;
12
13              public ClientSender(Socket socket, String name){
14                      this.socket = socket;
15                      this.name = name;
16                      try {
17                              this.out = new DataOutputStream(socket.getOutputStream());
18                      } catch (IOException e) {
19                              e.printStackTrace();
20                      }
21              }
22
23              public void run(){
24                      Scanner sc = new Scanner(System.in);
25
26                      if(out != null){
27                              try {
28                                      out.writeUTF(name);
29                                      while(out != null){
30                                              System.out.print("대화 입력 : ");
31                                              out.writeUTF("[" + name + "]" +   sc.nextLine());
32                                      }
33
34                              } catch (IOException e) {
35                                      e.printStackTrace();
```

```
36                                  }
37                          }
38                  }
39  }
```

client.ClientServer.java

```
01  package client;
02
03  import java.io.IOException;
04  import java.net.*;
05  import java.util.Scanner;
06
07  public class ClientServer {
08          public ClientServer(){
09                  String serverIp = "127.0.0.1";
10                  Scanner sc = new Scanner(System.in);
11                  System.out.print("대화명을 입력하세요 : ");
12                  String name = sc.nextLine();
13
14                  try {
15                          Socket socket = new Socket(serverIp, 8888);
16                          System.out.println("서버에 연결 되었습니다.");
17                          Thread sender = new Thread(new ClientSender(socket, name));
18                          Thread receiver = new Thread(new ClientReceiver(socket));
19                          sender.start();   //receiver.start();
20
21                  } catch (UnknownHostException | IOException e) {
22                          e.printStackTrace();
23                  }
24          }
25  }
```

client.Run.java

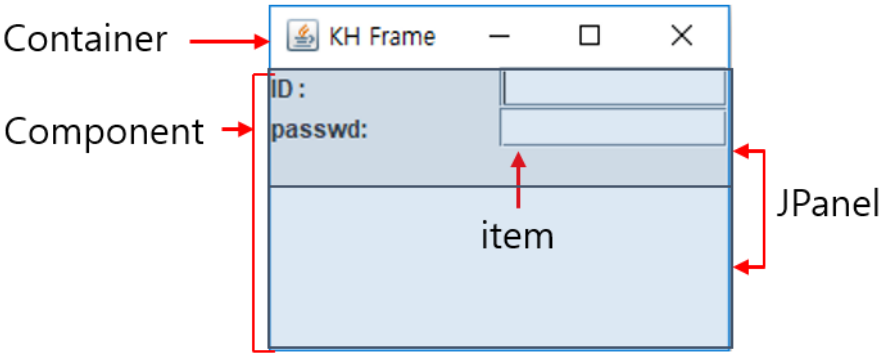
```
01  package client;
02
03  public class Run {
04          public static void main(String[] args) {
05                  ClientServer cs = new ClientServer();
06          }
07  }
```

Part 4. 프로그램 최적화하기

Chapter 12. 자바 GUI

1 GUI

자바에서는 그래픽을 이용하여 사용자에게 이해하기 쉬운 모양으로 정보를 제공하는 프로그램을 작성할 수 있다. GUI 프로그램을 구현 시 유의사항이 있는데, 그것은 바로 GUI 설계의 순서이다. 처음 바탕이 되는 Frame이라는 컨테이너를 만들고 그 안에 요소를 하나씩 추가해 간다고 생각을 하면 이후 구현을 직접 수행할 때 이해하기 더 쉬울 것이다.



1.1 GUI 용어 정리

1.1.1 컨테이너(Container)

자바에서 창(Window)의 역할을 한다. 컨테이너에 컴포넌트들이 올려지게 된다.

ex) Frame, Window, Panel, Dialog, Applet

1.1.2 컴포넌트(Component)

컨테이너 위에 올려지며 화면을 구성하는 요소

ex) Button, TextField, Lable 등

1.1.3 배치 관리자(LayoutManager)

컨테이너에 컴포넌트가 올라갈 때 자리를 어떻게 배치할 건지 결정하는 요소

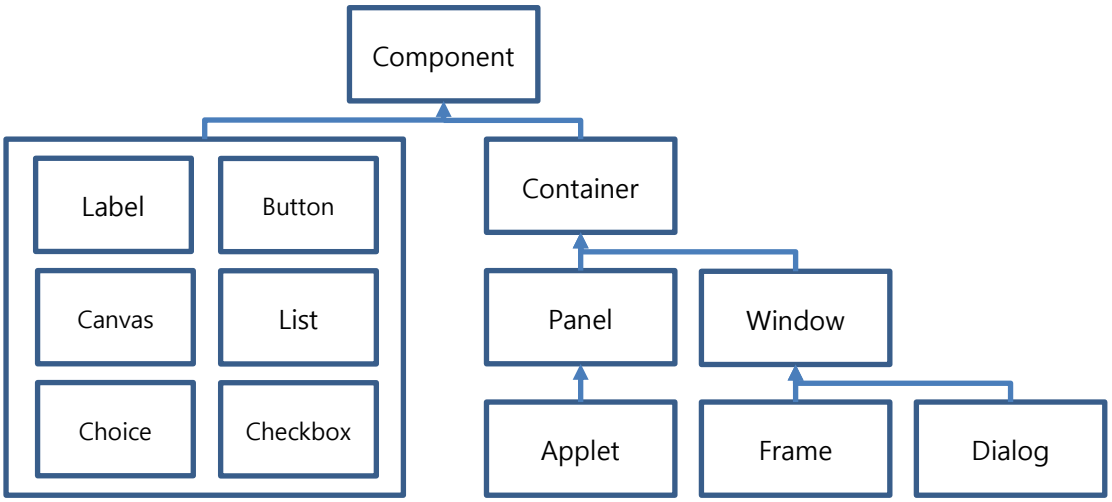
ex) FlowLayout, GridLayout, BorderLayout 등

2 AWT(Abstract Window Toolkit)

초기의 자바에서 배포된 GUI 라이브러리

java.awt 패키지에 포함되어 있으며 Frame, Window, Panel, Button, Label 등의 컴포넌트를 가진다. 운영체제의 자원을 이용하여 그래픽을 표현하며 운영체제와 프로그램 사이의 연결 라이브러리로 볼 수 있다. 이 때문에 운영체제에 따라서 프로그램의 모양이 조금씩 달라질 수 있다.

- AWT 패키지의 기본 구조



- AWT로 작성한 간단한 GUI 프로그램

myGui.MyFrame.java

```
01  package myGui;
02
03  import java.awt.*;
04  import java.awt.event.*;
05
06  // AWT 화면을 구성하는 클래스
07  public class MyFrame {
08      private Frame fr; // 메인 화면 객체
09      private Label label;
10      private Panel panel;
11
12      public MyFrame() {
13          initFrame();
14      }
15
16      // 메인 화면에 대한 설정
17      private void initFrame() {
```

```
18      fr = new Frame("First GUI");
19      fr.setSize(300,300);
20      fr.setLayout(new GridLayout(3, 1));
21      fr.setVisible(true);
22
23      // GUI 프로그램 종료이벤트 처리
24      fr.addWindowListener(new WindowAdapter() {
25          @Override
26          public void windowClosing(WindowEvent e) {
27              System.exit(0);
28          }
29      });
30
31      // 간단한 구문을 표현하는 label 객체
32      label = new Label();
33      label.setText("Status Lable");
34      label.setAlignment(Label.CENTER);
35      label.setSize(200, 100);
36
37      // 버튼을 담고 있을 panel 객체
38      panel = new Panel();
39      panel.setLayout(new FlowLayout());
40
41      // panel과 label을 frame에 추가하고 화면에 보이도록 함
42      fr.add(panel);
43      fr.add(label);
44      fr.setVisible(true);
45  }
46
47  // 버튼을 생성하고 클릭 이벤트를 처리하는 메소드
48  void showButton() {
49      Button btnOk = new Button("OK");
50      Button btnCancel = new Button("Cancel");
51
52      // btnOk 버튼을 클릭할 경우 label에 "확인" 문구 출력
53      btnOk.addActionListener(new ActionListener() {
54          public void actionPerformed(ActionEvent e) {
55              label.setText("확인");
56          }
```

```
57          });
58
59          // btnCancel 버튼을 클릭할 경우 label에 "취소" 문구 출력
60          btnCancel.addActionListener(new ActionListener() {
61              public void actionPerformed(ActionEvent e) {
62                  label.setText("취소");
63              }
64          });
65
66          // panel 객체에 2개의 버튼을 추가하고 frame에 출력
67          panel.add(btnOk);
68          panel.add(btnCancel);
69          fr.setVisible(true);
70      }
71  }
```

myGui.ExGUI.java

```
01  package myGui;
02
03  public class ExGUI {
04      public static void main(String[] args) {
05          MyFrame mfr = new MyFrame();
06          mfr.showButton();
07      }
08  }
```

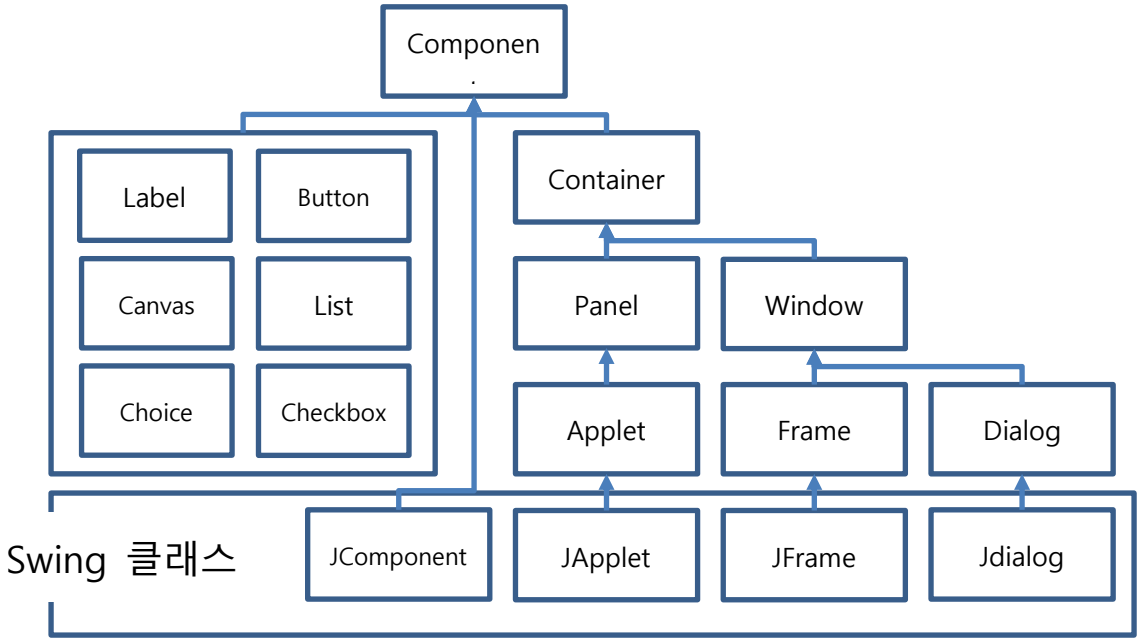
3 스윙(Swing)

AWT 기술을 기반으로 작성된 자바의 라이브러리

모든 AWT의 기능을 포함하고 있으며 스윙에서 추가된 화려한 고급 컴포넌트를 사용할 수 있게 되었다. 스윙의 클래스들은 'J'로 클래스명이 시작된다. ex) JFrame, JWindow, JPanel, JButton 등

순수 자바 언어로 구현되었으며 JDK 1.1 부터 제공된다. javax.swing 패키지에 라이브러리를 가지고 있다. 스윙 컴포넌트들은 AWT와 다르게 운영체제에 의존적이지 않아 운영체제에 관계없이 항상 동일한 작동을 하며 동일한 모양을 유지한다.

- Swing 클래스 기본 구조



스윙 클래스는 AWT 라이브러리의 상속을 받고 있다.

3.1 스윙 컨테이너와 컴포넌트

3.1.1 컨테이너(Container)

다른 컴포넌트를 포함할 수 있는 GUI 컴포넌트로써 java.awt.Container를 상속받는다.

다른 컨테이너에 포함될 수 있으며 Jpanel, JFrame, JApplet, JDialog, JWindow가 있다.

3.1.2 최상위 컨테이너

다른 컨테이너에 속하지 않고 독립적으로 존재 가능한 컨테이너

모든 컴포넌트는 컨테이너에 포함되어야 화면에 출력 가능하지만 JFrame, JDialog, JApplet는 독립적으로 화면에 출력 가능한 최상위 컨테이너이다.

3.1.3 컴포넌트(Component)

컨테이너에 포함되어야 화면에 출력될 수 있는 GUI 객체

javax.swing.JComponent의 상속을 받고 있으며 스윙에서는 JButton, JLabel, JTextField, JList, JTable, JToolBar 등 다양한 컴포넌트를 제공한다.

3.2 스윙 컴포넌트와 AWT 컴포넌트의 차이점

- 스윙 컴포넌트는 'J' 로 시작한다 (Button, JButton)
- AWT컴포넌트는 java.awt 패키지안에, Swing 컴포넌트는 javax.swing 패키지 내에 존재한다
- 모든 스윙 컴포넌트는 javax.swing.Jcomponent를 상속한다
- 스윙의 버튼은 텍스트뿐만 아니라 이미지도 표시 할 수 있다
- 스윙 컴포넌트는 직사각형 이외의 모양도 가능하다 (둥근 모서리 버튼)
- 스윙에서는 Border를 그릴수 있다. AWT는 Peer(운영체제의 네이티브코드)를 거치므로 모양에 관여할수 없었지만, 스윙은 내재된 컴포넌트를 사용하므로 GUI를 직접 구현할 수 있어 Border를 쉽게 그릴수 있다.
- 컴포넌트의 최대/최소 크기와 가장 적절한 크기를 지정 할 수 있다. AWT에서는 Peer에 의해 제어되므로 마음대로 크기를 제어하기가 어렵다
- ToolTip을 만들기가 쉽다 (Jcomponent에 ToolTip에 대한 메소드가 존재)
- 투명 Component가 가능하다 (setOpaque() 이용하여 설정가능)

4 스윙 프레임(JFrame)

모든 스윙 컴포넌트를 담는 최상위 GUI 컨테이너

JFrame을 상속받아 구현한다. 컴포넌트가 화면에 보이려면 스윙 크레임에 올라가야 하며 프레임을 닫으면 프레임 내의 모든 컴포넌트가 보이지 않게 된다.

4.1 프레임 만들기

프레임을 만드는 방법에는 두가지가 있다. main 메소드에서 JFrame 객체를 생성하는 방법과 JFrame 클래스를 상속받아 클래스를 작성하는 방법이다.

main 메소드에서 JFrame 객체를 생성하는 경우 별도의 클래스를 작성하지 않고 간단하게 프레임을 만들 수 있는 장점이 있지만 확장성과 융통성이 결여되므로 많이 사용하지 않는 방법이다.

JFrame클래스를 상속받는 클래스를 작성하여 프레임을 만드는 방법은 추가적인 클래스를 작성해야 하지만 프레임에 대한 확장과 수정이 용이하여 main 메소드에서 직접 생성하는 방법보다 많이 사용되는 방법이다. 프레임 클래스는 프레임에 대한 설정과 컴포넌트에 대한 관리를 담당하며 main 메소드는 프레임객체를 생성하는 단순한 역할만 하게 된다.

- 기본 프레임 생성 1

  main 메소드에서 JFrame 객체 생성

swingGUI.FrameEx.java

```
01  package swingGUI;
02
03  import javax.swing.JFrame;
04
05  public class FrameEx {
06      public static void main(String[] args) {
07          JFrame jfr = new JFrame("Swing GUI 프로그램"); // JFrame 객체 생성
08          jfr.setSize(300, 300); // JFrame 크기 설정
09          jfr.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 종료동작 설정
10          jfr.setVisible(true); // JFrame 활성화
11      }
12  }
```

- 기본 프레임 생성 2 (프레임 생성에 일반적으로 사용되는 방법)

  JFrame 객체를 상속받는 스윙 프레임 클래스 작성

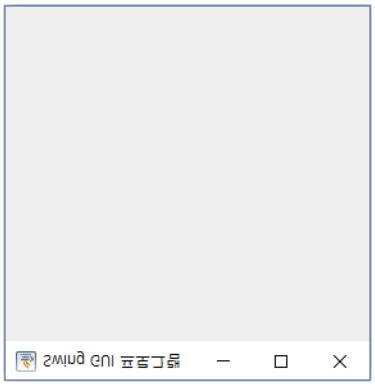
swingGUI.MyFrame.java

```
01  package swingGUI;
02
03  import javax.swing.JFrame;
04
05  public class MyFrame extends JFrame{
06      public MyFrame() {
07          JFrame jfr = new JFrame("Swing GUI 프로그램"); // JFrame 객체 생성
08          jfr.setSize(300, 300); // JFrame 크기 설정
09          jfr.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 종료동작 설정
10          jfr.setVisible(true); // JFrame 활성화
11      }
12  }
```

swingGUI.Main.java

```
01  package swingGUI;
02
03  import swingGUI.MyFrame;
04
05  public class Main {
06      public static void main(String[] args) {
07          new MyFrame();
```

```
08      }
09  }
```



4.2 프레임생성 후 컴포넌트 추가하기

화면 생성은 프로그램의 동작 초기에 이루어지는 것이 일반적이므로 주로 생성자에서 작업이 이루어진다.

```
// 컨테이너 위에 컴포넌트 추가
프레임.add(Component component);

// 크기 지정
프레임.setSize(int x, int y);

// 컨테이너 보이기
프레임.setVisible(Boolean b);
```

4.3 배치 관리자 (LayoutManager)

스윙의 컨테이너 계열의 컴포넌트들은 다른 컴포넌트를 포함할 수 있는데 이 때 컴포넌트를 포함할 때 어떠한 배치를 따를 것인지에 대한 방법을 레이아웃이라고 한다. 레이아웃을 지정하는 클래스를 배치관리자(LayoutManager)라고 하며 5가지 종류가 있다.

배치 관리자	설 명
FlowLayout	컴포넌트를 컨테이너의 왼쪽부터 차례대로 하나씩 채우고 오른쪽 끝에 다다르면 다음 줄로 넘어가는 방식
BorderLayout	프레임의 영역을 동, 서, 남, 북, 중앙으로 다섯 개의 영역으로 나누어 각 위치에 컴포넌트를 배치
GridLayout	주어진 열과 행의 크기로 컨테이너 영역을 나누고 동등한 크기의 각 셀에 컴포넌트를 하나씩 배치
CardLayout	여러장의 카드를 겹쳐 놓은 방식으로 컴포넌트를 배치
GridBagLayout	기본적으로 GridLayout과 비슷. 하나의 컴포넌트가 하나의 셀이 아닌 여러 셀을 차지할 수 있다.

주로 FlowLayout, BorderLayout, GridLayout을 사용하며 컨테이너들은 각각의 디폴트 배치관리자를 가지고 있다.

- 각 컨테이너의 기본 배치관리자

AWT와 스윙의 컨테이너	디폴트 배치관리자
Window	BorderLayout
Frame, JFrame	BorderLayout
Dialog, JDialog	BorderLayout
Panel, JPanel	FlowLayout
Applet, JApplet	FlowLayout

컨테이너의 레이아웃을 변경하고 싶다면 원하는 배치관리자로 변경한 후 배치관리자에 맞게 설정을 하고 컴포넌트를 추가한다.

- 컨포넌트의 레이아웃 변경 메소드

```
setLayout(LayoutManager lm);
```

4.3.1 FlowLayout

- FlowLayout의 생성자

생성자	설명
FlowLayout()	가운데 정렬. hGap과 vGap이 5px인 FlowLayout 생성
Flowlayout(int align)	정렬방법 지정하여 hGap과 vGap이 5px인 FlowLayout 생성
Flowlayout(int align, int hGap, int vGap)	정렬방법과, hGap, vGap을 지정하여 FlowLayout 생성

align : 컴포넌트 정렬방법 (FlowLayout.LEFT, FlowLayout.RIGHT, FlowLayout.CENTER(기본)로 지정)

hGap : 각 컴포넌트 간의 수평 간격 (디폴트 5px)

vGap : 각 컴포넌트 간의 수직 간격 (디폴트 5px)

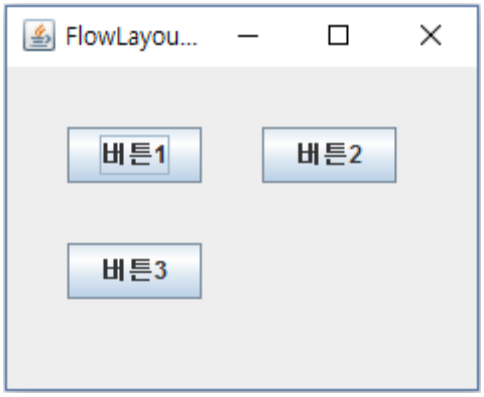
- FlowLayout 예제

swingGUI.FlowLayoutEx.java

```
01  package swingGUI;
02
03  import java.awt.FlowLayout;
04  import javax.swing.JButton;
05  import javax.swing.JFrame;
06
07  public class FlowLayoutEx extends JFrame {
08    // 버튼 생성
09    JButton b1 = new JButton("버튼1");
10    JButton b2 = new JButton("버튼2");
11    JButton b3 = new JButton("버튼3");
12
13    public FlowLayoutEx() {
14      setTitle("FlowLayout 예제");
15      setSize(250, 200);
16      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17
18      setLayout(new FlowLayout(FlowLayout.LEFT, 30, 30)); // 배치 관리자 변경
19
20      // 버튼 추가
21      add(b1);
22      add(b2);
23      add(b3);
24
25      setVisible(true);
26    }
27  }
```

swingGUI.Main.java

```
01  package swingGUI;
02
03  import swingGUI.FlowLayoutEx;
04
05  public class Main {
06    public static void main(String[] args) {
07      new FlowLayoutEx();
08    }
09  }
```



4.3.2 BorderLayout

- BorderLayout의 생성자

생성자	설명
BorderLayout()	영역사이의 간격이 없는 BorderLayout 생성
BorderLayout(int hGap, int vGap)	각 영역 사이에 간격을 지정하여 BorderLayout 생성

hGap : 좌우 두 컴포넌트 사이의 수평 간격 (디폴트 0px)

vGap : 상하 두 컴포넌트 사이의 수직 간격 (디폴트 0px)

- BorderLayout 예제

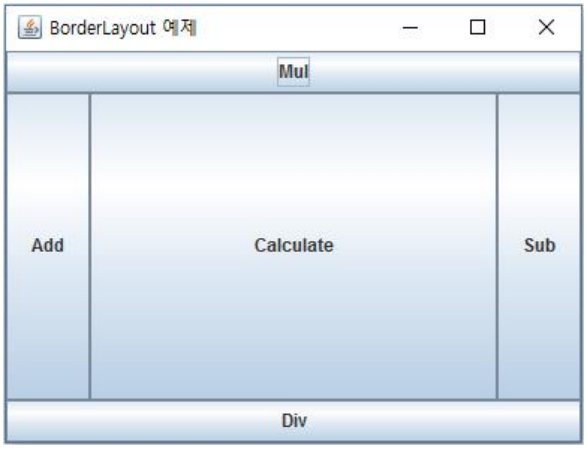
swingGUI.BorderLayoutEx.java

```
01  package swingGUI;
02
03  import java.awt.BorderLayout;
04
05  import javax.swing.JButton;
06  import javax.swing.JFrame;
07
08  public class BorderLayoutEx extends JFrame {
09    // 버튼 생성
10    JButton btnAdd = new JButton("Add");
11    JButton btnSub = new JButton("Sub");
12    JButton btnMul = new JButton("Mul");
13    JButton btnDiv = new JButton("Div");
14    JButton btnCalc = new JButton("Calculate");
15
16    public BorderLayoutEx() {
17      setTitle("BorderLayout 예제");
```

```
18        setSize(400, 300);
19        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
20
21        setLayout(new BorderLayout()); // 배치 관리자 설정
22
23        // 버튼 추가
24        add(btnSub, "East");
25        add(btnAdd, "West");
26        add(btnDiv, "South");
27        add(btnMul, "North");
28        add(btnCalc, "Center");
29
30        setVisible(true);
31    }
32 }
```

swingGUI.Main.java

```
01 package swingGUI;
02
03 import swingGUI.BorderLayoutEx;
04
05 public class Main {
06    public static void main(String[] args) {
07       new BorderLayoutEx();
08    } }
```



4.3.3 GridLayout

- GridLayout의 생성자

생성자	설명
GridLayout()	1 row 1 col의 GridLayout생성
GridLayout(int rows, int cols)	영역들 간의 간격 없이 행과 열의 개수를 지정하여 GridLayout 생성
GridLayout(int rows, int cols, int hGap, int vGap)	행과 열의 개수를 지정하고 간격을 지정하여 GridLayout 생성

rows : 컨테이너를 나눌 행의 개수

cols : 컨테이너를 나눌 열의 개수

hGap : 각 컴포넌트 간의 수평 간격

vGap : 각 컴포넌트 간의 수직 간격

- GridLayout 예제

swingGUI.GridLayoutEx.java

```
01  package swingGUI;
02
03  import java.awt.FlowLayout;
04  import java.awt.GridLayout;
05
06  import javax.swing.JButton;
07  import javax.swing.JFrame;
08
09  public class GridLayoutEx extends JFrame {
10    // 버튼 생성
11    JButton b1 = new JButton("버튼1");
12    JButton b2 = new JButton("버튼2");
13    JButton b3 = new JButton("버튼3");
14    JButton b4 = new JButton("버튼4");
15    JButton b5 = new JButton("버튼5");
16
17    public GridLayoutEx() {
18      setTitle("GridLayout 예제");
19      setSize(200, 300);
20      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
21
22      setLayout(new GridLayout(3, 2)); // 3행 2열 GridLayout 설정
```

```
23
24          // 버튼 추가
25          add(b1);
26          add(b2);
27          add(b3);
28          add(b4);
29          add(b5);
30
31          setVisible(true);
32      }
33  }
```

swingGUI.Main.java

```
01  package swingGUI;
02
03  import swingGUI.GridLayoutEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new GridLayoutEx();
08      }
09  }
```



4.3.4 배치 관리자 없는 컨테이너

컨테이너의 배치 관리자를 제거하여 컴포넌트의 위치를 직접 지정하는 방식

- 배치 관리자 제거 방법

```
컨테이너.setLayout(null); // 배치관리자를 null 로 설정한다
```

- 컴포넌트 크기 설정

```
컴포넌트.setSize(int width, int height);
```

- 컴포넌트 위치 설정

```
컴포넌트.setLocation(int x, int y);
```

- 컴포넌트 위치와 크기 동시 설정

```
컴포넌트.setBounds(int x, int y, int width, int height);
```

- 배치 관리자 없는 컨테이너 예제

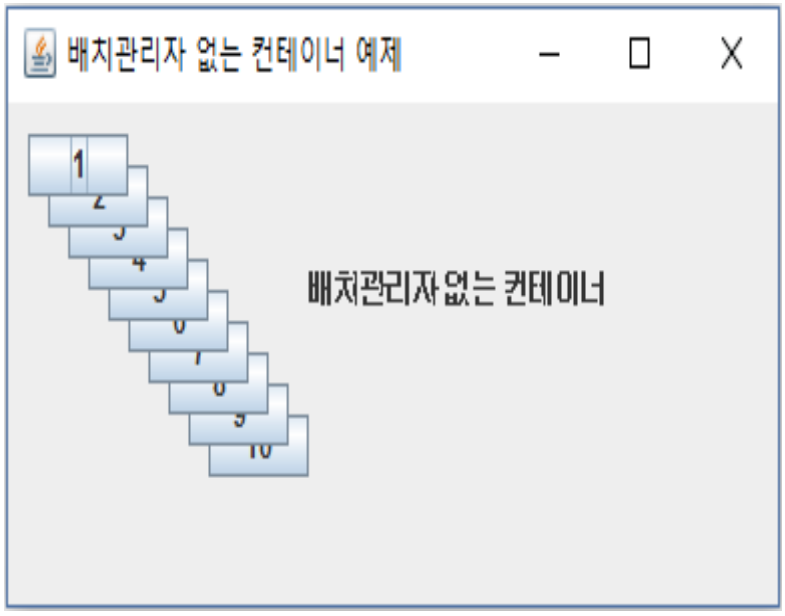
swingGUI.NullLayoutManagerEx.java

```
01  package swingGUI;
02
03  import javax.swing.JButton;
04  import javax.swing.JFrame;
05  import javax.swing.JLabel;
06
07  public class NullLayoutManagerEx extends JFrame {
08     public NullLayoutManagerEx() {
09        setTitle("배치관리자 없는 컨테이너 예제");
10        setSize(400, 200);
11        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12
13        setLayout(null); // 배치 관리자 제거
14
15        JLabel label = new JLabel("배치관리자 없는 컨테이너");
16        label.setLocation(150, 50); // 컨테이너 위치 설정
17        label.setSize(200, 20); // 컨테이너 크기 설정
18
19        for(int i=1; i<=10; i++){
```

```
20          JButton b = new JButton(Integer.toString(i));
21          b.setBounds(i*10, i*10, 50, 20); // 컨테이너의 크기와 위치를 한번에 설정
22          add(b);
23       }
24
25       add(label);
26       setVisible(true);
27    }
28 }
```

swingGUI.Main.java

```
01 package swingGUI;
02
03 import swingGUI.NullLayoutManagerEx;
04
05 public class Main {
06    public static void main(String[] args) {
07       new NullLayoutManagerEx();
08    }
09 }
```



4.4 복합 배치 관리자

복합 배치관리자는 Panel을 활용하여 하나의 프레임에 둘 이상의 배치 관리자를 적용하는 것을말한다

- 복합 배치 관리자 예제

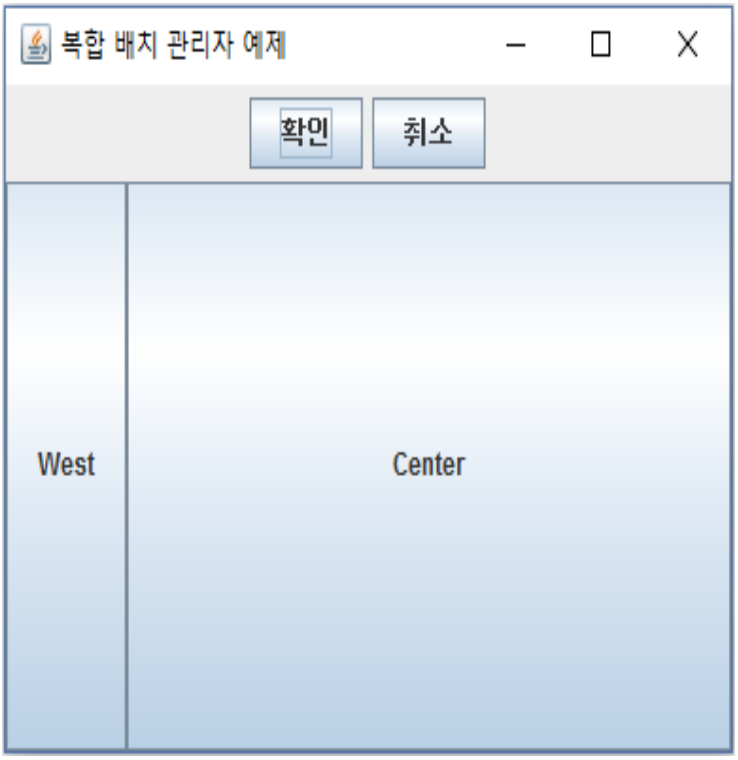
swingGUI.CompositeLayoutEx.java

```
01  package swingGUI;
02
03  import javax.swing.JButton;
04  import javax.swing.JFrame;
05  import javax.swing.JPanel;
06
07  public class CompositeLayoutEx extends JFrame {
08     JButton b1 = new JButton("West");
09     JButton b2 = new JButton("Center");
10
11     // JPanel에 버튼 2개를 추가하여 구성할 예정
12     JPanel jp = new JPanel();
13     JButton b3 = new JButton("확인");
14     JButton b4 = new JButton("취소");
15
16     public CompositeLayoutEx() {
17        setTitle("복합 배치 관리자 예제");
18        setSize(400, 300);
19
20        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
21
22        // JPanel에 b3, b4 버튼 추가
23        jp.add(b3);
24        jp.add(b4);
25
26        // JPanel로 묶인 b3, b4와 b1, b2 버튼을 추가
27        // JPanel의 기본 배치 관리자 : FlowLayout
28        // JFrame의 기본 배치 관리자 : BorderLayout
29        add(jp, "North");
30        add(b1, "West");
31        add(b2, "Center");
32
33        setVisible(true);
```

```
34      }
35  }
```

swingGUI.Main.java

```
01  package swingGUI;
02
03  import swingGUI.CompositeLayoutEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new CompositeLayoutEx();
08      }
09  }
```



4.5 스윙 프로그램의 종료

스윙 응용프로그램 내에서 강제 종료코드인 System.exit(0); 코드를 수행한다면 스윙 응용프로그램은 언제 어디서나 무조건 종료시킬 수 있다.

그렇지만 GUI 프로그램에서의 종료는 코드로 종료하는 것이 아닌 프레임의 종료버튼(X)를 클릭하거나 Alt + F4와 같은 단축키를 이용하여 종료 이벤트를 발생시켜 프로그램을 종료하게 된다. 스윙 응용프로그램에서 프레임의 종료 버튼을 클릭했을 경우 프레임 윈도우가 닫히는 것을 볼 수 있다. 종료 버튼을 클릭했을 때 프레임에 닫히게 되고 추가적으로 응용프로그램이 종료되게 된다.

스윙 응용프로그램의 종료에는 주의할 사항이 한 가지 숨어있다. 위의 프레임 생성코드에서 jfr.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); 코드는 스윙 응용프로그램의 프레임 종료버튼(X)을 클릭했을 때의 작동을 설정하고 있는 코드이다. 해당 코드를 작성하지 않은 프로그램을 종료시켰을 경우 프레임이 닫히는 동작은 같지만 프로세스 내역을 확인하면 자바 응용프로그램이 종료되지 않고 그대로 남아있는 것을 확인할 수 있다.

자바의 응용프로그램은 해당 프로그램에서 생성된 스레드가 하나도 없을 때 종료되는 특징을 가지고 있다. 스윙 응용프로그램이 실행 되면 자바 응용프로그램의 실행을 시작한 메인 스레드(main 메소드가 실행되는 스레드)와 스윙 응용프로그램의 이벤트를 처리하는 스레드가 생성된다. 스윙 응용프로그램의 프레임의 종료버튼을 클릭하면 메인스레드는 종료되지만 프레임에 대한 이벤트를 처리하기 위한 이벤트 처리 스레드가 살아 있어 자바 응용프로그램이 종료되지 못 하고 남아있는 것이다. 프레임 종료버튼을 클릭하면 프레임 윈도우가 닫히지만 응용프로그램이 종료된 것은 아니기 때문에 setVisible(true)를 호출하면 프레임 윈도우가 다시 활성화되며 닫히기 전처럼 잘 동작하게 된다.

프레임 종료버튼이 클릭될 때 프레임이 닫히고 응용프로그램 또한 종료되도록 하는 코드인 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); 코드를 반드시 추가해주어야 한다.

4.6 dispose vs exit

응용프로그램을 종료시키는 방법으로 dispose와 exit가 있다. exit는 응용프로그램을 종료시키는 방법으로 응용프로그램에서 실행된 모든 프레임을 닫고 응용프로그램을 종료시킨다. dispose는 해당 프레임을 닫으며 다른 프레임이 활성화되어 있다면 응용프로그램을 종료시키지 않는다. 만약 dispose 후 활성화되어있는 프레임이 없다면 응용프로그램까지 종료시키게 된다.

- dispose 와 exit 비교

swingGUI.MainFrame.java

```
01  package swingGUI;
02
03  import java.awt.event.ActionEvent;
04  import java.awt.event.ActionListener;
05
```

```
06  import javax.swing.JButton;
07  import javax.swing.JFrame;
08
09  public class MainFrame extends JFrame {
10      public MainFrame() {
11          setTitle("MainFrame");
12          setSize(400, 110);
13          setLayout(null); // 기본 배치관리자 무시
14          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 기본 종료 동작 설정. 전체종료
15
16          // dispose로 종료 동작하는 버튼
17          JButton btnDispose = new JButton("Dispose");
18          btnDispose.addActionListener(new ActionListener() {
19              public void actionPerformed(ActionEvent arg0) {
20                  dispose(); // 프레임 닫기
21              }
22          });
23          btnDispose.setBounds(10, 10, 100, 50); // 위치와 크기 설정
24          add(btnDispose); // 프레임에 버튼 추가
25
26          // exit 로 종료 동작하는 버튼
27          JButton btnExit = new JButton("Exit");
28          btnExit.addActionListener(new ActionListener() {
29              public void actionPerformed(ActionEvent arg0) {
30                  System.exit(0); // 응용프로그램 종료
31              }
32          });
33          btnExit.setBounds(120, 10, 100, 50); // 위치와 크기 설정
34          add(btnExit); // 프레임에 버튼 추가
35
36          // 다른 프레임 활성화 버튼
37          JButton btnShow = new JButton("Another Frame");
38          btnShow.addActionListener(new ActionListener() {
39              public void actionPerformed(ActionEvent arg0) {
40                  new AnotherFrame();
41              }
42          });
43          btnShow.setBounds(230, 10, 140, 50); // 위치와 크기 설정
44          add(btnShow); // 프레임에 버튼 추가
```

```
45
46          setVisible(true); // 프레임 활성화
47      }
48  }
```

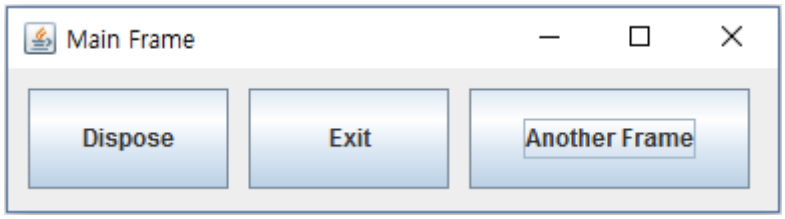
swingGUI.AnotherFrame.java

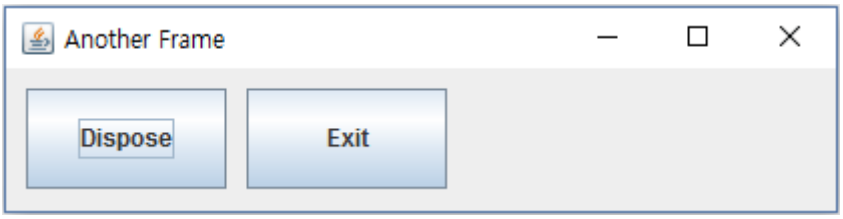
```
01  package swingGUI;
02
03  import java.awt.event.ActionEvent;
04  import java.awt.event.ActionListener;
05  import javax.swing.JButton;
06  import javax.swing.JFrame;
07
08  public class AnotherFrame extends JFrame {
09      public AnotherFrame() {
10          setTitle("AnotherFrame");
11          setSize(430, 110);
12          setLocation(0, 120); // 프레임 위치 설정
13          setLayout(null); // 기본 배치관리자 무시
14          setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // 기본 종료 동작 설정. 프레임종료
15
16          // dispose로 종료 동작하는 버튼
17          JButton btnDispose = new JButton("Dispose");
18          btnDispose.addActionListener(new ActionListener() {
19              public void actionPerformed(ActionEvent arg0) {
20                  dispose(); // 프레임 닫기
21              }
22          });
23          btnDispose.setBounds(10, 10, 100, 50);
24          add(btnDispose);
25
26          // exit 로 종료 동작하는 버튼
27          JButton btnExit = new JButton("Exit");
28          btnExit.addActionListener(new ActionListener() {
29              public void actionPerformed(ActionEvent arg0) {
30                  System.exit(0); // 응용프로그램 종료
31              }
32          });
33          btnExit.setBounds(120, 10, 100, 50);
```

```
34          add(btnExit);
35
36          setVisible(true);
37      }
38  }
```

swingGUI.Main.java

```
01  package swingGUI;
02
03  import swingGUI.MainFrame;
04
05  public class Main {
06      public static void main(String[] args) {
07          new MainFrame();
08      }
09  }
```





MainFrame 의 Another Frame 버튼을 눌러 두 번째 프레임을 활성화 시킨 상태에서 dispose 버튼을 누를 경우 프로그램이 종료되지 않고 해당 프레임만 닫히는 것을 확인할 수 있다. 반대로 exit 버튼을 누를 경우 활성화 되어있는 모든 프레임이 닫히면서 프로그램이 종료된다.

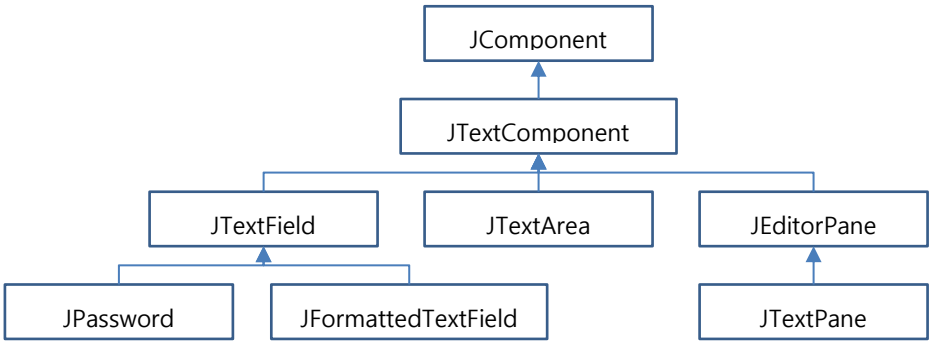
5 스윙 컴포넌트

5.1 텍스트 컴포넌트

스윙에서 텍스트를 나타내는 컴포넌트는 JTextField, JTextArea, JPasswordField, JLabel 등이 있다

모두 javax.swing.textJTextComponent 클래스를 상속 받고 있다.

- JTextComponent 클래스의 상속구조



5.1.1 JTextField 클래스

한줄짜리 텍스트 입력용 컴포넌트

ActionEvent에 의한 이벤트처리를 한다

5.1.2 JTextArea 클래스

여러 줄 텍스트 입력용 컴포넌트

AWT의 TextArea는 스크롤바가 자동으로 생기지만 스윙의 JTextArea는 스크롤바가 자동으로 생기지 않아 스크롤바를 붙여주어야 한다. JScrollPane 클래스 객체를 사용해서 스크롤바를 붙여줄 수 있다.

5.1.3 JPasswordField 클래스

JTextField 클래스를 상속받는 컴포넌트로써 암호를 입력할 때 편리하게 사용할 수 있다

입력한 값을 기본적으로 "*"로 표시한다

5.1.4 JFormattedTextField 클래스

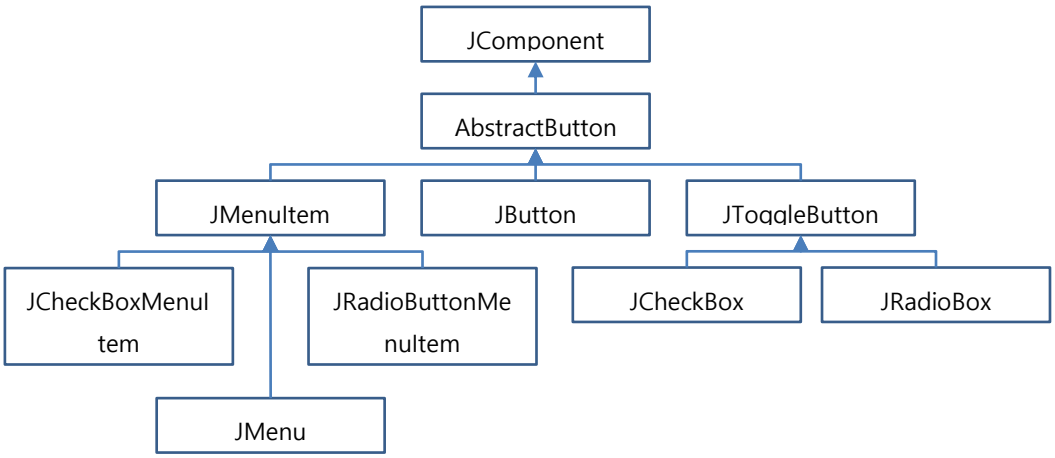
지정된 객체에 따르는 형식으로 형식화 해주거나 그에 맞게 입력을 유도할 수 있는 컴포넌트

5.2 버튼 컴포넌트

AbstractButton 클래스로부터 파생되며 다양한 기능을 가지고 있다

버튼의 내용을 기본적으로 텍스트로 넣을 수 있으며 이미지 또한 버튼의 내용으로 채우는 것이 가능하다

- 버튼 상속구조

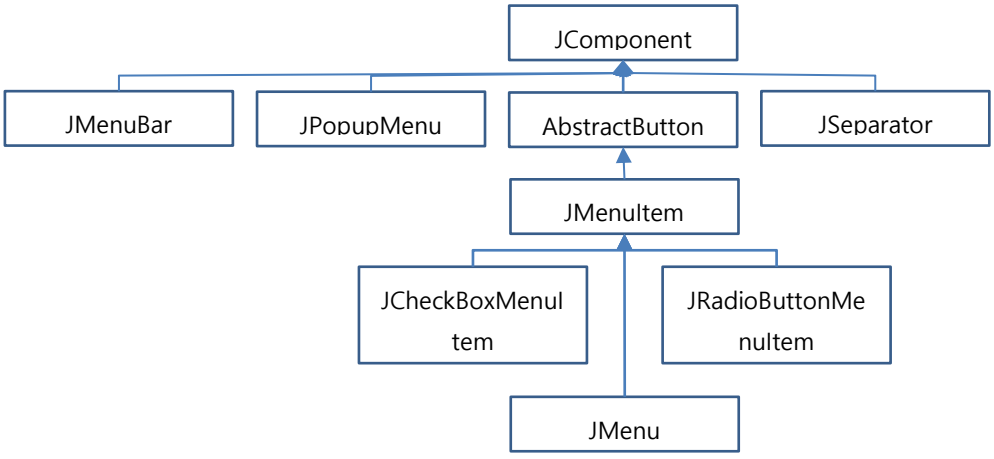


JButton 컴포넌트가 많이 사용되며 JMenuItem을 상속받고 있는 컴포넌트들은 메뉴에서 사용된다

5.3 메뉴 컴포넌트

메뉴에 관련된 컴포넌트로써 JMenu, JMenuBar, JMenuItem, JCheckBoxItem 등이 있다.

- 메뉴 관련 컴포넌트 상속 구조



5.3.1 JMenuBar

JMenu가 올라가는 메뉴 바

5.3.2 JMenu

메뉴버튼를 눌렀을 때 서브 메뉴가 존재할 경우 사용하는 컴포넌트

5.3.3 JMenuItem

메뉴버튼을 눌렀을 때 바로 실으로 연결되는 경우 사용하는 컴포넌트

5.3.4 스윙 메뉴 생성

JMenuItem을 생성하고 JMenu에 추가한 후, JMenu를 JMenuBar에 추가하여 프레임에 추가한다

스윙 프로그램의 메뉴는 프레임의 North에 배치하는 것이 아니라 메뉴가 들어갈 레이아웃이 따로 존재한다

- 메뉴 추가 방법

```
프레임.setJMenuBar(JMenuBar);
```

- 간단한 메뉴 만들기 예제

swingGUI.MenuEx.java

```
01  package swingGUI;
02
03  import java.awt.event.InputEvent;
04
05  import javax.swing.JFrame;
06  import javax.swing.JMenu;
07  import javax.swing.JMenuBar;
08  import javax.swing.JMenuItem;
09  import javax.swing.JScrollPane;
10  import javax.swing.JTextArea;
11  import javax.swing.KeyStroke;
12
13  public class MenuEx extends JFrame {
14     JTextArea textArea = new JTextArea("안녕하세요!", 5, 10); // 스크롤 바 없는 텍스트영역
15     JScrollPane jp = new JScrollPane(textArea); // textArea에 스크롤바 생성
16
17     // 메뉴 컴포넌트 생성
18     JMenuBar menuBar = new JMenuBar();
19     JMenu fileMenu = new JMenu("파일");
20     JMenu viewMenu = new JMenu("보기");
21     JMenu helpMenu = new JMenu("Help");
22
23     public MenuEx() {
24        setTitle("메뉴 만들기");
25        setSize(300, 300);
26        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27
```

```
28        // 파일 메뉴(fileMenu) 생성
29        fileMenu.add(new JMenuItem("새 파일"));
30        // 단축키 설정
31        fileMenu.getItem(0).setAccelerator(KeyStroke.getKeyStroke('N', InputEvent.CTRL_MASK));
32        fileMenu.add(new JMenuItem("열기"));
33        fileMenu.add(new JMenuItem("저장"));
34        fileMenu.addSeparator(); // 구분선 추가
35        fileMenu.add(new JMenuItem("종료"));
36
37        // 메뉴를 메뉴바에 등록
38        menuBar.add(fileMenu);
39        menuBar.add(viewMenu);
40        menuBar.add(helpMenu);
41
42        // 메뉴바 등록
43        setJMenuBar(menuBar);
44
45        // textArea 등록
46        add(jp, "Center");
47
48        setVisible(true);
49    }
50 }
```

swingGUI.Main.java

```
01 package swingGUI;
02
03 import swingGUI.MenuEx;
04
05 public class Main {
06    public static void main(String[] args) {
07        new MenuEx();
08    }
09 }
```

,

test.table.SimpleTableTest.java

```
01  package test.table;
02
03  import java.awt.Dimension;
04
05  import javax.swing.DefaultCellEditor;
06  import javax.swing.JComboBox;
07  import javax.swing.JFrame;
08  import javax.swing.JOptionPane;
09  import javax.swing.JScrollPane;
10  import javax.swing.JTable;
11  import javax.swing.event.TableModelEvent;
12  import javax.swing.event.TableModelListener;
13  import javax.swing.table.TableColumn;
14  import javax.swing.table.TableModel;
15
16  public class SimpleTableTest extends JFrame
17                                  implements TableModelListener{
18
19          public SimpleTableTest(){
20                  String[] columnNames = {"이름", "주소", "나이", "가입여부"};
21
22                  Object[][] data = {   {"김철수", "서울", new Integer(24), new Boolean(false)},
23                                        {"김영희", "부산", new Integer(27), new Boolean(true)},
24                                        {"이영철", "천안", new Integer(32), new Boolean(false)} };
25
26                  this.setTitle("JTable test");
27                  this.setSize(500, 200);
28                  this.setDefaultCloseOperation(this.EXIT_ON_CLOSE);
29
30                  final JTable table = new JTable(data, columnNames);
31                  table.setPreferredScrollableViewportSize(
32                                        new Dimension(500, 150));
33                  table.setFillsViewportHeight(true);
34
35                  //셀이 변경되면 이벤트 작동 처리
36                  table.getModel().addTableModelListener(this);
37                  //컬럼 해더를 클릭하면 자동 정렬 처리
38                  table.setAutoCreateRowSorter(true);
```

```
39
40                  //주소를 입력할 때 지정된 도시만 입력이 가능하도록 편집기 설정
41                  TableColumn cityColumn =
42                          table.getColumnModel().getColumn(1);
43                  JComboBox comboBox = new JComboBox();
44                  comboBox.addItem("서울");
45                  comboBox.addItem("부산");
46                  comboBox.addItem("광주");
47                  comboBox.addItem("대구");
48                  comboBox.addItem("대전");
49                  comboBox.addItem("천안");
50                  cityColumn.setCellEditor(new DefaultCellEditor(comboBox));
51
52                  //스크롤판에 테이블 추가
53                  JScrollPane scrollPane = new JScrollPane(table);
54
55                  //프레임에 스크롤판 추가
56                  this.add(scrollPane);
57
58                  this.setVisible(true);
59          }
60
61          @Override
62          public void tableChanged(TableModelEvent event) {
63                  // 테이블의 내용이 변경되었을 때 작동됨
64                  int row = event.getFirstRow();
65                  int column = event.getColumn();
66                  if(column == 2){
67                          TableModel model = (TableModel)event.getSource();
68                          String columnName = model.getColumnName(column);
69                          Object data = model.getValueAt(row, column);
70                          String s = (String)data;
71                          if(Integer.parseInt(s) > 100){
72                                  JOptionPane.showMessageDialog(this, "범위를 초과하였습니다." + s,
73                                                  "경고", JOptionPane.WARNING_MESSAGE);
74                          }
75                  }
76          }
77
```

```
78              public static void main(String[] args) {
79                      // JTable test
80                      new SimpleTableTest();
81              }
82      }
```

test.dialog.LoginDialog.java

```
01      package test.dialog;
02
03      import java.awt.*;
04      import java.awt.event.*;
05
06      import javax.swing.*;
07
08      public class LoginDialog extends JDialog
09              implements ActionListener{
10              private JTextField idTF;
11              private JPasswordField pwdTF;
12              private JButton loginBtn, closeBtn;
13              private JPanel centerPan, southPan;
14              private MainFrame owner;
15
16              public LoginDialog(MainFrame owner){
17                      super(owner);
18                      this.owner = owner;
19
20                      this.setModal(true);
21                      this.setBounds(100, 100, 300, 200);
22                      this.setLayout(new BorderLayout());
23
24                      centerPan = new JPanel();
25
26                      idTF = new JTextField(20);
27                      pwdTF = new JPasswordField(20);
28                      centerPan.setLayout(new GridLayout(2, 2));
29                      centerPan.add(new JLabel("아이디 : "));
30                      centerPan.add(idTF);
31                      centerPan.add(new JLabel("암 호 : "));
```

```
32                      centerPan.add(pwdTF);
33
34                      southPan = new JPanel();
35
36                      loginBtn = new JButton("로그인");
37                      closeBtn = new JButton("닫기");
38                      southPan.add(loginBtn);
39                      southPan.add(closeBtn);
40
41                      loginBtn.addActionListener(this);
42                      closeBtn.addActionListener(this);
43
44                      this.add(centerPan, BorderLayout.CENTER);
45                      this.add(southPan, BorderLayout.SOUTH);
46              }
47
48              @Override
49              public void actionPerformed(ActionEvent event) {
50                      // 버튼 클릭시 동작 처리
51                      switch(event.getActionCommand()){
52                      case "닫기":            this.setVisible(false); break;
53                      case "로그인":          String userId = idTF.getText();
54                              String userPwd = new String(pwdTF.getPassword());
55                              System.out.println("아이디 : " + userId + ", 암호 : " + userPwd);
56
57                              if(userId.equals("u1234") == true && userPwd.equals("p1234") == true){
58                                      JOptionPane.showMessageDialog(this, "로그인 성공!");
59                                      this.setVisible(false);
60                                      owner.showUser(userId, userPwd);
61                              }else{
62                                      this.setVisible(false);
63                                      owner.errorDisplay("아이디나 암호가 일치하지 않습니다.");
64                              }
65                      }
66              }
67      }
```

test.dialog.MainFrame.java

```
01      package test.dialog;
02
```

```
03  import java.awt.BorderLayout;
04  import java.awt.event.*;
05
06  import javax.swing.*;
07
08  public class MainFrame extends JFrame{
09          private JTextArea tarea;
10          private LoginDialog loginDlg;
11
12          public MainFrame(){
13                  this.setTitle("test Dialog");
14                  this.setBounds(0, 0, 800, 600);
15                  this.setDefaultCloseOperation(this.DISPOSE_ON_CLOSE);
16
17                  this.setJMenuBar(makeMenuBar());
18                  makeComponents();
19
20                  loginDlg = new LoginDialog(this);
21                  loginDlg.setVisible(true);
22
23                  this.setVisible(true);
24          }
25
26          private void makeComponents() {
27                  // 컴포넌트 객체 생성하고, 이벤트 연결 처리 메소드
28                  tarea = new JTextArea(10, 100);
29                  tarea.setText("텍스트 에리어 영역입니다.");
30                  this.add(tarea, BorderLayout.CENTER);
31
32                  //익명클래스 작성 방식으로 이벤트 연결, 동작처리
33                  tarea.addMouseListener(new MouseAdapter(){
34
35                          @Override
36                          public void mouseClicked(MouseEvent event) {
37                                  // 적용 컴포넌트를 클릭했을 때
38                                  if(event.getButton() == MouseEvent.BUTTON3){
39                                          //텍스트에리어에서 마우스 오른쪽버튼을 눌렀을 때
40                                          showPopup(event.getX(), event.getY());
41                                  }
```

```
42                                  }
43                  });
44          }
45
46          private void showPopup(int x, int y) {
47                  // popup menu 만들기
48                  JPopupMenu popupMenu = new JPopupMenu();
49                  popupMenu.add(new JMenuItem("복사하기"));
50                  popupMenu.add(new JMenuItem("붙여넣기"));
51
52                  popupMenu.show(tarea, x, y);
53          }
54
55          private JMenuBar makeMenuBar() {
56                  // 메인프레임에 셋팅할 메뉴바 만드는 메소드
57                  JMenuBar menuBar = new JMenuBar();
58                  JMenu fileMenu, editMenu, viewMenu, helpMenu;
59
60                  fileMenu = new JMenu("파일(F)");
61                  fileMenu.setMnemonic(KeyEvent.VK_F);
62
63                  editMenu = new JMenu("편집(E)");
64                  editMenu.setMnemonic(KeyEvent.VK_E);
65
66                  viewMenu = new JMenu("보기(V)");
67                  viewMenu.setMnemonic(KeyEvent.VK_V);
68
69                  helpMenu = new JMenu("도움말");
70
71                  //메뉴 아이템 만들기
72                  JMenuItem fileNew = new JMenuItem("새로 만들기");
73                  JMenuItem fileOpen = new JMenuItem("열기...");
74                  JMenuItem fileSave = new JMenuItem("저장");
75                  JMenuItem fileSaveAs = new JMenuItem("다른 이름으로 저장...");
76                  JMenuItem fileExit = new JMenuItem("끝내기(X)");
77                  fileExit.setMnemonic(KeyEvent.VK_X);
78                  fileExit.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_X, ActionEvent.CTRL_MASK));
79
80                  fileExit.addActionListener(new ActionListener(){
```

```
81
82                          @Override
83                          public void actionPerformed(ActionEvent arg0) {
84                                  // 끝내기 메뉴아이템 선택시 작동할 코드 구현
85                                  int result = JOptionPane.showConfirmDialog(getParent(),
86                                                                  "프로그램을 종료하시겠습니까?",
87                                                                  "프로그램 종료",
88                                                                  JOptionPane.YES_NO_OPTION);
89                                  if(result == 0)
90                                          System.exit(0);
91                          }
92                  });
93
94                  fileMenu.add(fileNew);
95                  fileMenu.add(fileOpen);
96                  fileMenu.addSeparator();
97                  fileMenu.add(fileSave);
98                  fileMenu.add(fileSaveAs);
99                  fileMenu.addSeparator();
100                 fileMenu.add(fileExit);
101
102                 ButtonGroup group = new ButtonGroup();
103                 JRadioButtonMenuItem statusShow =
104                         new JRadioButtonMenuItem("상태바 표시");
105                 JRadioButtonMenuItem statusHide =
106                                 new JRadioButtonMenuItem("상태바 숨기기");
107                 statusHide.setSelected(true);
108                 group.add(statusShow);
109                 group.add(statusHide);
110
111                 //checkBox 메뉴 만들기
112                 JCheckBoxMenuItem toolbarToggle =
113                         new JCheckBoxMenuItem("툴바 보기/숨기기");
114                 toolbarToggle.addItemListener(new ItemListener(){
115
116                         @Override
117                         public void itemStateChanged(ItemEvent event) {
118                                 // 메뉴아이템의 상태가 변경되었을 때 작동되는 이벤트임
119                                 JOptionPane.showMessageDialog(getParent(), event.getStateChange());
```

```
120                                     }
121                         });
122
123                         viewMenu.add(statusShow);
124                         viewMenu.add(statusHide);
125                         viewMenu.addSeparator();
126                         viewMenu.add(toolbarToggle);
127
128                         menuBar.add(fileMenu);
129                         menuBar.add(editMenu);
130                         menuBar.add(viewMenu);
131                         menuBar.add(helpMenu);
132
133                         return menuBar;
134             }
135
136             public void showUser(String userId, String userPwd) {
137                         // 로그인 성공시 아이디, 암호 처리용
138                         tarea.setText("로그인 사용자 정보 : " + userId + ", " + userPwd);
139             }
140
141             public void errorDisplay(String message) {
142                         // 로그인 실패시 에러메시지 처리용
143                         JOptionPane.showMessageDialog(this, message);
144             }
145 }
```

test.dialog.Start.java

```
01  package test.dialog;
02
03  public class Start {
04              public static void main(String[] args) {
05                          // test dialog
06                          new MainFrame();
07              }
08  }
```

Chapter 13. 이벤트처리

1 이벤트

이벤트란 사용자가 UI 컴포넌트에 대해 취하는 행위로 인해서 발생하는 하나의 사건을 뜻한다

윈도우 환경의 대부분의 응용프로그램은 이벤트 발생을 기다리다 이벤트에 응답하는 형태로 동작하는 이벤트 처리 기반의 프로그래밍(Event Driven Programming)을 한다. 사용자가 컴포넌트에 액션을 취할 때 발생하는 이벤트를 인식하여 적당한 처리 코드를 실행하도록 하는 것을 이벤트 처리라고 한다.

이벤트처리에 관련된 클래스는 java.awt.event 패키지에 존재하여 다양한 이벤트를 처리하기 위한 인터페이스와 이벤트 클래스를 xxxListener, xxxEvent, xxxAdapter의 형태로 제공한다.

- 이벤트 종류

이벤트 클래스	설명
ActionEvent	컴포넌트가 활성화될 때 발생하는 이벤트
ContainerEvent	컨테이너에 컴포넌트가 추가/삭제될 시 발생하는 이벤트
FocusEvent	컴포넌트에 포커스가 들어왔을 때 발생하는 이벤트
ItemEvent	리스트박스와 같은 컴포넌트에서 목록을 선택할 때 발생하는 이벤트
KeyEvent	키보드 입력에 의해서 발생하는 이벤트
MouseEvent	마우스의 움직임에 의해 발생하는 이벤트
WindowEvent	윈도우 열기/닫기, 활성화와 관련된 이벤트

- 이벤트 Listner와 Adaptor의 차이

이벤트를 처리하기 위해서 우리가 상속 받을 수 있는 인터페이스는 두가지가 있는데, Listener 와 Adapter 가 그 두 가지에 해당한다. 해당 인터페이스의 차이는 간단하게 말해, 제공하는 이벤트 메소드를 전부 오버라이딩하여 사용할 것이냐, 아님 필요한 것만 끌어와서 사용할 것이냐라고 나눌 수 있겠다

1.1 이벤트 처리

- 구현 방법

1. 이벤트를 발생시키는 주체 찾기
2. 이벤트 주체의 해당 이벤트 종류 선택
3. 이벤트 등록
4. 이벤트 처리 핸들러 구현

- 이벤트를 처리하는 방법에는 크게 3가지가 존재한다.
  1. 이벤트에 대한 독립적인 클래스로 작성하는 방법
  2. 내부 클래스로 선언하여 처리하는 방법
  3. 익명클래스를 통해 즉석으로 선언 후 처리하는 방법

각각의 방법에는 선언의 차이, 접근의 용이성의 차이만 있을 뿐 내용은 동일하므로, 하기 소스코드를 통해 사용법을 익히고, 자신의 소스에 맞게 선택하여 작업하면 될 것이다. 그럼 각각의 방법에 차이가 무엇이 있는지 소스 코드를 하나씩 봐가며 확인해 보도록 하겠다.

1.1.1 ActionEvent 처리 예제 1 / 독립적인 클래스로 작성 후 이벤트 처리

이벤트를 독립적인 클래스로 선언 함으로써 이후 다른 클래스의 JButton 객체가 해당 이벤트가 필요할 시 공유할 수 있다는 특징이 있으나, 내부에서만 사용할 경우 접근이 상대적으로 어렵다는 단점이 존재한다

swingGUI.ActionEventEx1.java

```
01  package swingGUI;
02
03  import java.awt.Color;
04  import java.awt.FlowLayout;
05  import java.awt.event.ActionEvent;
06  import java.awt.event.ActionListener;
07  import java.text.ParseException;
08  import javax.swing.*;
09
10  public class ActionEventEx1 extends JFrame{
11              public ActionEventEx1(){
12          this.setTitle("KH JAVA GUI");
13          this.setLayout(new FlowLayout());
14          this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
15
16          JButton btn = new JButton("클릭");
17          MyActionListener listener = new MyActionListener();
18          btn.addActionListener(listener);
19          btn.setBackground(Color.WHITE);
20
21          this.add(btn);
22          setSize(300,150);
23          setResizable(false);
24          setVisible(true);
25      }
```

```
26  }
27   // 이벤트 실행 시 자신이 원하는 액션이 일어나도록 별도 클래스로 작성하였다
28  class MyActionListener implements ActionListener{
29      public void actionPerformed(ActionEvent e) {
30          JButton btn = (JButton)e.getSource();
31          if(btn.getText().equals("클릭")) {
32              btn.setText("클릭하셨습니다.");
33              btn.setBackground(Color.ORANGE);
34          } else {
35              btn.setText("클릭");
36              btn.setBackground(Color.WHITE);
37          }
38      }
39  }
```

swingGUI.Main.java

```
01  package swingGUI;
02
03  import swingGUI.ActionEventEx1;
04
05  public class Main {
06     public static void main(String[] args) {
07        new ActionEventEx1();
08     }
09  }
```

ActionListener 인터페이스를 implements 하여 클래스 자신이 이벤트를 처리하는 구조이다. 인터페이스의 추상메소드를 모두 재정의하여 처리하여야 한다.

1.1.2 ActionEvent 처리 예제 2 / 내부 클래스 선언 후 이벤트 처리

이벤트를 한 클래스의 내부 클래스로 선언하는 방법으로써 같은 클래스 내부에서는 접근이 용이하고, 내부 클래스의 입장에서 바깥에 정의된 객체에 쉽게 접근할 수 있다는 장점을 가지고 있다.

swingGUI.ActionEventEx2.java

```
01  package swingGUI;
02
03  import java.awt.*;
04  import java.awt.event.*;
05  import javax.swing.*;
06
07  public class ActionEventEx2 extends JFrame{
```

```
08              public ActionEventEx2(){
09          this.setTitle("KH JAVA GUI");
10          this.setLayout(new FlowLayout());
11          this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12
13          JButton btn = new JButton("클릭");
14          MyActionListener listener = new MyActionListener();
15         btn.addActionListener(listener);
16         btn.setBackground(Color.WHITE);
17
18          this.add(btn);
19
20          setSize(300,150);
21          setResizable(false);
22          setVisible(true);
23      }   // 이벤트 실행 시 내부 클래스의 액션리스너가 실행된다
24              private class MyActionListener implements ActionListener{
25                  public void actionPerformed(ActionEvent e) {
26                      JButton btn = (JButton)e.getSource();
27                      if(btn.getText().equals("클릭")) {
28                          btn.setText("클릭하셨습니다.");
29                          btn.setBackground(Color.ORANGE);
30                      } else {
31                          btn.setText("클릭");
32                          btn.setBackground(Color.WHITE);
33                      }
34                  }
35              }
36  }
```

swingGUI.Main.java

```
01  package swingGUI;
02
03  import swingGUI.ActionEventEx2;
04
05  public class Main {
06      public static void main(String[] args) {
07          new ActionEventEx2();
08      }
09  }
```

1.1.3 ActionEvent 처리 예제 3 / 익명 클래스로 이벤트 즉석 처리

익명 클래스로 즉석으로 선언할 경우 별도 클래스 이름을 정의할 필요 없이 즉석으로 사용할 수 있다는 간편함이 있지만, 공유의 목적으로 사용할 수없다는 단점이 있다

swingGUI.ActionEventEx3.java

```
01  package swingGUI;
02
03  import java.awt.*;
04  import java.awt.event.ActionEvent;
05  import java.awt.event.ActionListener;
06  import javax.swing.*;
07
08  public class ActionEventEx3 extends JFrame{
09              public ActionEventEx3(){
10          this.setTitle("KH JAVA GUI");
11          this.setLayout(new FlowLayout());
12          this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13
14          JButton btn = new JButton("클릭");
15          }   // 이벤트를 익명 클래스로 JButton 객체에 직접 선언하였다
16          btn.addActionListener(new ActionListener() {
17              public void actionPerformed(ActionEvent e) {
18                      JButton btn = (JButton)e.getSource();
19                      if(btn.getText().equals("클릭")) {
20                          btn.setText("클릭하셨습니다.");
21                          btn.setBackground(Color.ORANGE);
22                      } else {
23                          btn.setText("클릭");
24                          btn.setBackground(Color.WHITE);
25                      }
26                  }
27          });
28          btn.setBackground(Color.WHITE);
29          this.add(btn);
30
31          setSize(300,150);
32          setResizable(false);
33          setVisible(true);
34      }
35  }
```

swingGUI.Main.java	
01	package swingGUI;
02	
03	import swingGUI.ActionEventEx3;
04	
05	public class Main {
06	public static void main(String[] args) {
07	new ActionEventEx3();
08	}
09	}

2 내부 클래스

클래스 내부에 정의된 클래스를 내부클래스(Inner Class == Nasted Class)라고 한다
필드나 메소드와 같이 클래스를 기존 클래스의 내부에 선언하므로써, 코드 복잡성을 줄일 수 있다. 긴밀한 관계를 가지고 있는 클래스를 같이 두면 외부에는 불필요하게 노출할 필요가 없어지고 해당 클래스 간에 쉽게 접근 가능한 장점이 생긴다.

다음은 외부(Outer) 클래스와 내부(Inner) 클래스를 사용한 예이다

```
class OuterClass{
        public OuterClass() {
                System.out.println("외부클래스 입니다.");
                InnerClass in = new InnerClass();
                in.print();
        }
        private class InnerClass{
                public InnerClass() {}
                void print() {
                        System.out.println("내부클래스 입니다.");
                }
        }
}
```

이렇게 작성된 내부 클래스도 하나의 클래스이기 때문에 컴파일 시 Outer클래스명$Inner클래스명.class 으로 컴파일 되는 것을 확인할 수 있다

2.1 내부 클래스의 종류

내부 클래스에는 4가지가 있다.

- 멤버 내부클래스(인스턴스 클래스) : 클래스의 인스턴스 멤버변수와 같이 선언한 클래스
- 지역 내부클래스(로컬 클래스) : 메소드 내에 지역변수처럼 선언한 클래스
- 정적 내부클래스(스태틱 클래스) : 정적 멤버변수와 같이 class 앞에 static 키워드를 붙인 클래스.
- 익명(anonymus) 클래스 : 기본적으로 지역 클래스와 같지만 클래스 이름을 지정하지 않고 생성하면서 실행되게 하는 클래스.

nested.sample.Outer.java

```
01  package nested.sample;
02
03  public class Outer {
04    private int a = 10;
05
06    public class MemberInner {    // 멤버 클래스
07      private int b = 20;
08
09      public void print() {
10        System.out.println("인스턴스 이너 클래스");
11      }
12    }
13
14    public static class StaticInner {     // 정적 이너 클래스
15      private int b = 200;
16      private static int c = 300;
17
18      public static void print() {
19        System.out.println("정적 중첩 클래스");
20      }
21    }
22
23    public void print() {
24      class LocalInner {    //  지역 이너 클래스
25        private int b = 2000;
26
27        public void print() {
28          System.out.println("지역 이너 클래스");
```

```
29          }
30      }
31      System.out.println("Outer 클래스의 메소드");
32
33      LocalInner localInner = new LocalInner();
34      localInner.print();
35    }
36  }
```

test.nasted.InnerClassEx.java

```
01  package test.nasted;
02
03  import nested.sample.*;
04
05  public class InnerClassEx {
06     public static void main(String[] args) {
07        Outer outer = new Outer();
08        outer.print();
09
10        System.out.println("----------------");
11        Outer.MemberInner memberInner = outer.new MemberInner();
12        memberInner.print();
13
14        System.out.println("----------------");
15        Outer.StaticInner.print();
16        System.out.println("정적 중첩 클래스의 정적 멤버 : " + Outer.StaticInner.c);
17     }
18  }
```

```
Outer 클래스의 메소드
지역 중첩 클래스
---------------
멤버 중첩 클래스
---------------
정적 중첩 클래스
정적 중첩 클래스의 정적 멤버 : 300
```

2.2 익명 (Anonymus : 이름없는) 클래스

클래스 선언과 동시에 객체를 생성한다.

오직 하나의 객체만을 생성하며 일회용 클래스로 동작한다.

생성자를 가질 수 없으며, 하나의 클래스를 상속받거나 하나의 인터페이스만을 구현한다.

Anonymous.java

```
01  public interface Anonymous {
02      public void print();
03  }
```

AnonymousEx.java

```
01  public class AnonymousEx {
02      public static void main(String[] args) {
03          Anonymous anonymous1 = new Anonymous() {
04              @Override
05              public void print() {
06                  System.out.println("익명클래스 1");
07              }
08          };
09          anonymous1.print();
10
11          Anonymous anonymous2 = new Anonymous() {
12              @Override
13              public void print() {
14                  System.out.println("익명클래스 2");
15              }
16          };
17          anonymous2.print();
18      }
19  }
```

test.keyevent.KeyEventTest.java

```
01  package test.keyevent;
02
03  import javax.swing.*;
04
05  import java.awt.*;
06  import java.awt.event.*;
07
08  class MyFrameK extends JFrame implements KeyListener { // (1)
09          public MyFrameK() {
10                  setTitle("키보드 이벤트 테스트");
11                  setSize(300, 200);
12                  setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13
14                  JTextField tf = new JTextField(20);
15                  tf.addKeyListener(this); // (2)
16
17                  add(tf);
18                  setVisible(true);
19          }
20
21          public void keyTyped(KeyEvent e) { // (3)
22                  display(e, "KeyTyped ");
23          }
24
25          public void keyPressed(KeyEvent e) {
26                  display(e, "KeyPressed ");
27          }
28
29          public void keyReleased(KeyEvent e) {
30                  display(e, "Key Pressed ");
31          }
32
33          protected void display(KeyEvent e, String s) {
34                  char c = e.getKeyChar();
35                  int keyCode = e.getKeyCode();
36                  String modifiers = e.isAltDown() + " " + e.isControlDown() + " " + e.isShiftDown();
37                  System.out.println(s + " " + c + " " + keyCode + " " + modifiers);
38          }
```

```
39  }
40
41  public class KeyEventTest {
42          public static void main(String[] args) {
43                  MyFrameK f = new MyFrameK();
44          }
45  }
```

Appendix 1. 이클립스 디버깅 사용법

8 디버깅을 이용한 상속관계 및 다형성, collection정보 확인

디버깅을 이용해서 객체에 대한 정보를 더 정확하게 확인 할 수 있다. 예제에서 간단한 Person, Student객체를 만들어 Student객체가 Person을 상속하고 Person 변수에 Student를 보관하는 코드를 작성하겠다. 그리고 추가적으로 ArrayList에 Student객체를 생성해서 3개만 넣어보겠다.

간단히 각 객체에는 멤버변수만 지정하여 사용하겠다.

소스코드

Person 객체

Person.java

```
01  package kh.oop.inherit.dao;
02
03  public class Person {
04    private String name;
05    private int age;
06    private char gender;
07
08
09    public Person() {}
10    public Person(String name, int age, char gender) {
11              this.name=name;
12              this.age=age;
13              this.gender=gender;
14    }
15    public String getName() {
16              return name;
1`7   }
18    public void setName(String name) {
19              this.name = name;
20    }
21    public int getAge() {
22              return age;
23    }
24    public void setAge(int age) {
25              this.age = age;
26    }
27    public char getGender() {
28              return gender;
30    }
```

```
31    public void setGender(char gender) {
32            this.gender = gender;
33    }
34    public String toString() {
35            return name+age+gender;
36    }
37    public void print() {
38            System.out.println("Person");
39    }
40  }
```

Student 객체

Student.java

```
01  package kh.oop.inherit.dao;
02
03  public class Student extends Person{
04
05    private String studentId;
06    private String major;
07    private int grade;
08
09
10    public Student() {System.out.println("Student");}
11    public Student(String studentId, String major, int grade) {
12            this.grade=grade;
13            this.studentId=studentId;
14            this.major=major;
15    }
16
1`7   public Student(String name, int age, char gender,String studentId, String major, int grade) {
18            super(name,age,gender);
19            this.grade=grade;
20            this.studentId=studentId;
21            this.major=major;
22            super.setAge(age);
23    }
24    public String getStudentId() {
25            return studentId;
26    }
```

```
27    public void setStudentId(String studentId) {
28            this.studentId = studentId;
30    }
31    public String getMajor() {
32            return major;
33    }
34    public void setMajor(String major) {
35            this.major = major;
36    }
37    public int getGrade() {
38            return grade;
39    }
40    public void setGrade(int grade) {
41            this.grade = grade;
42    }
43    public String toString() {
44            return super.toString()+studentId+grade+major;
45    }
46  }
```

Main 메소드

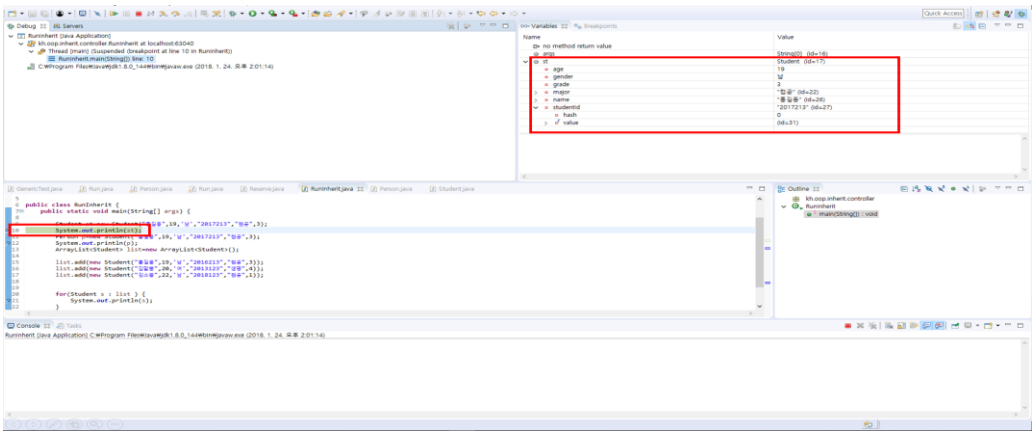
RunInherit.java

```
01  package kh.oop.inherit.controller;
02
03  import kh.oop.inherit.dao.*;
04  import java.util.*;
05
06  public class RunInherit {
07    public static void main(String[] args) {
08
09            Student st=new Student("홍길동",19,'남',"2017213","컴공",3);
10            System.out.println(st);   //Breakpoint 설정
11            Person p= new Student("홍길동",19,'남',"2017213","컴공",3);
12            System.out.println(p);   //Breakpoint 설정
13            ArrayList<Student> list=new ArrayList<Student>();
14
15            list.add(new Student("홍길동",19,'남',"2016213","컴공",3));
16            list.add(new Student("김말똥",20,'여',"2013123","생명",4));
17            list.add(new Student("김소똥",22,'남',"2018123","컴공",1));
18
```

```
19                  System.out.println(list.get(0));
20                  System.out.println(list.get(1));
21                  System.out.println(list.get(2));
22
23                  }
24    }
25  }
```

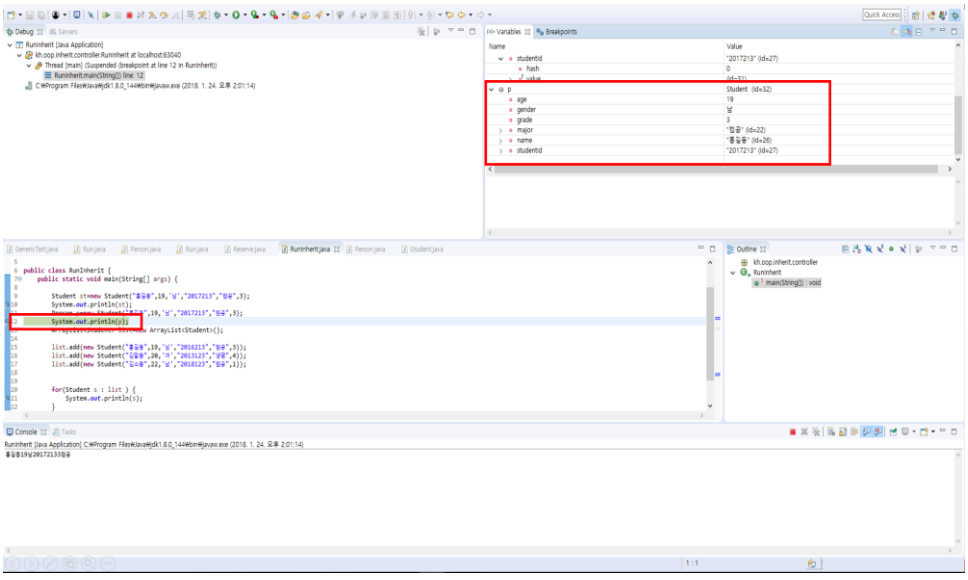
각 객체를 출력해주는 소스코드에 BreakPoint를 설정하고 Debug모드를 실행시켜보겠다.

첫번째 BreakPoint에서 student 객체확인



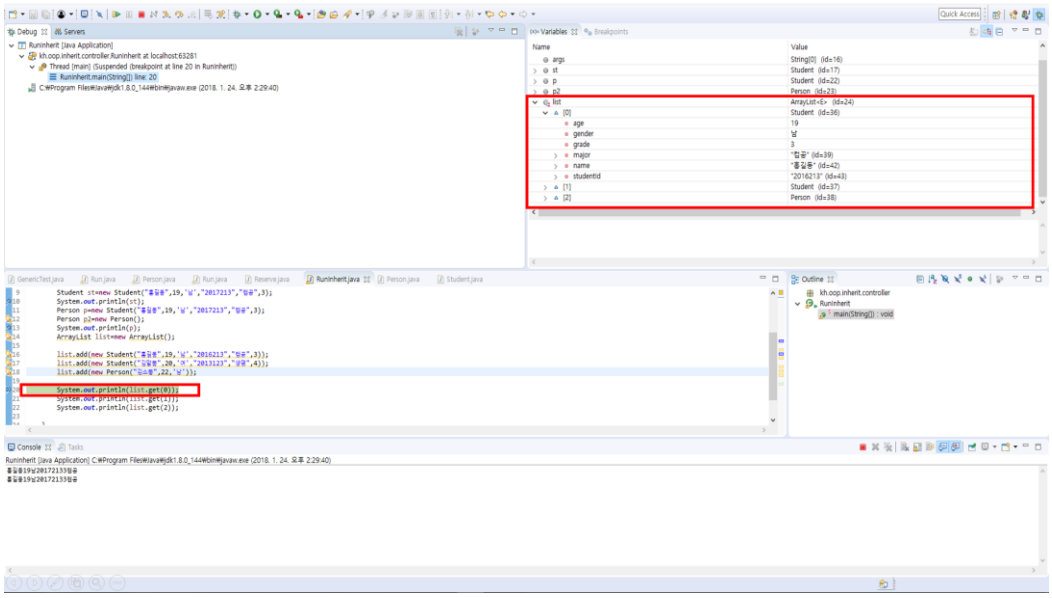
Student객체를 출력하는 코드에 BreakPoint를 설정 프로그램이 그 위치에서 중단된 상태이고 소스코드상에서 st변수에 Student객체가 생성되어 있어 Variables창에서 정보를 확인할 수 있다. Student객체는 Person객체를 상속받았기 때문에 Person이 가지고 있는 멤버변수도 같이 있는 것을 확인할 수 있다. 그리고 value를 보면 실제도 변수가 보관하고 있는 값을 볼 수가 있다.

두번째 BreakPoint에서 Person p 확인



다형성을 이용하여 Person 변수 p에 Student객체가 생성하여 넣었다. 실제로 값은 Student객체가 보관하고 있는 멤버변수를 모두 가지고 있는걸 확인할 수 있다. 하지만 소스코드상에서 현재 자료형이 Person이기 때문에 값을 가지고는 있지만 Person객체가 실제로 가지고 있는 값인 name, age, gender에만 get메소드로 접근할 수 있다. 그리고 이렇게 해당하는 정보를 다 가지고 있기 때문에 형변환을 Student로 하면 저장된 정보가 있기 때문에 모든 정보를 볼수있다는 것을 눈으로 확인 할 수 있다. 그리고 p변수의 자료형은 Person이지만 value값을 보면 Student라고 나오는것도 확인 가능하다.
만약 p변수에 Person객체를 생성해서 대입하면 Person이라는 값이 나올것이다. 이건 각자가 해보자.

세번째 BreakPoint에서 ArrayList 확인



ArrayList list변수에 추가된 객체를 확인 할 수도 있다. Variables창에서 각각의 객체명, 멤버변수들의 값을 확인할 수 있다.

소스코드상에서 에러가 발생하는 경우 변수에 이상한 값이 저장되어 있어서 프로그램이 에러가 발생하는 경우가 많다. 이런것을 위와 같이 디버거를 이용하면 값을 확인하기가 쉽고 에러를 조금더 빨리 찾아낼 수 있다.